

Backlog — OpenCX Block Theme

Navbar: estructura real del dropdown de About (resuelto)

Error previo (corregido). Una versión anterior de este backlog afirmó que `Resources`, `Contact` y `Customer stories` eran items top-level apilados verticalmente en `x=926.5`, `y=123/168/213`, y que se solapaban con `About`. **Esas coordenadas eran inventadas** y provenían de interpretar mal el dump del `.fig`.

Método correcto. El dump original calculaba las cajas aplicando la matriz de transform a los vertices, lo que produce coordenadas negativas y anchos imposibles. Las medidas fiables salen de `node.size` y de las propiedades de auto-layout, leídas directamente del nodo. Con ese método, `Navbar / 1 / (id 10310:1173)` da:

```
Navbar / 1 / 1441x160
└─ Container 1280x44
  └─ Content 157x42 → Company Logo 158x42
  └─ Column (nav) 532x27
    └─ Nav links 532x27
      └─ Link 89x27 "Ecosystem ▼" Manrope 18 (texto 104x27)
      └─ Link 70x27 "Solution" Manrope 18
      └─ Link 96x27 "Industries ▼" Manrope 18
      └─ Link 89x27 "insights" Manrope 18 (texto 67x27)
      └─ Link 51x27 "Nav Link Dropdown"
        └─ "About" 51x27
        └─ Chevron Down 24x24
        └─ Menu 186x167 ← desplegable, hijos de About
          └─ "Resources" Widescreen 16 (81x24)
          └─ "Contact" Widescreen 16 (64x24)
          └─ "Customer stories" Widescreen 16 (135x24)
    └─ Column (acciones) 240x44
      └─ Actions 240x44 → Button 100x44, Button 124x44
```

Conclusiones verificadas:

- Hay **5 items top-level**, no 8. `Ecosystem`, `Solution`, `Industries`, `insights`, `About`.
- `Resources`, `Contact` y `Customer stories` son **hijos del dropdown de About**, dentro del frame `Menu 186x167`. Nunca fueron top-level.
- El header mide 160px y el `Container 44px` ⇒ padding vertical de 58px por lado.
- Botones de 44px de alto: 100x44 y 124x44.
- `Ecosystem` e `Industries` llevan caret (▼) pero **no tienen hijos definidos en ningún punto del archivo**: `Ecosystem` aparece como ítem de navbar una sola vez (10310:1180) e `Industries` una sola vez (10310:1184). Quedan como links simples hasta que se definan sus desplegables.
- `Widescreen 16px` **es una fuente real**, usada 61 veces en el archivo, no un placeholder. WordPress solo muestra caret en items con hijos, así que `Ecosystem` e `Industries` no lo tendrán hasta que existan submenús reales.

Duda sin resolver: los frames `Link` declaran anchos que no coinciden con sus textos (`Ecosystem 89` vs texto 104; `insights 89` vs texto 67), y los gaps derivados del total (532) dan 36px mientras que las posiciones del dump sugieren 26px. Se mantiene 24px; hay que confirmar el gap real con el diseñador. | Alto botones | 44 | 44 / 44 | 0 |

El Δ de 2–4px en los botones viene del ancho de texto (métricas de Manrope en el navegador vs render de Figma), no del layout: el borde derecho y los gaps cuadran exactos.

Bug corregido durante la implementación. El botón dorado heredaba `border: 1px solid black` de `styles.elements.button`; Figma no tiene stroke en ese botón. Se le asignó `border.width: 0` y `padding: 10px` vertical para que ambos botones midan 44px exactos (el botón con borde usa 9px: $24 + 9 + 9 + 2 = 44$).

Navbar responsive (resuelto)

El breakpoint del menú hamburguesa está hardcoded en `wp-includes/blocks/navigation/style.css` (`@media (min-width: 600px)`) y no es configurable por `theme.json`. A 1024 con logo (159px) + 8 links + 2 botones hacen falta ~1267px, o sea **una fila es geométricamente imposible** en ese rango: los links saltaban a 2 filas y el header pasaba de 160px a 217.8px.

Decisión: hamburguesa por debajo de 1200px, fila única por encima.

Implementación: `assets/css/patterns.css` (único CSS custom del theme) restaura el comportamiento overlay de core en el rango 600–1199.98px. Los selectores se prefijan con `.wp-block-navigation` para superar a los de core por especificidad y así no depender del orden de carga de las hojas de estilo:

Selector	theme	core
<code>...__responsive-container:not(.hidden-by-default):not(.is-menu-open)</code>	0,4,0	0,3,0
<code>...__responsive-container-open:not(.always-shown)</code>	0,3,0	0,2,0

Verificado por interacción (no solo por CSS):

Viewport	Botón hamburguesa	Contenedor	Resultado
1440px	<code>display: none</code>	<code>block</code> (inline)	fila única, header 160px
1024px	<code>display: flex</code>	<code>none</code> → <code>flex fixed</code>	abre con los 8 links, header 158.9px
390px	<code>display: flex</code>	<code>none</code> → <code>flex fixed</code>	abre con los 8 links

Pendiente — header en móvil (<600px)

- El header mide **214.4px** a 390px porque el padding raíz de 80px deja solo 230px útiles: no caben hamburguesa (24px) + los dos botones (221.2px) en una fila, así que los botones bajan a una segunda fila. En páginas interiores el logo (159px) empeora el déficit.
- No es corregible con `theme.json`.** Se probó `settings.viewport` + claves `@tablet` / `@mobile` dentro de `styles.spacing.padding` y el resolver las descarta en silencio: los estados responsive del schema solo existen para `styles.elements` y `styles.blocks` (`wp-includes/class-wp-theme-json.php:1306–1332`), no para el `styles` raíz. No volver a intentar esa vía.
- Figma no define frame móvil, así que no hay objetivo de diseño. Candidatos: reducir el padding raíz en móvil, ocultar `Sign In` por debajo de 600px, o subir el logo a su propia fila.

Menú editable (resuelto)

Decisión: Apariencia → Menús es la única fuente de verdad, con un filtro PHP.

Por qué no `__unstableLocation`. El bloque `core/navigation` tiene un atributo `__unstableLocation` que hace exactamente esto, pero todas las funciones que lo respaldan (`block_core_navigation_get_menu_items_at_location()`, `block_core_navigation_get_inner_blocks_from_unstable_location()`, `block_core_navigation_sort_menu_items_by_parent_id()`) están declaradas dentro de `if ( defined( 'IS_GUTENBERG_PLUGIN' ) && IS_GUTENBERG_PLUGIN )` en `wp-includes/blocks/navigation.php:1057–1135`.

Sin el plugin de Gutenberg —que este theme no usa— esas funciones no existen. **No volver a intentar `__unstableLocation`.**

Implementación: `inc/navigation.php`, hook `render_block_data`.

- Se elige `render_block_data` y no `render_block` a propósito: el primero corre **antes** de que `render_block_core_navigation()` construya el markup, así que el bloque conserva su overlay y su Interactivity API nativos. Con `render_block` habría que reescribir el `<nav>` a mano y se perdería el menú hamburguesa.
- Se reconstruye el árbol: `wp_get_nav_menu_items()` devuelve todos los items en un array plano relacionados por `menu_item_parent`, así que se agrupan por padre y se recorre nivel a nivel.
- Un item con hijos se convierte en `core/navigation-submenu`; sin hijos, en `core/navigation-link`. Los desplegables no requieren ninguna configuración extra.
- Gate: solo bloques con la clase `ocx-nav--menu` y sin inner blocks. Un bloque editado a mano en el Site Editor gana sobre el menú.

Verificado:

Comprobación	Resultado
Apariencia → Menús → <code>Primary</code> → location <code>primary</code>	8 items, submenú <code>Ecosystem</code> con 2 hijos
Geometría del header tras el cambio	160px, nav x=80, gap 24px — idéntica
Overlay hamburguesa a 1024/390	abre con los 8 items
Hover sobre <code>Ecosystem</code>	<code>opacity:1</code> , <code>visibility:visible</code> , 2 hijos visibles
Caret <code>wp-block-navigation__submenu-icon</code>	presente (con <code>showSubMenuIcon: true</code>)
DOM	10 labels, 1 solo <code></code> de contenedor, sin duplicación
Avisos/warnings PHP	ninguno

`Platform` e `Integrations` son **datos de prueba** con URLs inventadas, para verificar el mecanismo. Hay que borrarlos o reemplazarlos cuando se definan los hijos reales.

Pendiente — desplegables

- `Ecosystem` e `Industries` llevan caret de desplegable en Figma. El mecanismo ya funciona con cualquier item que tenga hijos en el menú (ver arriba); solo faltan los hijos reales.

Pendiente — logo

- `core/site-logo` está en `parts/header.html` (páginas interiores) y correctamente **ausente** en `parts/header-home.html`, usado por `templates/front-page.html`.
- El theme no tiene aún un logo configurado, así que el bloque no renderiza nada. Hay que copiar el asset desde `OpenCX Wireframes V1.0.fig` (los assets del theme anterior se borraron en el reset).

Pendiente — footer

- La estructura ya está correcta y verificada: `ocx-footer` en `constrained` (recibe el padding raíz), columnas `66.67%` / `33.33%`, separador con `margin-top: 80px` y fila de credits con `space-between`. Alto total 393.8px a 1440px.

- Los 5 social links renderizan con href (facebook.com , instagram.com , x.com , linkedin.com , youtube.com) e incluyen el icono de X correctamente.
- Falta contenido real: las URLs apuntan a dominios desnudos, Link One ... Link Ten son párrafos y no enlaces, y Privacy Policy / Terms of Service / Cookies Settings tampoco son enlaces.
- Falta también la dirección, el teléfono y el email reales (ahora son datos de muestra).

Notas de la extracción Figma

- El dump de Home • Desktop tiene escalas corruptas (anchos negativos en varios nodos), por lo que solo es confiable para tipografía, colores, orden y copy — no para alturas verticales.
- Figma no define frames tablet ni mobile, así que el breakpoint del navbar no tiene objetivo de diseño documentado (ver "Pendiente — navbar responsive").
- La API de Figma sigue devolviendo 429 ; todo se extrajo del .fig local.

Variantes responsive del navbar (investigado)

El archivo contiene **dos componentes de header distintos**, y no deben confundirse:

Componente	Tamaño	Contenido	Es el que usa OpenCX
Navbar / 1 / (10310:1173)	1441x160	5 links Manrope 18 + dropdown About + 2 botones 44px	Sí
Navbar (design system)	1440x72	Solo logo 159x68 + "Already have an account? / Log In" (Roboto 16)	No
Navbar (design system)	1280x72	Solo logo + "Don't have an account? / Sign Up" (Roboto 16)	No
Navbar (design system)	592x72	Solo el logo, sin links ni botones	No
Navbar (design system)	375x64	Solo logo + "Already have an account? / Log In" (Roboto 16)	No

De los 346 frames llamados Navbar , **solo uno** (10310:1173) contiene los links de navegación.

Consecuencia: el navbar de marketing **no tiene variante móvil/tablet en el archivo**. Los frames de 592x72 y 375x64 pertenecen al otro componente (auth header), que no tiene dropdown ni links. El comportamiento responsive del navbar de marketing hay que decidirlo, no deducirlo.

Bug del Figma, no del theme: en el frame de 375x64 el texto "Already have an account?" mide 189px de ancho dentro de un Content de 44px, o sea se desborda. No es un diseño intencional.

Pendiente de decisión (no bloqueante)

- **Gap del navbar:** los frames Link declaran anchos incoherentes con sus textos, así que el gap real no se puede determinar con certeza. Figma sugiere 26px (posiciones) o 36px (total 532 menos 388 de texto). Ahora mismo hay 24px.
- **Submenús de Ecosystem e Industries :** tienen caret en Figma pero ningún hijo definido en todo el archivo. WordPress solo dibuja el caret si hay hijos, así que ahora aparecen sin caret. Hay que crear esos desplegables o quitarlos del diseño.
- **Box del logo:** Navbar / 1 / usa 158x42, mientras el design system usa 159x68 y el asset real es 798x190 (ratio 4.2:1, ninguno de los dos). El SVG solo envuelve el raster con un viewBox 159x68.

Geometría del navbar medida en navegador (Chrome, no calculada)

Con acceso al MCP de Figma y lectura de `getBoundingClientRect()` sobre `http://localhost:8888/sample-page/`. Figma = nodo 10310:1173.

Elemento	Figma x	Medido x	Δ	Figma w	Medido w
Container	80	80	0	1280	1280
Logo	80	80	0	157	159
Nav	454	476	+22	532	478
Actions	1120	1124.5	+4.5	240	235.5
Borde derecho	1360	1360	0		
Gap entre links	26	26.0 / 25.9 / 26.0 / 26.0	0		
Alto header	160	160	0		
Botones	100 y 124	235.5 total		44 alto	44

El nav queda 54px más angosto que en Figma y 22px desplazado a la derecha. La causa es que los frames Link de Figma son internamente inconsistentes (declaran 89px para un texto de 104px), así que el ancho de 532 no es reproducible con los anchos de texto reales.

Padding horizontal: 64px, no 80px

Todas las secciones de Home • Desktop y el Footer / 3 / usan `padding: 80px 64px`, o sea **64px horizontales** → contenido de 1312px. Solo el navbar usa 80px (container 1280 centrado). El footer ya está corregido a 64px con `alignFullWidth`; el padding raíz de 80px se mantiene para que el container del navbar siga siendo 1280.

Obsoleto para el footer. Cuando se reconstruyo Footer / 3 / desde el nodo, el Container paso a 1280 (antes 1312) y las columnas a 816 / 400 en vez de 624 / 624. El padding de la raíz **si** sigue en 80 / 64; el centrado sale de capear el container a 1280. Ver Footer / 3 / – template part compartido reconstruido.

Trampas de CSS de bloques de WordPress (documentadas)

- `core/navigation` y `core/buttons` soportan `style.spacing.blockGap` pero **no** `style.spacing.margin`: el atributo se descarta en silencio. Por eso el offset del nav va en CSS.
- El `className` del bloque de navegación se **copia también al interno**. Un selector `.ocx-nav--menu` coincide con dos elementos y aplica el margen dos veces. Hay que usar `nav.ocx-nav--menu`.
- `styles.css` de nivel superior en `theme.json` se ignora, y `css` anidado en `styles.blocks` se reescribe a `.wp-block-navigation.<clases del bloque>` sin espacio, lo que nunca coincide.
- El `<ul>` de navigation trae `flex-wrap: wrap` de core; como el nav es un flex item que encoge, los 5 links (478px) se comprimían a 435.5px y About saltaba de línea.

Hero de la home: `templates/front-page.html`

Figma Header / 25 / (#10218:66905) + Container (#10218:66906) + Actions (#10218:66911).

Elemento	Figma	Medido	
Padding sección	80 / 64 / 51 / 64	80px 64px 51px 64px	OK
Posición Y	160 (nav) → 240 (box)	160 → 240	OK
Container	1312 x 413, radio 16, borde 1px #D8D8D8	igual	OK
Actions	768 de ancho	768	OK
Logo	147 x 35	147 x 35	OK
Form	768 x 209, radio 15, fondo #FCFBF9	igual	OK
Párrafo	500 de ancho, Manrope 18	500, Manrope 18	OK
Botón Industry	padding 10/24, Widescreen 16	igual	OK
Flecha	24 x 24	24 x 24	OK

El logo mide **147x35**, no 159x68: el rectángulo `#10252:33712` usa el `imageRef 4acb42918f135000`, que es exactamente `logo-opencx.png` (798x190, ratio 4.2 → $147/4.2 = 35$). `custom-logo` y `site-logo` se corrigieron a 147x35.

Más atributos de WordPress que no hacen lo que dicen

- `layout.flexDirection` **no existe** en `core/group`: se ignora en silencio y el grupo queda en `row`. El atributo correcto es `layout.orientation: "vertical"`. Esto ponía el logo y el form uno al lado del otro y hacía que el form encogiera a 550px.
- `styles.spacing.blockGap` genera `:where(.wp-site-blocks) > * { margin-block-start: 20px }`. Figma no usa márgenes entre secciones, solo padding, así que se fijó en `0px`. Antes el hero arrancaba en `y=180` en vez de `y=160`.
- **El hero no aparece en el header de la home en Figma**: `Navbar / 1 /` sí trae logo (`Container` → `Content 157x42` → `Company Logo`). Se mantiene la decisión del usuario de dejar el logo solo en el hero, lo que mueve el nav de `x=444` (con logo) a `x=297` (sin logo).
- `figma_download_figma_images` **ignora** `localPath` y escribe en un directorio por defecto de otro proyecto. Hay que mover el archivo a mano.

Responsive verificado (Chrome, emulación por viewport)

Viewport	hamburguesa	items	filas	header h	overflow
1440	no	5	1	160	no
1200	no	5	1	159.4	no
1024	sí	5	1	158.9	no
390	sí	5	1	214.4	no

Border beam en la caja del prompt: la unica vez que no se sigue a Figma

`10218:70363` dibuja la caja del prompt **plana y sin borde**. Se le agrego un **border beam**: un conic-gradient que rota y del que solo sobrevive un anillo de 1px, con un destello dorado de la marca recorriendo el perímetro. Es el unico lugar del theme donde el nodo queda deliberadamente afuera, y es a pedido del usuario (referencia: el componente `border-beam` de 21st.dev, que es React + Tailwind y se instala con API key de pago; el efecto es CSS puro y se reimplemento en `assets/css/patterns.css` al final de las reglas de `.ocx-hero__field`).

```
@property --ocx-beam-angle { syntax: '<angle>'; initial-value: 0deg; inherits: false; }
```

Tres detalles que no son opcionales:

1. **@property es obligatorio.** Una custom property sin registrar **no interpola**: el gradiente daría un salto de 0deg a 360deg por vuelta en vez de girar. En los navegadores que no lo soportan la declaración `background` entera se descarta en tiempo de computo, así que la caja se queda como esta hoy (sin borde), que es la degradación correcta.
2. **La máscara es el truco.** Se pinta el mismo gradiente negro dos veces, una recortada a `content-box` (la caja de padding menos el `padding: 1px`) y otra sobre toda la caja, y `mask-composite: exclude` resta la primera de la segunda. Eso deja el anillo y nada más. `border-radius: inherit` es lo que hace que el anillo siga las esquinas de 16px.
3. **pointer-events: none es carga real, no cosmética.** Los controles `Industry` y la flecha de envío están posicionados en absoluto dentro de la caja (`top: 137px` y `top: 169px`), y un anillo pintado pero clickeable se comería los clics.

El destello es un **cometa**, no un anillo: encendido 70 grados de los 360 y transparente en el resto, del dorado de marca al vecino más oscuro (`--Color-Husk-Dark` → `--Color-Husk`), así que en cada instante hay alrededor de un quinto del perímetro encendido. Gira en 5s lineales.

Con `prefers-reduced-motion`: reduce la animación se detiene y queda un realce estático al 35% de opacidad, para que la caja se siga leyendo como destacada sin que nada se mueva.

Verificado (Chrome CDP). La caja no cambio de geometría en ningún ancho: 768x209 a 1920 (`x=576`), 1440 (`x=336`) y 1199; 638x209 a 768; 260x244 a 390. El botón `Industry` y la flecha siguen en 24x24, y el overflow sigue en 0 en los cinco. Muestreando el anillo en 8 instantes separados, el punto encendido recorre el perímetro (`x = 128, 10, 322, 460, 398, 190, 10, 290`) mientras `--ocx-beam-angle` avanza ~72 grados por muestra, con entre 62 y 234 píxeles dorados de 1938 muestreados (la variación es del cometa cruzando las esquinas). Hit-testing con `elementFromPoint`: el centro del botón `Industry` sigue siendo el `<a>`, el centro de la flecha sigue siendo el `<img class="ocx-hero__arrow">`, y un clic real de ratón sobre la flecha navega a `/contact/`. Con `prefers-reduced-motion`: reduce emulado, la animación baja a `animation: none` y `opacity: 0.35`. Sin errores de consola.

Intro del logo: del header a la caja del prompt

`assets/js/logo-intro.js`. Al cargar la home, el logo **aparece en el slot del header y vuela a la caja de chat**, y al terminar la página queda exactamente como esta hoy: el logo solo en el hero.

El hallazgo que lo hace barato: **el header ya tenía reservado el lugar del logo**. `.ocx-header__logo` es un group vacío de 157x42 (el "Slot del logo: 157x42 en ambos headers" de más arriba), vacío únicamente porque el Site Editor no tiene un site logo configurado y `core/site-logo` no renderiza nada. Como el slot mide lo mismo que el logo del hero (147x35), **el vuelo es una traslación pura**: sin escalado, sin nada que recalcul.

Slot (origen)	81,59 157x42, logo a 81,63 (izquierda, centrado en vertical)
Destino	647,317 147x35
Técnica	FLIP con Web Animations API: se miden las dos cajas y se anima solo transform

FLIP y no transición CSS porque el logo del header y el del hero viven en ramas distintas del DOM y no hay ningún ancestro común que los conecte.

Es un clon, no el logo real. Es un `img` pelado que apunta al `src` que el hero ya había descargado: verificado por `Network.requestWillBeSent`, `logo-opencx-300x71.png` se pide **una sola vez** con y sin intro. Además el clon se cuelga del slot, que ya es un flex de tamaño fijo con el `align-items: center` del header, así que queda

exactamente donde renderizaria un site logo real (sin CSS de posicionamiento: el unico `.ocx-logo-intro__clone` aporta `width: 147px`, `height: auto` y `pointer-events: none`).

`pointer-events: none` es load-bearing por la misma razon que lo es en el beam: el clon esta **encima** de la fila del header, y un logo pintado pero clicable se comeria el primer clic del nav. Verificado con `elementFromPoint`: en el centro del clon responde el `wp-block-group` de abajo, y en el nav sigue respondiendo `wp-block-navigation-item__label`.

El arco

El keyframe del medio se desplaza **perpendicular a la cuerda**, no a media distancia: `(dy, -dx)` normalizado, con el desplazamiento limitado a `min(28px, longitud * 0.08)`. Limitarlo importa: sin tope, una diagonal larga hace pendular el logo. La escala se calcula igual `(end/start)` aunque hoy sea 1 en los dos ejes, para que un cambio futuro de anchos no estire el logo a medio vuelo.

Secuencia y tiempos

Fase	Duracion	Que pasa
Espera	<code>img.decode()</code> + <code>document.fonts.ready</code> (tope 1200ms)	El logo del hero esta bajo el H1: una web font tardia lo puede mover, y volar a una caja vieja lo deja caer en el lugar equivocado
Aparecer	300ms	<code>opacity 0→1</code> con <code>scale(0.94)→1</code>
Quieto	200ms	Para que el ojo lo registre
Vuelo	1000ms	<code>cubic-bezier(0.4, 0.05, 0.2, 1)</code> , 3 keyframes con el arco en el del medio
Cross-fade	220ms	El clon se disuelve y el logo real vuelve
Flash	150ms + 700ms	El pulso del borde

Total ~2.4s desde la carga. Medido a 1440: `81,63 → 84 → 129 → 283 → 445 → 550 → 605 → 631 → 645 → 647,317`, frenando siempre. A 390 el slot esta en `0,32 157x42` y el vuelo es casi vertical hasta `122,245`; el slot tampoco se mueve.

El flash del borde

`--ocx-beam-glow`, un `@property` de tipo `<length>` con `inherits: true`, hace que **la subida y la bajada sean transiciones de verdad** y no un salto: 120ms hasta el pico y 700ms de vuelta. Medido: `5.7 → 26px → 20 → 13 → 8 → 4 → 0.87 → 0`. El `filter` del anillo vive en CSS detras de `data-ocx-beam-flash` y el atributo se saca al terminar.

El filter NO es permanente a proposito. Un `drop-shadow(0 0 0px ...)` no es del todo un no-op sobre un anillo de 1px con antialiasing: la sombra queda detras de una fuente semitransparente y aclara el pixel mas brillante del cometa. Medido con el cometa pausado: **diferencia maxima de 42/255 en el 0.18% de los pixeles de la caja**. Con el filtro acotado al pulso, la caja en reposo se midio **pixel a pixel contra el beam original: diferencia maxima 0, 0 pixeles distintos de 160.512**.

Del cometa no se toca nada: mismo gradiente, mismo spin, mismo `animation-duration`. Cambiar la duracion en pleno vuelo reiniciaria visiblemente la rotacion.

Repeticion: una vez por sesion

`sessionStorage` , decidido por el usuario. Recargas 2, 3 y 4 verificadas: **no clonan**. Con `prefers-reduced-motion: reduce` : no clonan, `glow=0px` , sin atributo, geometria normal. La bandera se escribe recién cuando la intro se decide a correr, así que un `return` temprano no la consume.

El estado final es el unico que importa

`cleanup()` corre desde un `catch` y desde `pagehide` , y **no depende de que la animacion haya terminado**. El `pagehide` esta a proposito: el `bfcache` congela el DOM en pleno vuelo y los scripts no vuelven a correr, así que sin ese handler una restauracion podia dejar el clon puesto y el logo real invisible. Verificado disparando `pagehide` con `persisted: true` a mitad de vuelo: el clon desaparece y el logo vuelve en el acto. Al volver con `history.back()` : `clone=no` , `heroOp=(limpio)` .

Tres salidas legitimas, ninguna es un error

- no hay logo en el hero (pagina interior, o se borro el bloque);
- no hay slot en el header (una plantilla sin el group);
- **el slot ya tiene un logo real**, que es lo que pasa en cuanto alguien configure un site logo en el Site Editor: la intro pondria un segundo logo al lado del primero, así que se desactiva sola.

Los links: `color:inherit` y quitar el subrayado

El theme tiene `color.link` activo y el enlace salia ya en negro, pero por casualidad: el color venia de la paleta del theme, no del H4. Con `color:inherit` el link toma `--Color-Scheme-1-Text` del H4, que es el token del que esta derivado el shimmer, así que el futuro tema oscuro se lleva a los dos a la vez.

`text-decoration:none` quita el subrayado que WordPress pone a los links y que Figma no pide en este bloque: la separacion visual la hace el borde superior de 1px del H4, y el subrayado compite con el barrido. El anillo de foco del navegador se deja como esta a proposito, porque es la garantia de que el teclado se ve.

Verificado (Chrome CDP)

Geometria **identica con y sin intro**, caja por caja: `header 1440x160` , `row 1279x44` , `slot 157x42` , `nav 455,67 532x27` , `heroLogo 147x35` , `form 768x209` , `frame 1312x413` . En reposo no queda `style inline`, ni atributo, ni animaciones (`anim=0` , `glow=0px`). Consola vacia, ningun request fallido, y la regresion de los cinco anchos identica a la linea base (`docH 9669/9905/8936/9121/11788` , `footerH 582/518/518/756`, overflow 0).

Deltas que quedan

- `parts/header-home.html` sigue teniendo el group vacio. Es correcto: el slot tiene que reservar 157x42 para que la intro pueda clonar ahi, y llenarlo de verdad cambiaria el header en reposo.
- La imagen del clon es la del hero, así que el `alt` se fuerza a "" y el clon lleva `aria-hidden="true"` : es decorativo y duplicaria el logo para un lector de pantalla. El logo real solo se oculta durante el vuelo, ya esta cargado y con `opacity: 0` (no `visibility`), así que el enlazado sigue siendo alcanzable por teclado.

Botones del hero: `Industry` y `arrow_upload_ready`

Ambos se implementaron desde las coordenadas reales de Figma, no desde una normalización.

Elemento	Nodo	Figma	Implementado
Form	10310:1282	768x209	768x209
Botón Industry	10310:1284	x 24.5, y 137	x 24.5, y 137

Elemento	Nodo	Figma	Implementado
Flecha	10310:1285	x 732, y 169, 24x24	x 732, y 169, 24x24

Decisiones tomadas por el usuario:

- **La flecha va en la esquina inferior derecha, literal.** Figma no la pone en una fila con el botón: sus insets rompen el padding de 25px del form (12px a la derecha, 9px abajo) y su centro queda 22px por debajo del centro del botón. Se reprodujeron las coordenadas en vez de "arreglarlas" a una fila, porque el objetivo es llegar al 100% de Figma.
- **La flecha es un botón accesible.** En Figma es un `IMAGE-SVG` suelto y sin acción. Ahora es un `core/button` con `role="button"`, `aria-label="Enviar consulta"`, destino `/contact/` (misma URL que el item de menú Contact) y `outline` visible en `:focus-visible`.

Errores encontrados al medir

- **El caret ▼ estaba mal.** Heredaba Widescreen 16 y navy del label. Figma lo define como texto inline con el estilo `ts1: 12px y #CBBB85`. El tema tiene un preset `gold` pero es `#b69e52`, así que se usó el hex literal para no falsear el design system.
- **position: absolute sin contenedor posicionado.** El form es un `core/group` con `position: static`, así que los offsets se resolvían contra el origen de la página y los dos controles acababan flotando sobre el header. El form necesita `position: relative`.
- **fontSize: var: preset|font-size|body no servía para el botón.** Ese preset es fluido (`clamp(14px, 0.875rem + ((1vw - 3.2px) * 0.357), 18px)`) y resuelve a **18px** en desktop, mientras el estilo de texto de Figma fija **16px**. Con 18px el botón media 50.8px de alto (28.8 de line box + 20 de padding + 2 de borde) en vez de 47.6px. Se fijó `fontSize: "16px"`. El párrafo del form sí mantiene el preset fluido, porque su 18px de Figma viene del design system.
- **left: 732px rompe en móvil.** Por debajo de 1200px el form ya no mide 768px, así que una flecha absoluta a 732px cae fuera de él. Se resolvió con un wrapper `ocx-hero__controls`:
- `>=1200px`: `display: contents` (el wrapper desaparece y los absolutos se resuelven contra el form, que ya es `position: relative`).
- `<1200px`: el wrapper vuelve a ser una fila `space-between` y se resetean los absolutos.

Trampas de CSS de esta implementación

- `.ocx-hero__controls{display: contents}` **pierde contra** `.wp-block-group{display: flex}` del block library: misma especificidad y `patterns.css` se encola sin dependencias, así que el orden no está garantizado. Se resolvió con `.wp-block-group.ocx-hero__controls` (0,2,0).
- Los `position / top / left` viajan en el **estilo inline** del bloque, que solo `!important` puede sobreescribir. El reset responsive de los botones lo necesita.

Verificado en Chrome

Viewport	wrapper	posicion Industry	Flecha	Ambos dentro del form	overflow
1440	contents	absolute x24.5 y137	x732 y169	sí	no
1024	flex	static x25 y74.5	x141.6 y85.7	sí	no
390	flex	static x25 y110.5	x137.7 y120.8	sí	no

Layout del nav del home (spec provisto por el usuario)

```
display: flex; width: 532px; height: 27px; padding-right: 33px;
align-items: flex-start; gap: 26px; flex-shrink: 0;
```

Aplicado a `nav.ocx-nav--menu .wp-block-navigation__container` (el `<ul>` de los links). Las 8 propiedades verificadas por `getComputedStyle` a 1440 y 1200. La caja mide **532x27 exactos** en ambos anchos, los 4 gaps miden 26 exactos, una sola fila, sin overflow.

Datos de Figma que confirman el spec (Navbar / 1 / 10310:1173 , Container 10310:1174):

	Figma	medido
Container	1280x44 en x=80.5	1280 en x=80
Column (nav)	x374, 532x27	532x27
Nav links	532x27	532x27
Link Ecosystem / Solution / Industries / insights / About	89 / 70 / 96 / 89 / 51	90.9 / 69.4 / 83.5 / 66.5 / 63.7
Column (actions)	x1040, 240x44, radio 16	236, alineado a la derecha

El `padding-right:33px` no es decorativo: los gaps dan $89+26+70+26+96+26+89+26+51 = 499$, o sea que los 33px son exactamente el sobrante del frame de 532.

box-sizing: border-box es obligatorio

Sin él la caja medía **565px** (532 de contenido + 33 de padding). En Figma los 532px son el ancho exterior del frame, así que el `width` tiene que ser `border-box` para que el padding vaya dentro.

Diferencias que quedan abiertas

- **Ancho por link no se hardcodea a proposito.** Figma mezcla anchos derivados del texto (Solution 70, Industries 96, About 51) con slots fijos de 89 (Ecosystem e insights, y el texto de insights solo mide 67). Como el menú es editable en Apariencia -> Menús, fijar anchos por link se rompería en cuanto cambie una etiqueta. Las diferencias miden:
- Ecosystem e Industries: ours mas angostos, Figma les dibuja un caret que en WP no existe porque no tienen submenú.
- About: ours 12.7px mas ancho, por el icono de submenú (`showSubmenuIcon`); el frame de Figma no lo tiene.
- **El texto del nav usa el font-size fluido** del design system, así que los anchos cambian entre 1200 (Ecosystem 86.6) y 1440 (90.9). Figma fija 18px. Es el mismo problema que tenía el botón `Industry` ; aca sigue sin resolver a proposito, pendiente de decision.
- **La posicion x del nav en paginas interiores es 444, Figma da 454.5.** Los 10.5px de diferencia son exactamente el ancho del logo: 147 actual contra los 157 del `Content` de Figma.

Hallazgos colaterales

- **Los 8 items del menu devuelven 404** salvo `/` . No existen paginas para ecosystem, solution, industries, insights, about, resources, contact ni customer-stories. El boton `Industry` del hero tambien apunta a `/industries/` , o sea tambien 404.
- `parts/footer.html:7` tiene `wp:site-logo {"width":159}` , valor anterior a decidir el custom logo en 147x35. El footer si renderiza un site-logo, así que al medir el home hay que evitar confundirlo con el logo del header (que el home no tiene).

Layout del container del navbar (spec del usuario)

```
width: 1280px; height: 44px; flex-shrink: 0;
```

Aplicado a `.ocx-header__row`, el group interno de `parts/header.html` y `parts/header-home.html` (antes no tenia clase). Verificado a 1440: la caja mide **1280x44 exactos** con `flex-shrink:0`, tanto en home como en paginas interiores.

Coincide con Figma Container (10310:1174): 1280x44 en x=80.5 dentro del frame de 1440. Los 58px de padding vertical del group exterior dan los 160px de alto del header (58+44+58).

`max-width:100%` hace de guarda: los 1280px solo existen de 1440 para arriba. A 1200 el container mide 1040 (el padding raiz de 80+80 deja 1040 utiles) y sin la guarda se pasaria de la pantalla.

Posiciones de los hijos dentro del container (1440, pagina interior)

	Figma	medido	delta
Container	1280x44	1280x44	0
logo	x0, w157, h42, y1	x0, w147, h34.8, y4.6	10px de ancho
nav	x374, w532, y8.5	x364, w532, y8.5	10px
acciones	x1040, w240, y0	x1044.5, w235.5, y0	4.5px

Los 10px del nav son **exactamente** el delta del logo: Figma separa logo y nav con 217px (157+217 = 374), y el `margin-left:217px` actual ya es ese valor. O sea que el margen no es un hack, es la medida de Figma; lo que corre al nav es que el logo mide 147 en vez de 157. Los 4.5px de las acciones vienen del ancho de los botones, que otra vez arrastran el font-size fluido.

En vertical, nav y8.5 y acciones y0 coinciden con Figma al decimal.

Pendiente: los 217px del home no tienen logo delante

El home usa `header-home.html`, que no tiene `core/site-logo` (decision previa del usuario: el logo va solo en el hero). Pero el `margin-left:217px` sigue aplicando, asi que el nav arranca en x=217 y deja 217px vacios arriba a la izquierda. En interiores el gap lo ocupa el logo; en el home no hay nada que lo ocupe. Opciones: devolver el logo al header del home, poner el nav en x=0, o aceptar el hueco para que el nav quede alineado con los interiores.

Layout completo del navbar (spec del usuario)

```
display: flex; width: 1441px; height: 160px;
padding: 0 var(--Page-Padding-padding-global, 80.5px);
justify-content: center; align-items: center; flex-shrink: 0;
```

Aplicado a `.ocx-header`, que paso a ser el group exterior flex de `parts/header.html` y `parts/header-home.html` (antes era `layout:constrained` con 58px de padding vertical).

El `align-items:center` **reemplaza** el padding de 58px: centrar una fila de 44px dentro de 160px da los mismos 58px, asi que el layout es identico pero expresado sobre el elemento que Figma describe (Navbar / 1 / 10310:1173 = 1441x160). Tambien explica el 1441 = 80.5 + 1280 + 80.5.

El token `--Page-Padding-padding-global` no existe en el theme

Solo aparece en el fallback de `patterns.css`. El theme tiene `styles.spacing.padding` en 80px fijos (no responsivo) y nunca define ese custom property, así que el padding de 80.5px se aplicaba en todos los anchos. A 390px eso dejaba solo 229px útiles: los botones caían a 60px de ancho y sus etiquetas se enrollaban en bloques de 147px de alto.

Solución: el padding de 80.5px y el `height:160px` quedan acotados a `>=1200px`. Por debajo el header vuelve a `height:auto` con el padding raíz del theme, y los botones llevan `white-space:nowrap` para que una etiqueta apretada nunca se parta en varias líneas.

Slot del logo: 157x42 en ambos headers

Content de Figma (10310:1175) es 157x42 en y=1, y son esos 157px los que colocan la columna del nav en x=374 (157 + 217). Con el slot reservado, **los 217px de margin-left dejan de ser un hack**: es la medida de Figma.

- `parts/header.html` : group `.ocx-header__logo` con el `core/site-logo` adentro.
- `parts/header-home.html` : el mismo group **vacio**, que reserva 157x42 sin pintar nada.

core/spacer no renderiza en WordPress 7.1.2

`<!-- wp:spacer {"height":42,"width":157} /-->` esta registrado pero `do_blocks()` devuelve cadena vacía y el bloque se descarta. Un `core/group` vacío sí renderiza. Se usó el group vacío para el slot invisible del home, que además tiene la ventaja de compartir clase con el interior.

Verificado

Viewport	navbar	row	slot	botones	burger	nav	overflow
1440	1440x160 pad 80.5	1279x44	157x42	236x44	no	visible	no
1200	1440x160 pad 80.5	1039x44	157x42	232x43	no	visible	no
1024	auto pad 0	411x43	157x42	230x43	si	overlay	no
768	auto pad 0	407x42	157x42	226x42	si	overlay	no
390	auto pad 0	390x98	157x42	209x98	si	overlay	no

Home e interior miden **exactamente igual** en las cuatro piezas: slot 157x42 en y=1, nav x=374 w=532 y=8.5, container 1279x44, sin overflow.

Deltas que quedan

- **1px**: el container mide 1279 en vez de 1280. El frame de Figma es 1441 (el 80.5 redondeado hacia arriba), pero un viewport de 1440 solo tiene 1440; `max-width:100%` + `border-box` recorta el box en vez de desbordar 1px.
- **3.5px** en las acciones (x1043.5 w235.5 contra x1040 w240): el ancho de los botones sigue arrastrando el font-size fluido.
- **A 390 los dos CTA se apilan** verticalmente (209x98). Es legible y no rompe nada, pero es una decisión de diseño pendiente: en mobile normalmente se muestra un solo CTA o ninguno.

Tokens reales del design system

El usuario exportó las variables de Figma a `Variables/` (formato DTCG). `assets/css/tokens.css` se genera desde ahí y es la única fuente de los nombres reales.

Variables/

Primitives.json	Color (36) + Opacity (17)
Spacing & Sizing/Desktop.tokens.json	Container, Max Width, Page Padding, Section Padding
Spacing & Sizing/Mobile.tokens.json	los mismos tokens, valores de telefono
Typography/Desktop.tokens.json	Text Sizes
Typography/Mobile.tokens.json	los mismos tokens, valores de telefono
UI Styles.json	Radius, Stroke
Color Schemes.json	esquemas semanticos de Figma (no se usan como tokens)

Convencion de nombres: un token en "Page Padding" > "padding-global" se aplana a `--Page-Padding-padding-global` (la ruta del grupo y la clave, unidas con guiones, espacios convertidos en guiones). Es la misma forma que usan los snippets de código de Figma, y de ahí venían los dos nombres que paso el usuario.

Colores (36)

Familia	Tokens
Neutrals	White Neutral-Lightest Neutral-Lighter Neutral-Light Neutral Neutral-Dark Neutral-Darker Neutral-Darkest
Husk	Husk-Lightest Husk-Lighter Husk-Light Husk Husk-Dark Husk-Darker Husk-Darkest
Nile Blue	Nile-Blue-Lightest ... Nile-Blue-Darkest
Voodoo	Voodoo-Lightest ... Voodoo-Darkest
Spring Wood	Spring-Wood-Lightest ... Spring-Wood-Darkest

Todas van como `--Color-<nombre>`. Los pasos son semanticos (Light / Dark), no numericos: el Husk es el paso central de 7 y vale #B69E52, el Husk-Light es #CBBB85.

Slugs de theme.json que no coinciden con el design system

theme.json sigue siendo la paleta de WordPress, pero tres de sus slugs son valores que **no existen en ninguna rampa de Figma**:

Slug de theme.json	Valor	Token real equivalente
gold	#b69e52	<code>--Color-Husk</code> (identico)
bone	#f8f6f0	<code>--Color-Spring-Wood</code> (identico)
off-white	#fcfbf9	no existe
sand	#f2eee1	no existe
navy	#162227	no existe (lo mas cercano es <code>--Color-Nile-Blue-Dark</code> #163A48)

off-white y sand los usa el boton Industry y Sign In; navy lo usa el parrafo del hero y el texto de Sign In. Son los tres valores que habria que reemplazar por tokens reales, pero eso cambia el color visible y queda pendiente de decision.

`--Page-Padding-padding-global` es 64, no 80.5

El export dice **64px** en desktop y **20px** en mobile. Los 80.5px que veníamos usando eran la medida del padding del frame del header de Figma ($1441 = 80.5 + 1280 + 80.5$), no el token global.

Como el header depende de 80.5 para su geometria ($80.5 + 157$ del logo + $217 =$ nav en x374; con 64 la nav caería en x438), se separaron en dos tokens:

- `--Page-Padding-padding-global`: 64px / 20px en mobile, el token real del design system.
- `--Header-padding`: 80.5px , el valor del frame del header, con el porque documentado en el CSS.

Esto es una discrepancia sin resolver entre el design system y el frame, no un error de medicion.

Breakpoint de los modos Desktop/Mobile

Los JSON traen `Desktop.tokens.json` y `Mobile.tokens.json` como archivos separados porque Figma los trata como modos, no como breakpoints. El unico breakpoint que el theme ya declara es 1199.98px (el del nav), y se uso ese para el override mobile de los tokens. Puede que un diseno responsive normal quiera 768 o 1024 en lugar de 1200: queda para ajustar.

Radius y Stroke

Los tres radios son **16px** (`Small` , `Medium` y `Large` tienen el mismo valor), y hay `--Stroke-Border-Width: 0px` mas `--Stroke-Divider-Width: 1px` . El radio del form del hero paso de 15px hardcodeado a `var(--Radius-Medium)` , o sea 16px: coincide con el design system y deja 1px de delta contra el frame, que se dibujó a 15.

Tokens aplicados en el CSS

Selector	Token	Valor
<code>.ocx-header</code>	<code>--Header-padding</code>	80.5px
<code>.ocx-header__row</code>	<code>--Container-container-large</code>	1280px
<code>.ocx-hero__actions</code>	<code>--Container-container-small</code>	768px
<code>.ocx-btn--primary bg</code>	<code>--Color-Husk</code>	#B69E52
<code>.ocx-btn--primary texto</code>	<code>--Color-Spring-Wood</code>	#F8F6F0
form del hero	<code>--Radius-Medium</code>	16px
caret de <code>Industry</code>	<code>--Text-Sizes-Text-Tiny</code> + <code>--Color-Husk-Light</code>	12px / #CBBB85

Botones del header con token real

`parts/header.html` y `parts/header-home.html` dejaron de llevar estilos inline (que ganan a cualquier hoja de estilo) y ahora marcan `ocx-btn--primary` / `ocx-btn--secondary` , con layout y tipografia en CSS:

	Figma	Medido 1440
Let's meet	flex, pad 10/24, gap 8, #B69E52 , texto #F8F6F0 , Manrope 300 16	123.05x44, flex, pad 10 24, gap 8, rgb(182,158,82) , rgb(248,246,240) , Manrope 300 16/24
Sign In	#FCFBF9 , borde 1px #F2EEE1 , texto #162227 , Widescreen 16	99.81x44, rgb(252,251,249) , texto rgb(22,34,39) , Widescreen 16/24
actions	240x44	238.86x44

`Sign In` necesita `line-height:1.5` : con 1.6 el alto daba 45.59 en vez de 44. Manrope 300 se agrego a la URL de Google Fonts (`wght@300;400;...`), que antes solo pedia 400 en adelante.

Verificado

Viewport	padding-global	header pad	row	slot	primary	secondary	overflow
1440	64px	80.5px	1279x44	157x42	123.05x44	99.81x44	no
1200	64px	80.5px	1039x44	157x42	123.05x44	99.81x44	no

Viewport	padding-global	header pad	row	slot	primary	secondary	overflow
390	20px	0px	390x104	157x42	123.05x44	99.81x44	no

El switch de modo mobile se confirma en el padding global (64 -> 20). Sin overflow horizontal en ninguno de los tres.

Hero editable: dos trampas de WordPress que costaron una sesion de depuracion

El hero paso de `patterns/hero-home.php` a `templates/front-page.html` para que el parrafo del campo sea editable en el Site Editor (el contenido de un `wp:pattern` dentro de un `template` no lo es). Eso introdujo dos fallos que no son evidentes y que conviene no repetir.

1. Las plantillas `.html` de un block theme NO son PHP

Un `.php` de `pattern` si se evalua, pero `templates/*.html` se cargan como plantillas de block y **cualquier `<?php ?>` se imprime como texto literal**, sin ejecutarse. El efecto no es un error visible, es markup corrupto: el `<?php` se escapa a `<?php`, el atributo `href` se traga el resto del HTML y el `class="ocx-hero__caret"` del `<span>` se convierte en `aria-hidden=" true"=""`. Por eso `ocx-hero__form` y `ocx-hero__caret` aparecian como `null` en el DOM mientras `curl` si encontraba las clases en el HTML plano.

Resolucion: cero PHP en el template.

Necesidad	Como se resuelve en el template
Logotipo	<code>core/site-logo</code> , que lee el logo de la biblioteca de medios
Flecha de envio	<code></code> <code>root-relative</code>
URLs de menu	<code>href="/industries/"</code> , <code>href="/contact/"</code>

El path `root-relative` hardcodea el slug del theme. Es la practica habitual de los block themes y ya se usaba en el mismo template para los destinos de menu. La alternativa (un bloque dinamico registrado en `inc/`) es correcta si el slug llegara a cambiar.

2. Un `core/button` con el ancla vacia se descarta entero

Al `no_poner` la flecha de envio, el `<a>` quedo sin contenido y el bloque **no llego a la pagina**: `grep -c hero__submit` daba `0` y `document.querySelector('.ocx-hero__submit')` daba `null`. No habia error de PHP, no habia `debug.log`, y los 8 bloques hermanos renderizaban bien.

Consecuencia PRACTica: **no se puede dibujar el icono con un `::before` sobre un ancla vacia**, que era el primer intento. El icono va como `<img class="ocx-hero__arrow">` dentro del ancla. Cuando se depure un bloque que desaparece sin error, contar los delimitadores `<!-- wp:x -->` y comprobar que el HTML de guardado no este vacio.

Verificado tras los cambios (Chrome CDP, / con cache desactivado)

Elemento	Figma	Medido 1440
Form	768 x 209, radio 16, fondo #FCFBF9	768 x 209, radio 16, #FCFBF9
Field	inline-flex, padding 25, gap 12, borde transparente 1px	identico, 768 x 106
Field texto	Manrope 18 / 27, #162227	identico

Elemento	Figma	Medido 1440
Industry	x 24.5, y 137, Manrope 400 18 / 27, #162227	identico, 133.11 x 49
Caret	Manrope 400 10 / 15.5	identico, 6.39 x 15.5
Submit	x 732, y 169, 24 x 24	identico
Logo	147 de ancho, centrado	147 x 34.78, x 310.5

Sin overflow horizontal en 1440, 1200 y 390. En 390 los controles pasan a flow y quedan simetricos (Industry x 25, submit x 211, form 260 de ancho).

Validacion de markup con `parse_blocks()` : 11 bloques en el template, **0 sin registrar**, **0 wp:pattern** (o sea, editable).

Desviaciones conocidas

Ninguna pendiente de decision. Las dos que habia se resolvieron con el usuario:

- **Sign In queda en 46px de alto**, no 44. Es la consecuencia aritmetica del `padding: 10px 24px` pedido (10 + 10 + 1 + 1 + 24 de line-height). Decision del usuario: el padding 10 manda sobre el alto de Figma, asi que no se toco el padding para clavar los 44.
- **El color de la flecha se separo del de Industry**. `Color Schemes.json` define `Color Scheme 1 > Text` como `#000000`, que es el fallback que dio el usuario para el caret, pero el snippet de `Industry` cae a `#162227`. Un solo token custom no puede ser dos valores, asi que:
 - `--Color-Scheme-1-Text: #000000` sigue al export y pinta **solo la flecha**.
 - `--Color-Subnav-Text: #162227` es un token propio para el texto de 18px del hero (`Industry` y el campo del prompt, que comparten color y tamano).
- Ninguna primitiva exportada vale `#162227`, asi que el nombre es acuñado, no exportado. Se documento en `tokens.css` para que quede claro por que existe.

Dropdown: la caja aplicada a cualquier submenu

Spec de la caja, pedida por el usuario para el dropdown de Industries:

```
display: flex; padding: 24px; flex-direction: column; align-items: flex-start; gap: 16px;
position: absolute; right: -12px; bottom: -160px;
border-radius: var(--Radius-Small, 16px);
border: var(--Stroke-Border-Width, 0) solid var(--Color-Scheme-1-Border, rgba(0, 0, 0, 0.15));
background: var(--Color-Scheme-1-Background, #FFF);
```

El requerimiento real no era un panel en el hero sino que **todo submenu agregado en el menu de WordPress adopte el diseño sin tocar codigo**. Por eso la caja se aplica al contenedor de submenu de la navegacion (`.wp-block-navigation__submenu-container`) y no a un elemento hecho a mano. El submenu de Industries (Financial Services, Hospitality, Retail, Education, Healthcare, Manufacturing) y cualquier submenu futuro usan la misma regla.

Dos desviaciones conscientes respecto del snippet

- **No se aplican `right:-12px; bottom:-160px` en el menu**. Esos offsets son la colocacion del control `Industry` del hero. Un item del nav vive en `y=80` dentro de un header de 160px, asi que los mismos offsets llevarian el borde inferior del panel a `y=240` y arrastrarian su parte superior fuera de pantalla. Se conserva la colocacion de core (`top:27px; left:-1px` , debajo del item padre). Los offsets quedan documentados para cuando el control del hero abra un panel.

- **El borde se queda en 0px.** `--Stroke-Border-Width` vale `0px` en `UI Styles.json`, así que `border: 0 solid` es el snippet al pie de la letra, no un estilo faltante. Core sí trae `1px solid rgba(0,0,0,0.15)`, y esta regla lo quita a propósito.

Tokens agregados

- `--Color-Scheme-1-Background: #FFFFFF`, que si viene del export.
- `--Color-Scheme-1-Border: #000000`, también del export, pero es el único token donde el export parece un default sin rellenar: los cuatro esquemas listan `Border` y `Accent` en negro puro, mientras el snippet cae a `rgba(0, 0, 0, 0.15)`. Se sigue el export por consistencia con el resto de `Color Scheme 1` y la diferencia es invisible porque el ancho de borde es 0px. A revisar solo si ese ancho pasa a 1px.

La pelea de especificidad con core

Core llega al mismo elemento con dos reglas que ganan contra un selector razonable:

Regla de core	Props	Especificidad
<code>.wp-block-navigation .has-child .wp-block-navigation__submenu-container</code>	<code>align-items:normal</code>	(0,3,0)
<code>.wp-block-navigation:not(.has-background) .wp-block-navigation__submenu-container</code>	<code>background-color:#fff</code> , <code>border:1px solid</code> <code>rgba(0,0,0,0.15)</code>	(0,3,1)
<code>.wp-block-navigation .has-child .wp-block-navigation__submenu-container > .wp-block-navigation-item > .wp-block-navigation-item__content</code>	<code>padding:0.5em 1em</code>	(0,5,1)

Un selector plano `.ocx-nav--menu .wp-block-navigation__submenu-container` es (0,2,0) y pierde los tres: el panel conservaba `align-items:normal` y el borde de 1px de core, y los links seguían con su padding. Los selectores de este theme replican la forma de los de core con una clase extra (`.wp-block-navigation.ocx-nav--menu ...`) para quedar por encima sin recurrir a `!important`.

También se llevó el padding de los links a 0 y se dejó el espaciado solo en la caja. Mantener el padding de core (0.5em 1em) mas el padding de 24px del panel dejaba las etiquetas a 42px de inset y hacía que el `gap:16px` se leyera como 34px. `align-items:flex-start` es lo que permite que cada fila abraza su etiqueta en vez de estirarse a la más ancha.

Verificado (Chrome CDP, submenu de Industrias abierto con hover)

Propiedad	Spec	Medido 1440
padding	24px	24px
gap	16px	16px
flex-direction	column	column
align-items	flex-start	flex-start
border-radius	16px	16px
border-width	0	0px
background	#FFF	rgb(255,255,255)
Panel	-	200 x 290 (24 + 6x27 + 5x16 + 24)

Mismo resultado en 1440, 1200 y 390, sin overflow horizontal. El panel lo abre el propio core (`data-wp-on--pointerenter` + botón toggle con `aria-expanded`), no hizo falta `core/details` ni JS propio: el hover y el click

ya estan resueltos por la Interactivity API de WordPress.

Pendiente

- El boton **Industry** ▾ del hero sigue siendo un link a `/industries/` y no abre panel. Para que abra la misma caja habria que registrar un bloque dinamico en `inc/` que lea el submenu del menu: una plantilla `.html` no puede ejecutar PHP, asi que no se puede resolver la lista desde `templates/front-page.html`.
- La tipografia de los items mide **18px**, no los 16px que afirma el comentario de "Dropdown children typography" en `patterns.css`: esa regla usa `var(--wp--preset--font-size--body)` y en `theme.json` `body` vale 18px. El `line-height: 1.5` si aplica. Se intento corregir a `var(--Text-Size-Text-Regular)` (16px) y el usuario pidio **dejarlo como estaba**, asi que quedo en 18px. El comentario de la regla ahora avisa de la discrepancia para que no se "arregle" sin preguntar.

Secciones posteriores al hero: `.ocx-section` + modificador

Spec del usuario para las bandas de seccion que van despues del hero:

```
display: flex; width: 1440px;
padding: var(--Section-Padding-padding-section-medium, 80px) var(--Page-Padding-padding-global, 64px);
flex-direction: column; align-items: center; gap: 80px;
background: var(--Color-Scheme-1-Background, #FFF);
```

La caja se separo en una clase base y un modificador de ancho, para que una tercera seccion no repita padding ni fondo:

Clase	Rol
<code>.ocx-section</code>	<code>box-sizing</code> , <code>padding</code> con tokens, fondo con token
<code>.ocx-section--frame</code>	<code>width:1440px</code> ; <code>max-width:100%</code> (la spec que pegaba el usuario)
<code>.ocx-section--stretch</code>	<code>align-self:stretch</code> ; <code>align-items:stretch</code> (el otro, de Figma)

Que vive en el bloque y que vive en el CSS

El markup es un solo `core/group` con cuatro atributos y nada de CSS inline:

```
<!-- wp:group {"alignFullWidth":true,"className":"ocx-section ocx-section--frame","style":
{"spacing":{"blockGap":"80px"}}, "layout":
{"type":"flex","orientation":"vertical","justifyContent":"center"}} -->
```

Propiedad del spec	Donde vive	Por que
<code>display:flex</code>	atributo <code>layout.type:flex</code>	core lo emite, editable en la UI
<code>flex-direction:column</code>	atributo <code>layout.orientation:vertical</code>	idem
<code>align-items:center</code>	atributo <code>layout.justifyContent:center</code>	en un flex vertical, el <code>justify-content</code> de core mapea al eje cruzado
<code>gap:80px</code>	atributo <code>spacing.blockGap</code>	idem
<code>width:1440px</code>	CSS	no existe atributo de ancho en <code>core/group</code>
<code>padding</code> con tokens	CSS	un <code>style.spacing.padding</code> hornea 64px literal y el mobile es 20px

Propiedad del spec	Donde vive	Por que
background con token	CSS	color.background solo puede apuntar a un preset de theme.json , no a --Color-Scheme-1-Background

Poner flex/align/gap tambien en el CSS duplicaria lo que la UI del editor ya escribe, asi que la UI queda como fuente unica de esos tres.

width:1440px es el ancho del frame de Figma, no una decision de diseno, y solo funciona junto a max-width:100% y box-sizing:border-box : sin ellos los 64px de padding horizontal se suman a los 1440 y la caja mide 1568 y desborda cualquier viewport menor a 1440. Es el mismo truco que usa .ocx-header con sus 1441px.

Verificado (Chrome CDP)

Viewport	box	padding	tokens	fondo	overflow
1440	1440x160	80px 64px	80 / 64	#FFF	no
1200	1200x160	80px 64px	80 / 64	#FFF	no
768	768x96	48px 20px	48 / 20	#FFF	no
390	390x96	48px 20px	48 / 20	#FFF	no

display:flex , column , align-items:center y gap:80px se midieron correctos en los cuatro anchos. El Alto es solo el padding porque **la seccion esta vacia**: 80+80 en desktop y 48+48 en mobile. inlineStyle es null , o sea que ningun valor quedo horneado en el HTML.

parse_blocks() : 12 bloques, 0 sin registrar, 0 wp:pattern , 0 tags PHP.

Pendiente

- **La banda de 1440 se elimino del template.** Estaba vacia y costo 160px de desktop / 96px de mobile de blanco entre el hero y "Header / 90 /", que es justo lo que reporto el usuario como "mucha distancia entre ambos". Un core/group vacio no es un spacer: sigue aplicando su padding de token (80/80 desktop, 48/48 mobile) y nada dentro lo justifica. Las clases .ocx-section y .ocx-section--frame siguen en patterns.css , asi que recuperar la banda es volver a agregar un core/group con "className":"ocx-section ocx-section--frame".
- El fondo blanco es el mismo del hero (--Color-Scheme-1-Background y --Color-White valen ambos #FFFFFF), asi que entre el hero y la seccion no hay cambio de color. Si en Figma hay una banda de color distinta entre medio, el token correcto todavia no esta en el export.

Ritmo vertical medido (Chrome CDP)

Los gaps entre bloques hermanos son **0px**: nada de esto viene de margin . Toda la separacion sale de los padding de token de cada banda.

Viewport	header	hero	seccion	footer	blanco hero-fin -> card
1440	160	160-704 (pad 80/51)	704-1504 (pad 80/80)	1504+	131px (antes 291px)
390	106	106-650 (pad 80/51)	650-1386 (pad 48/48)	1386+	99px (antes 195px)

El padding-bottom:51px del hero **no es un error**: viene del atributo style.spacing.padding.bottom del core/group del hero, copiado de su frame de Figma. Igual conviene notar que mixing 51 con los 80/48 del resto deja un ritmo vertical desparejo, porque arriba del contenido del hero hay 80 y abajo 51.

- **Las dos secciones quedan de distinto ancho por encima de 1440.** `--frame` se queda en 1440 y alineada a la izquierda mientras `--stretch` sigue creciendo. Medido a 1920: 1440 vs 1920. No se resolvió con el usuario; si el sitio se va a ver en pantallas anchas conviene unificar en `stretch`.

Figma "Header / 90 /" (#10218:67151)

Frame real de Figma, leído con la API: `fileKey fqldP0lWh73cN5kTnLj3SK`. Es la sección que va **después** de la banda vacía de arriba.

```
"Header / 90 /" alignSelf:stretch, padding 80px 64px, alignItems:stretch, gap 80px, fill #FFFFFF
  "Container"    alignItems:stretch, gap 80px
    "Card"       alignSelf:stretch, padding 0px 64px, justifyContent:center, gap 8px, height 640, fill #4C4C4C, radius 16px
      "Column"   gap 32px, width 736 (fijo)
        "Content" gap 24px
          "AI-powered business results built on human insight, not just technology."
          "OpenCX is the AI operating system that connects your data, your team, and your workflows into one working system, trained on your business, not the open internet."
        "Actions" row, gap 16px
          "Button" Alternate=True, padding 10px 24px, fill #FFF, stroke #FFF 1px, Manrope 500 16px/24px, texto #000, radio 0
```

El spec pegado en el chat no coincide con el frame

El snippet que paso el usuario decía `align-items: center`. El frame de Figma dice `alignItems: stretch` en la raíz y en el Container. Gana Figma: sin `stretch` el Container midió 864px en vez de los 1312px de ancho de contenido, porque `stretch` es justamente lo que impide que el grupo se encoja al `max-content` de su hijo.

`align-items:stretch` **no alcanza: hace falta** `width:100%`

El gotcha real de esta sección. El Column tiene `width:736px` **definido**, y eso se convierte en el piso `min-content` del Card. Medido a 768px, probando una variable `at a time`:

Estado	Section	Container	Card	Column	overflow
base (<code>align-items:stretch</code> en los dos)	768	776	776	736	si
solo Container <code>width:100%</code>	768	728	776	736	si
Container + Card <code>width:100%</code>	768	728	728	688	no

`min-width:0` en el Container y en el Card **no** lo arregla: el `max-width:100%` del Column se resuelve contra un ancho de Card que todavía se está calculando, y Chrome cierra ese ciclo a favor de `max-content`. Darle un `width:100%` definido al Card rompe la circularidad, su content box queda conocido, y el `max-width:100%` del Column por fin tiene contra que resolverse.

El centrado vertical del Card no se puede pedir por atributo

Para un `wp:group` vertical, core compila `layout.justifyContent` a `align-items`. O sea que `"justifyContent":"center"` centraría el Column en horizontal y lo dejaría pegado arriba. El centrado del eje principal tiene que estar en CSS como `justify-content:center`.

640px es height fijo en Figma, aqui es minHeight

`dimensions.minHeight` es la aproximacion disponible. Un `height` fijo cortaria el texto en cuanto la copia envuelva distinto o el viewport se angoste, que es justo lo que pasa a 390.

La tipografia se engancha por selectores de hijo, no por clase

`.ocx-h90__card .wp-block-heading` y `.ocx-h90__card p` en vez de clases en el heading y el parrafo, para que la copy siga siendo `core/heading` y `core/paragraph` pelados, reescribibles desde el editor sin arrastrar una clase.

55px **no es un token**: `--Text-Sizes-Heading-1` vale 72px desktop / 40px mobile. Los 55px son un override local dentro del propio text style "Heading/H1" de Figma, asi que hay que dejar el literal.

Verificado (Chrome CDP)

Viewport	Section	Container	Card	Column	H1	parrafo	Button	overflow
1440	1440x800	1312x640	1312x640	736	736x165 (3 lineas)	18/27, 2 lineas	127.86x46	no
1200	1200x800	1072x640	1072x640	736	736x165	18/27	127.86x46	no
768	768x736	728x640	728x640	688	688x220 (4 lineas)	16/24	127.86x46	no
390	390x736	350x640	350x640	310	310x385 (7 lineas)	16/24	127.86x46	no

Card: `#4C4C4C`, radio 16px, padding 0 64px (0 20px en mobile), gap 8px, `justify-content:center`, `min-height:640px`. H1: Manrope 500 55px/55px, `letter-spacing -0.01em` (medido -0.55px), `#FFFFFF`. Parrafo: Manrope 400 18px/27px con `--Text-Sizes-Text-Medium` (16/24 en mobile), `#FFFFFF`. Boton: `#FFFFFF` de fondo, borde 1px solid `#FFFFFF`, texto `#000`, radio 0, 10px 24px, gap 8px.

`parse_blocks()` : 21 bloques, 7 unicos, 0 sin registrar, 0 `wp:pattern`, 0 tags PHP.

Desviaciones conscientes y pendientes

- El padding horizontal del Card usa `--Page-Padding-padding-global` (64 -> 20) en vez del literal `0px 64px`, para que siga al breakpoint. Figma solo define el valor desktop.
- Los 55px del H1 se usan en todos los anchos. A 390 el titular ocupa 7 lineas y es lo mas discutible de la seccion. **OJO: esto estaba anotado antes como "Figma no define version mobile de este frame" y era falso** (ver seccion "Mobile del Header / 90 /" mas abajo). El component set `#4174:3432` tiene variante Mobile y define el H1 en 55px tambien, asi que 55 en mobile es lo que pide Figma, no una decision mia.
- El heading se marco como `level:1`. La home no tiene ningun otro heading, asi que si no fuera `h1` la pagina se quedaria sin ninguno; si el hero deberia llevar el `h1`, es un click de editor.
- `align-items:center` del spec original quedo descartado a favor del `stretch` de Figma.

Figma "Layout / 71 /" (#10218:67647)

Tercera seccion, va despues de "Header / 90 /". `fileKey fqldP0lWh73cN5kTnLj3SK`.

```
"Layout / 71 /" alignSelf:stretch, padding 112px 64px, alignItems:stretch, gap 80px, fill #FFFFFF
  "Container"    alignItems:stretch, gap 80px
    "Component"  ROW, alignSelf:stretch, gap 80px
      "Column"   alignItems:stretch, gap 16px (fill)
        H2 "You don't need more tech. You need tech that gets your business."
```

```
"Column" alignItems:stretch, gap 32px (fill)
"Content" alignItems:stretch, gap 32px
P "You already have technology. What you're missing is understanding. ..."
```

El padding es `--Section-Padding-padding-section-large` (112 desktop / **64** mobile), no el medium que ya trae `.ocx-section`. Por eso existe el modificador `.ocx-section--large`, que solo sobreescribe el vertical y deja el horizontal heredado de `--Page-Padding-padding-global`.

Las dos columnas: `flex:1 1 0`

"Component" es una fila y sus dos Column son `sizing:fill`, o sea Figma las reparte en partes iguales: $(1312 - 80 \text{ de gap}) / 2 = \mathbf{616}$ cada una a 1440. `core/group` no tiene atributo de "fill", y dejarlo en `auto` las dimensiona por contenido en vez de repartir la fila, así que `flex:1 1 0` es lo que reproduce el fill. El basis es `0` y no `auto` para que las dos queden iguales sin importar cuanto texto lleve cada una. `min-width:0` por el mismo motivo que en "Header / 90 /".

`align-items:stretch` también es de Figma, y es lo que necesita el H2: con el `flex-start` default de core el h2 se vuelve un flex item shrink-to-fit y se traba en su min-content en vez de llenar la columna y envolver.

El H2 si es un token (a diferencia del 55px de "Header / 90 /")

Heading/H2 de Figma es Manrope 500 52px con -0.01em, y `--Text-Size-Heading-2` es exactamente 52px, así que va con token y no literal. El `line-height:1.2` se deja como ratio para que siga el 52 -> 36 del breakpoint. Único caso de la página donde el text style y el token coinciden.

Verificado (Chrome CDP)

Viewport	Section	padding	Component	col-lead	col-body	H2	parrafo	overflow
1920	1920x348.78	112/64	1792	856	856	52/62.4	18/27	no
1440	1440x411.17	112/64	1312	616	616	52/62.4	18/27	no
1200	1200x473.56	112/64	1072	496	496	52/62.4	18/27	no
768	768x300.75	64/20	728	324	324	36/43.2	16/24	no
390	390x488	64/20	350	135	135	36/43.2	16/24	no

H2 `ls -0.52px` a 52px y `-0.36px` a 36px, `#000`. Parrafo `#000`.

`parse_blocks()` : 28 bloques, 7 unicos, 0 sin registrar, 0 `wp:pattern`, 0 tags PHP.

BUG DE DISEÑO (abierto en Figma): la fila de dos columnas no puede llegar a mobile

Este bug es de Figma, no del theme, y sigue sin resolver en el archivo de diseño.

"Layout / 71 /" no tiene frame mobile. Su "Component" es una fila de dos columnas `fill` separadas por 80px, y a 390px eso no cierra:

	valor
contenido disponible	350
gap	80
cada columna	135
ancho de la palabra "business." a 36px	162.19

El H2 no entra en su propia columna y se sale 27px. No lo arregla `min-width:0` en el h2: eso solo achicaria la caja y la palabra seguiria desbordando, porque el problema es que no hay ancho, no que falte permiso para desbordar.

Piso geometrico: la palabra necesita 162.19, la columna da $(W - 208) / 2$, asi que hace falta **$W \geq 532$** . Por encima de eso la fila cierra bien: a 1200 son 496 contra 234, a 768 son 324 contra 162, y a 601 son 240.5 contra 162.

Workaround aplicado (decidido por el usuario, no por diseno): `@media (max-width: 600px)` apila las dos columnas en una sola. El corte va en 600 y no en 532 a proposito: 532 sale de este texto exacto y se moveria en cuanto se edite la copy, mientras que 600 ya es un numero del proyecto (es donde arranca el overlay del nav) y queda del lado seguro.

Lo que hay que llevarle al diseñador: o un frame mobile de "Layout / 71 /" que defina el apilado, o un ancho de columna mayor, o menos gap. Mientras Figma no lo defina, el apilado de 600px para abajo es una conjetura nuestra, no una especificacion.

Dentro del workaround hay que revertir `flex`: en la fila `flex:1 1 0` reparte el ancho, pero al apilarse el `basis 0` pasa a ser la altura y las dos columnas se estirarian a la mitad exacta del contenedor. Por eso el bloque del media query usa `flex:0 1 auto`.

Verificado con y sin el apilado (Chrome CDP)

Viewport	direccion	col-lead	col-body	H2	palabra	H2 dentro de su columna	page overflow
1200	row	496	496	496	234.27	ok	no
768	row	324	324	324	162.19	ok	no
700	row	290	290	290	162.19	ok	no
601	row	240.5	240.5	240.5	162.19	ok	no
600	column	560	560	560	162.19	ok	no
480	column	440	440	440	162.19	ok	no
390	column	350	350	350	162.19	ok	no

601 es el caso de fila mas apretado que queda en pie y entra justo: 240.5 de columna contra 162.19 de palabra. Por abajo de 600 ya va apilado y el titular tiene 350px de ancho, que es lo que necesitaba.

Otras notas

- El `gap:32px` de la Column derecha y el `gap:32px` de "Content" no se ven porque cada una tiene un solo hijo. Se conservaron igual para que el ritmo aparezca en cuanto se agregue un segundo bloque, que es lo que Figma esta describiendo.
- El heading va como `level:2`. "Header / 90 /" ya aporta el `h1` de la pagina.

Segundo frame "Header / 90 /" (#10218:68525)

Mismo nombre y misma estructura que #10218:67151, pero el **Column va en `sizing:fill` con `alignSelf:stretch`** en vez de los 736px fijos. O sea que el Column toma el ancho de contenido completo de la Card: $1312 - 128 = 1184px$ a 1440.

El resto de la caja (Card #4C4C4C, radio 16, 640, `padding 0 64px`, `justify-content:center`, `gap 8`, los gaps 32 y 24, la escala tipografica y el boton) es identico, asi que **se reusan las clases `.ocx-h90__*` en vez de**

duplicarlas. Lo unico que necesita su propio modificador es el ancho del Column:

```
<!-- wp:group {"className":"ocx-h90__column ocx-h90__column--full", ...} -->
```

Bug encontrado: el heading se encogia a max-content

`align-items:stretch` en `.ocx-h90__content` es el valor de Figma y resultaba necesario. Con el `flex-start` default de core, el heading es un flex item shrink-to-fit. En el primer frame no se notaba porque su max-content (1175px) ya superaba los 736px de columna y el ajuste daba lo mismo, pero en este la columna mide 1664 a 1920 y el titular se quedaba en una caja de **1175.14px**.

Es invisible con la copy actual porque el h2 esta alineado a la izquierda, pero en cuanto se edite el texto partira a 1175 en vez de usar los 1664.

Verificado (Chrome CDP), los dos cards a la vez

Viewport	card#1 col	card#1 h	card#2 col	card#2 h	page overflow
1920	736	736	1664	1664	no
1440	736	736	1184	1184	no
1200	736	736	944	944	no
768	688	688	688	688	no
390	310	310	310	310	no

card#1 conserva los 736 fijos y recién se activa el `max-width:100%` por debajo de 864px de viewport, como estaba. card#2 toma siempre el ancho completo.

Card: `min-height:640px`, `justify-content:center`, `#4C4C4C`, `radio 16px`, `padding 0 64px` (0 20px mobile). H2: Manrope 500 55px/55px, `letter-spacing -0.01em` (medido -0.55px), `#FFFFFF`. Parrafo: 18px/27px con `--Text-Size-Text-Medium` (16/24 mobile), `#FFFFFF`. Boton: `#FFFFFF` de fondo, 1px solid `#FFFFFF`, texto `#000`, `radio 0`, 10px 24px.

`parse_blocks()` : 37 bloques, 7 unicos, 0 sin registrar, 0 `wp:pattern`, 0 tags PHP. Jerarquia de headings de la pagina: **h1 x1**, **h2 x2**.

Desviaciones conscientes

- El heading va como `level:2`, aunque el text style de Figma se llama "Heading/H1" y mide 55px. El primer frame quedo como `h1` porque la pagina no tenia ningun heading; este segundo es del mismo nivel visual, asi que dos `h1` en la misma pagina quedarian mal. El nivel de etiqueta es semantico y el tamaño visual lo fija el CSS, que sigue en 55px.
- Los 55px y el resto de las desviaciones del primer frame se repiten aqui: no hay token para 55px y el `height:640px` fijo es `minHeight`. **La nota de "Figma no define mobile" que estaba en este punto tambien era falsa**, ver la seccion siguiente.

Mobile del "Header / 90 /" (#4174:3442) — CORRECCION

Estaba anotado en el BACKLOG que Figma no define mobile para "Header / 90 /". **Es falso**. No se habia porque se consultaban los frames de la home (#10218:67151 y #10218:68525), que son instancias y no traen las variantes. El component set real es #4174:3432 y tiene dos componentes:

- #4174:3433 "Breakpoint = Desktop " (1440)
- #4174:3442 "Breakpoint = Mobile" (375)

La variante Mobile **arrastra el 55px del heading**, así que los 7 líneas a 390 son lo que pide el diseño y no una desviación mía. El resto del delta:

	Desktop #4174:3433	Mobile #4174:3442	Estado
Sección	padding 80px 64px , gap 80	padding 48px 20px , gap 24	ok (tokens)
Container	gap 80	gap 48	irrelevante (un solo hijo)
Card padding	0px 64px	32px (4 lados)	corregido
Card height	fixed 640 , justify-content:center	hug , sin center	corregido
Card radius	16	16	ok
Column gap	32	24	corregido
Content gap	24	20	corregido
Heading	55/55	55/55	ok
Parrafo	Text/Medium/Normal 18/27	Text/Regular/Normal 16/24	ok (token)
Actions gap	16	16	ok
Button radius	0	12px 0px 0px 0px	corregido

Además la variante Mobile trae **dos** botones (Primary Alternate #4179:8942 + Secondary Alternate #4179:8962 , el segundo con fondo `rgba(255,255,255,0.1)` y borde `rgba(255,255,255,0)`). Las instancias de la home usan uno solo, así que el segundo no va, pero queda documentado por si aparece en otra página.

Todo esto está escrito en el `@media (max-width: 1199.98px)` de `patterns.css` . Dos detalles que costaron:

- El `min-height:640px` del Card viene como **estilo inline** del bloque, y el inline le gana a cualquier regla de hoja sin importar la especificidad. Anularlo necesita `!important` , y el selector `[style*="min-height"]` no hacía falta: `.ocx-h90__card { min-height:0 !important }` alcanza. Medido: computa 0px a 768 y 390, y el Card pasa a medir lo que la copy ocupe.
- El radio `12px 0 0 0` del botón Alternate **no** va en `.ocx-btn--invert` , porque ese botón lo comparte el hero y ahí el radio de 12 no aplica. Va scopeado a `.ocx-h90__card` .

Verificado (Chrome CDP) con el mobile puesto

Viewport	Card	pad	min-h	jc	col	der	gap Col	gap Content	radio btn	parrafo	overflow
1440	1312x640	0 64	640	center	736 / 560	512 / 64	32	24	0	18	no
1200	1072x640	0 64	640	center	736 / 560	272 / 64	32	24	0	18	no
768	728x422	32	0	flex-start	664 / 560	32 / 136	24	20	12 0 0 0	16	no
390	350x714	32	0	flex-start	286	32	24	20	12 0 0 0	16	no

A 1440 y 1200 el `der` del card#2 es **64px**, que es el padding del Card: la columna está pegada al borde derecho. En el card#1 queda el vacío de los 736 fijos, como corresponde.

Columna a la derecha del Card — IMPLEMENTADO

El segundo "Header / 90 /" lleva el contenido (H2, parrafo y boton) en una columna de **560px pegada a la derecha**, no repartida en todo el ancho. Se especifico asi:

```
display: flex;
max-width: var(--Max-Width-max-width-medium, 560px);
flex-direction: column;
align-items: flex-start;
gap: 32px;
align-self: stretch;
```

Clase: `.ocx-h90__card .ocx-h90__column--right` (antes `.ocx-h90__column--full`, que se renombro porque "full" ya no describia nada).

Por que `margin-left:auto` y no `flex-direction:row` en el Card

El Card es `mode:"column"` en Figma y sus 640px de alto con `justify-content:center` centran el contenido verticalmente. Pasarlo a `row` habria roto ese centrado. Como el eje transversal de una columna es el horizontal, un `margin-left:auto` pega la columna a la derecha sin tocar el modo: el `align-self:stretch` de Figma se mantiene y el auto margin reparte el espacio libre. La Card sigue centrando vertical igual que el primer frame.

El `margin-left:auto` no funcionaba al principio

Medido: la columna daba `izq 64 / der 688` a 1440, o sea alineada a la izquierda con 688 de vacio al otro lado, y `getComputedStyle` devolvia `margin-left: 0px` pese a que la regla estaba cargando (`max-width` computaba 560 bien).

Causa: core trae `.is-layout-flex > :is(*, div) { margin-left: 0px }`, que es **(0,1,1)** y le gana a un `.ocx-h90__column--right` pelado de **(0,1,0)**. Confirmado con `CSS.getMatchedStylesForNode`: la ultima declaracion que matchea el nodo es esa de core, no la del theme.

Fix: `.ocx-h90__card .ocx-h90__column--right` queda (0,2,0) y gana. `margin-left` computa **624px** a 1440 (= 1184 - 560) y la columna queda a 64px del borde del Card.

Se probó tambien `align-self:flex-end`, que funciona igual, pero se descarto para no apartarse del snippet (`align-self:stretch` es lo que dice Figma).

En mobile la columna vuelve a ocupar todo el ancho

Con el Card en 32px de padding a 375, el ancho de contenido es 311, asi que los 560 del token no aplican y el `margin-left:auto` dejaria la copy despegada del borde. En mobile no hay "derecha" que respetar: se vuelve a `width:100%` y `margin-left:0`. Medido 286 a 390 con `der 32` (el padding).

Copy de la columna

- H2: "AI that learns your business before it acts on it."
- Parrafo: "OpenCX doesn't drop a generic assistant into your company and hope it figures things out. It's trained on your data, your workflows, and the way your team actually works, so what it delivers isn't a guess dressed up as an answer. It's connected to your existing systems and grounded in how your business really operates, which is the difference between a tool that talks and a system that understands."
- Boton: "Let's meet" -> `/lets-meet/`, Alternate `#4179:8942`.

Nota sobre Figma

Esta columna a la derecha no existe en el archivo de Figma. Se reviso #10218:67151 , #10218:68525 , el component set #4174:3432 en sus dos variantes, #10218:68751 y los 12 frames de la home: en todos, el Card tiene un solo hijo Column en sizing.horizontal:fill . El max-width:560px tampoco aparece en ningun nodo. Se implemento igual porque es lo que pide el snippet, pero queda como desviacion respecto de Figma, no como una lectura del archivo.

Figma "Layout / 210 /" (#10218:68751)

Quinta seccion de la home, va despues del segundo "Header / 90 /". Imagen a la izquierda, texto a la derecha.

Layout dado:

```
display: flex;
padding: var(--Section-Padding-padding-section-large, 112px) var(--Page-Padding-padding-global, 64px);
flex-direction: column;
align-items: center;
gap: 80px;
align-self: stretch;
```

Style: background: var(--Color-Scheme-1-Background, #FFF) .

El align-items:center del spec no se aplica. El nodo de Figma dice alignItems:stretch , y con center el unico hijo (Container) se encoge a su max-content en vez de llenar los 1312px de contenido, que es el mismo bug ya documentado en .ocx-section--stretch . Quinta vez que el spec pegado y el frame discrepan; el frame gana.

Lo que si se reuso, para no repetir CSS: el padding 112 -> 64 y 64 -> 20 ya lo hacen .ocx-section--large sobre .ocx-section , y el fondo blanco tambien. La seccion lleva ocx-section ocx-section--large ocx-section--stretch ocx-l210__section .

Estructura

```
Component (row, gap 80, align-items center)
├─ media      (flex 1 1 0)  -> wp:image con placeholder-image.png, radio 16
└─ content    (flex 1 1 0)
    ├── title (gap 16)
    │   └─ title-content (gap 24)  -> h2 + 3 parrafos
    └─ actions (gap 24)           -> 2 botones link
```

flex:1 1 0 con basis 0 en las dos mitades, como en "Layout / 71 /", porque el nodo de Figma las tiene en sizing.horizontal:fill y core no tiene "fill" para un group. A 1440 da $(1312 - 80) / 2 = 616$ para cada una, que es exactamente el ancho natural del PNG (616x640), o sea que el 616 sale del diseno.

Los tres parrafos

Figma los tiene en un solo text node (#10218:68762) con dos saltos de doble linea. Se partieron en tres core/paragraph para que cada uno sea editable por separado, que es el requisito del theme.

Botones: style=Link , no Alternate

#4179:8937 es `Style=Link, Small=False, Alternate=False, Icon position=Trailing`. No es el boton relleno del "Header / 90 /": **sin fondo, sin borde, sin padding**, texto Manrope 500 de 16px en #B69E52 y un `chevron_right` de 24px con 8px de gap. Clase nueva `.ocx-btn--link`.

El `display:flex` en el `<a>` es obligatorio: el markup mete el `<img>` del chevron como hijo del `<a>`, y sin flex el icono caeria en su propia linea.

Bugs encontrados y corregidos

1. **La imagen no llenaba su mitad.** A 1920 `media` daba 856 y la imagen 616, clavada en su ancho natural. Causa: el `<figure>` es flex item de `.ocx-l210__media`, que es un group **vertical**, y core le emite `align-items:flex-start`, asi que el figure se encogia a `max-content`. Fix: `align-items:stretch` en `.ocx-l210__media`.
2. **height:100% en la imagen era un ciclo de resolucion.** `img -> figure -> media -> alto de la fila -> img`. Chrome lo cerraba en favor del alto del texto: a 1200 la fila daba 585 en vez de 640 y la imagen dejaba de aportar su alto. Figma dice `sizing.vertical:fixed, dimensions.height:640`, asi que ahora el alto es el literal 640 y la fila toma `max(640, alto del texto)`, que es lo que hace Figma. A 1440 da exacto 640 porque el texto mide 441.78.
3. **Apilada, la imagen se estiraba a 756.36 en vez de 756.** El `align-self:stretch` del figure pasa a ser el alto del eje transversal en la columna y estira la caja en vez de la imagen. Fix en el `@media: align-self:auto` en figure y media, y `height:auto + aspect-ratio:616/640` en el `img`. A 768 da 728x756.36, que es 728 respetando la proporcion del PNG.

Verificado (Chrome CDP)

Viewport	sec	pad	dir	media	img	content	centrado	H2	parrafo	overflow
1920	1792x640	112 64	row	856x640	856x640	856x414.78	113/113	52/62.4	18/27	no
1440	1312x640	112 64	row	616x640	616x640	616x441.78	99/99	52/62.4	18/27	no
1200	1072x640	112 64	row	496x640	496x640	496x585.17	27/27	52/62.4	18/27	no
768	728x1178.73	64 20	column	728x756.36	728x756.36	728x382.38	-	36/43.2	16/24	no
390	350x925.19	64 20	column	350x363.63	350x363.63	350x521.56	-	36/43.2	16/24	no

Gaps medidos: title 16, title-content 24, content 32, actions 24. Boton: fondo transparente, borde 0 none, padding 0, color `rgb(182, 158, 82)`, gap 8, 16px, icono 24x24. Todo coincide con #4179:8937.

Mobile

Figma no define variante mobile para este frame. No es un component set con variantes como "Header / 90 /", es una instancia de la home, asi que no hay desktop ni mobile que consultar. La seccion resuelve el padding sola por tokens. Lo unico decidido a mano es el apilado, con corte en **1200** y no en el 600 de "Layout / 71 /": alla el problema era que la palabra "business." no entraba en una columna de 135px, y aca no hay tal caso, asi que con partirlo en 600 la mitad de las tablets quedaria con la imagen a la mitad de ancho al lado de un bloque de texto estrecho.

Igual que en `.ocx-l71`, el `flex:1 1 0` hay que revertirlo a `flex:0 1 auto` al apilar, porque en columna el basis 0 pasaria a ser la altura.

Imágenes

`assets/images/placeholder-image.png` (616x640) es **byte-identico** a la imagen que exporta `#10218:68754` (`l210-image.png`, mismo hash, 11421 bytes), asi que se reusa la del theme y no se duplica. Se descargo tambien `chevron-right.svg` (24x24) desde `#4179:8939`.

Ambas imagenes van por ruta del theme en un `<img>` crudo, igual que hace el hero con `icon-arrow-submit.svg`. **Es una desviacion de las practicas de WordPress:** lo normal seria subirlas a la libreria de medios y referenciarlas por `id`, para que el Site Editor las trato como contenido y no como markup. Se dejo asi porque es el precedente que ya tenia el theme, pero queda pendiente migrarlas.

`parse_blocks()` : 52 bloques, 8 unicos, 0 sin registrar, 0 `wp:pattern`, 0 tags PHP. Jerarquia: **h1 x1, h2 x3**. 6 botones, 1 imagen.

Layout / 121 / — "Built on human insight" + timeline de 12 items

Nodo `#10218:70451`, el frame #10 de la home. Va antes del footer, despues de `Layout / 210 /`.

Va en dos columnas `fill` con 80px de gap: la de texto (tagline, H2, boton) y el timeline. Fondo blanco y `padding 112px 64px` (tokens, 64/20 en mobile), o sea el mismo marco que `Layout / 210 /`, asi que reusa `.ocx-section--large` sobre `.ocx-section` en vez de repetir el padding.

El nodo de Figma trae `alignItems:stretch` en el frame. El snippet que se paso con la seccion decia `align-items:center`; **gana Figma**, como ya paso en otras secciones.

Semantica

Figma usa "Heading/H5" para los titulos de item y "Heading/H6" para los de grupo, un salto que no existe en ningun otro lado de la home. Como **H5** ya es el nivel del H2 de esta seccion, los titulos de item pasaron a **H4** y los de grupo a **H3**, para que la pagina quede en `h1 x1, h2 x4, h3 x2, h4 x10` sin holes. Los estilos no cambian: cada clase CSS apunta al rol que le toco, no al nivel que pedia Figma.

El item de grupo es un `core/heading` suelto, sin icono ni divisor, con el subrayado de Figma (`ts1` con `textDecoration:UNDERLINE` dentro del mismo text node) resuelto como `text-decoration:underline` en el CSS.

1. El timeline se comia la mitad del ancho. A 1440 la columna de texto daba 58.16px en

vez de 616: el H2 de 52px partido en un caracter por linea y el boton colgando. Causa: el `flex:1 1 0` estaba solo en la columna de texto. El timeline se quedaba en el `flex:0 1 auto` de core y, con su `min-width:auto` default, crecia hasta el min-content de su item mas largo (1173.84px), o sea el ancho que le hace falta al otro. Fix: `flex:1 1 0 + min-width:0` tambien en `.ocx-l121__timeline`, igual que en las otras columnas del theme.

2. **Los 100px del divisor no se pueden mantener.** Cada item de Figma tiene `sizing.vertical:fixed` con una altura distinta (65, 127, 140, 148, 123, 122, 138, 143...), y esos valores estan medidos sobre el texto de wireframe: son ~1.3x mas altos que lo que el texto real ocupa en Manrope. Con el copy definitivo los items miden **94.8px** (2 lineas) o **118.8px** (3 lineas), asi que un divisor de 100px pediria $48 + 16 + 100 = 164\text{px}$ y cada item con rail se estiraba a 164 aunque su texto terminara en 94.8: media franja de rail colgando bajo el ultimo parrafo de cada item.

Fix: los items van con hug (sin altura fija) y el divisor es `flex:1 1 0` dentro de un `.ocx-l121__marker` con `align-self:stretch`. El divider toma justo el sobrante debajo del icono, así que **llega siempre al borde inferior del item** (medido: `divider.bottom - item.bottom = 0`) y el hueco de 16px del timeline lo separa del icono siguiente. Con 2 líneas son 30.8px de rail, con 3 líneas 54.8px. Es el mismo resultado visual que una línea fija, pero no corta contenido ni deja el rail a media altura si el copy se reescribe en el Site Editor.

Se descarto `min-height:100px` en el divisor a propósito: daba los 164px de vuelta. Y `height:100%` daba 0, porque el `overflow:hidden` que se había puesto de guarda se come el divisor al no tener resolución propia.

3. Figma es inconsistente con el último divisor. "One Vendor, Start to Finish" (último de "Human-Led") no tiene línea, pero "Enterprise Ready" (último de "Built to Be Trusted") sí. Se quito el de "Enterprise Ready": un rail que termina colgando después del último item no conecta con nada.

Layout de la columna derecha (indicado)

El espaciado entre conceptos de la columna derecha va con este layout en `.ocx-l121__timeline`:

```
display: flex;
flex-direction: column;
align-items: flex-start;
gap: 16px;
flex: 1 0 0;
```

Medido: `gap` de 16px entre los 12 items en los cinco anchos, sin overflow, y el alto del timeline sin cambios (1130.34 a 1920, 1250.34 a 1440, 1370.34 a 1200, 1099.88 a 768, 1605.06 a 390).

Del `flex:1 0 0` lo que resuelve el reparto de ancho es el par `grow 1 / basis 0`: a 1440 las dos mitades dan $616 = (1312 - 80) / 2$. El `shrink:0` es lo que evita que el timeline ceda espacio y vuelva al problema del punto 1, donde crecía hasta el min-content de su item más largo y dejaba la columna de texto en 58.16px.

`display:flex` y `flex-direction:column` van explícitos aunque core ya emita `is-layout-flex + is-vertical` en el group, para que la regla no dependa de lo que WordPress convierta a markup.

`align-items:flex-start + gap:16px` es lo que marca el espaciado entre conceptos: el gap del contenedor es el único espacio vertical entre items, y con `align-items:flex-start` cada item se mide contra su propio contenido en vez de estirarse a la altura del más alto de la columna.

Con el componente apilado, el `flex` de las dos mitades se revierte a `0 1 auto` en el `@media` porque el `basis 0` pasaría a ser la altura. Sin ese revert el timeline seguía midiendo bien solo por el `min-height:auto` de flex item, que lo floors contra su contenido: funciona, pero es un mecanismo implícito. El gap de 16 no se toca en mobile.

Separación título de grupo / concepto: 42px

El `gap:16px` del timeline es uniforme para los 12 items, así que el espacio extra va como `margin-top` de 26px sobre el item implicado: **16 (gap) + 26 (margen) = 42**.

Los títulos de grupo se leen como encabezado de su grupo y no como un concepto más de la lista, así que los 3 puntos del timeline donde un título se encuentra con un concepto van a 42:

```
.ocx-l121__group + .ocx-l121__item,
.ocx-l121__item + .ocx-l121__group { margin-top: 26px; }
```

Son dos selectores de hermano inmediato en vez de uno con `:has()`, para no depender de soporte de `:has()`:

Gap	42
"Human-Led" -> "Human-led AI"	concepto que abre un grupo
"One Vendor, Start to Finish" -> "Built to Be Trusted"	concepto que cierra el grupo anterior
"Built to Be Trusted" -> "Own Your AI"	concepto que abre un grupo

Los otros 8 gaps, entre conceptos sueltos, se quedan en 16.

Medido en los cinco anchos, sin overflow: **3 gaps de 42 y 8 de 16** en todos. El alto del timeline sube 26px respecto del estado de 16/16 en los tres puntos, y 78px en total respecto del gap uniforme original: 1208.34 a 1920, 1328.34 a 1440, 1448.34 a 1200, 1177.88 a 768, 1683.06 a 390.

Verificado (Chrome CDP)

Viewport	sec	pad	dir	col	timeline	H2	H3	H4	parrafo	overflow
1920	1354.34	112 64	row	856	856x1130.34	52/62.4	28/39.2	22/30.8	16/24	no
1440	1474.34	112 64	row	616	616x1250.34	52/62.4	28/39.2	22/30.8	16/24	no
1200	1594.34	112 64	row	496	496x1370.34	52/62.4	28/39.2	22/30.8	16/24	no
768	1407.06	64 20	column	728	728x1099.88	36/43.2	20/28	18/25.2	16/24	no
390	1998.63	64 20	column	350	350x1605.06	36/43.2	20/28	18/25.2	16/24	no

La columna de texto mide 656.12 a 768 y 285.57 a 390 (tagline + H2 + boton); el alto de la seccion es el maximo de columna y timeline mas el padding. 0 imagenes rotas y 0 sin cargar en los cinco anchos.

A 1440 las dos mitades dan $616 = (1312 - 80) / 2$, exacto. Tagline 16/24 peso 600 (literal: no hay token de font-weight), boton `rgb(182, 158, 82)` transparente con padding 0, gap 8 e icono 24x24, reusando `.ocx-btn--link` de `Layout / 210 /`.

Geometria del rail a 1440, en los 10 items: divisor centrado bajo el icono (offset 0), `bottom` del divisor contra `bottom` del item = 0, hueco hasta el icono siguiente = 16px. Los 2 items de grupo sin icono ni divisor, `h4` sin subrayado. Icono natural 48x48, 10/10 cargados.

Mobile

Figma no define variante mobile para este frame. Igual que `Layout / 210 /` es una instancia de la home y no un component set, asi que no hay desktop ni mobile que consultar. El padding lo resuelven los tokens; lo unico decidido a mano es el apilado, con corte en **1200**.

El `flex:1 1 0` de las dos mitades se revierte a `flex:0 1 auto` en el `@media`, porque apiladas el basis 0 pasaria a ser la altura y las dos columnas se estirarian a la mitad exacta del contenedor. El gap del item baja de 40 a 24: los 40 de Figma estan medidos a 1440 con el texto en 528 de ancho, y a 390 el texto queda en 278, donde 40 de gap se comen una fraccion mayor del disponible. Es el mismo criterio del workaround de `Layout / 71 /`, pero aca no hay un bug que lo justifique: es una decision de responsive que habria que confirmar contra un frame mobile de Figma.

Imagenes

`assets/images/Relume.svg` (48x48) es el componente `#4179:9441`, el mismo para los 10 items.

Los 10 van por ruta del theme en un `<img>` crudo con `alt=""`, **no** como bloques `core/image`. Es la misma desviacion que arrastra `Layout / 210 /` con `placeholder-image.png`: lo normal seria subirlas a la libreria de medios para que el Site Editor las trate como contenido. Como los diez iconos son identicos y decorativos, diez

bloques de imagen vacios en el editor no aportan nada, asi que se dejo el `<img>` crudo. Queda pendiente migrar imagenes a Media Library en general.

`parse_blocks()` : 116 bloques, 8 unicos, 0 sin registrar, 0 `wp:pattern` , 0 tags PHP. Jerarquia: **h1 x1, h2 x4, h3 x2, h4 x10**. 7 botones, 1 imagen de bloque.

Layout / 59 / — "Not sure where to start?" + parrafo con tramo en negrita

Nodo `#10218:70868` , el frame #11 de la home. Va antes del footer, despues de `Layout / 121 /` .

Mismo marco que las dos secciones anteriores: `padding 112px 64px` (tokens, 64/20 en mobile) y fondo blanco, asi que reusa `.ocx-section--large` sobre `.ocx-section` . Estructura: Container > Component (fila de 80px) > dos columnas `fill` , con `flex:1 1 0 + min-width:0` en cada una por el mismo motivo que en `Layout / 71 /` y `Layout / 121 /` : core no tiene "fill" para un group. A 1440 dan $616 = (1312 - 80) / 2$.

align-items: stretch VS center en el frame

El nodo de Figma trae `alignItems:stretch` y el layout indicado decia `align-items:center` . Gana Figma, pero **aca es indistinto**: el unico hijo del frame es "Container", que trae `alignSelf:stretch` , y el `align-self` del hijo pisa al `align-items` del padre. Con cualquiera de los dos el Container mide el ancho completo, asi que no hay diferencia de render y queda el del archivo. Verificado: `alignItems` computado `stretch` y el Container mide 1312 a 1440.

El desfase de 54px

La columna de texto arranca **54px mas abajo** que la de la izquierda (`padding-top:54px` en `.ocx-l59__column--body`). No es un descuido del wireframe: es lo que hace que el parrafo de la derecha no arranque al mismo alto que el tagline. Como el H2 de la izquierda arranca a 40px del borde (24 del tagline + 16 de gap), el parrafo queda 14px mas abajo que el H2, que es el desfase que se ve en Figma. Va como `padding` de la columna y no como margen del texto para que el bloque siga siendo editable sin arrastrar el desplazamiento.

Los `dimensions.height:316` de "Container" y "Component" en Figma son otro artefacto del wireframe: con el texto real en Manrope el contenido mide 272 a 1440, asi que las cajas van con hug. Se verifica que la mas alta de las dos columnas manda: 218 a 1920, 272 a 1440, 299 a 1200.

El tramo en negrita

El nodo del parrafo trae `boldWeight:700` , que es el `ts1` ("Bold") del final del texto: un tramo en negrita **dentro** del mismo text node, no un estilo del parrafo entero. Se hizo con `<strong>` para que siga siendo rich text editable en el Site Editor, en vez de partir el parrafo en dos bloques que despues hay que volver a joins. El 700 no tiene token de font-weight, asi que es literal.

Desviacion del archivo: Figma tiene el apostrofo recto (ASCII) en el parrafo ("don't", "isn't", "aren't") y el curvo (U+2019) en el boton ("Let's meet"). Se normalizo todo a U+2019 con `’` , que es lo correcto para un sitio en produccion y ademas lo que ya hacen los botones del theme. Si se quiere el ASCII literal del wireframe, son 5 caracteres en `templates/front-page.html` .

Verificado (Chrome CDP)

Viewport	sec	pad	dir	col-texto	col-body	padTop	H2	parrafo	strong	overflow
1920	1920x442	112 64	row	856x164.78	856x218	54	52/62.4 (2l)	18/27 (4l)	700	no

Viewport	sec	pad	dir	col-texto	col-body	padTop	H2	parrafo	strong	overflow
1440	1440x496	112 64	row	616x227.17	616x272	54	52/62.4 (3l)	18/27 (6l)	700	no
1200	1200x523	112 64	row	496x289.56	496x299	54	52/62.4 (4l)	18/27 (7l)	700	no
768	768x446.38	64 20	column	728x126.38	728x152	0	36/43.2 (2l)	16/24 (4l)	700	no
390	390x652.75	64 20	column	350x212.75	350x272	0	36/43.2 (4l)	16/24 (9l)	700	no

Tagline 16/24 peso 600, gap 16 con el H2. Parrafo 18/27 peso 400 con el strong en 700. Gap 32 entre parrafo y boton, 24 entre el tagline y el H2. Boton reusando `.ocx-btn--link: rgb(182, 158, 82)`, fondo transparente, padding 0, gap 8, texto 16/24 peso 500, icono 24x24. El `borderRadius:12px 0 12px 0` de Figma no se ve con fondo transparente, asi que no se replica.

Mobile

Figma no define variante mobile para este frame. Es una instancia de la home y no un component set, asi que no hay desktop ni mobile que consultar. El padding lo resuelven los tokens y el H2 baja a 36px, el parrafo a 16/24, por `--Text-Size-Heading-2` y `--Text-Size-Text-Medium`.

El corte va en **1200** y al apilar hay que quitar el `padding-top:54px` de la columna de texto: su unico proposito es el desfase contra el H2 de la izquierda, que apilada ya no existe. Dejarlo serian 54px de aire muerto arriba de cada bloque. El `flex:1 1 0` de las columnas se revierte a `0 1 auto` porque en columna el basis 0 pasaria a ser la altura.

`parse_blocks()` : 127 bloques, 8 unicos, 0 sin registrar, 0 `wp:pattern`, 0 tags PHP. Jerarquia: **h1 x1, h2 x5, h3 x2, h4 x10**. 8 botones, 1 imagen de bloque, 1 `<strong>`.

Stats / 55 / — H2 + grid de 8 tarjetas (4 filas de 2)

Nodo #10218:71520, el frame #12 y **ultimo** de la home. Va antes del footer, despues de Layout / 59 /. Con esta, las 12 secciones del wireframe estan implementadas.

Mismo marco que las tres anteriores: `padding 112px 64px` (tokens, 64/20 en mobile) y fondo blanco, asi que reusa `.ocx-section--large` sobre `.ocx-section`. `alignItems:stretch` de Figma otra vez contra el `align-items:center` del layout indicado: gana Figma y es indistinto, porque el unico hijo del frame ("Container") trae `alignSelf:stretch` y el `align-self` del hijo pisa al `align-items` del padre. Igual que en Layout / 59 /.

Estructura colapsada

Figma anida el H2 en tres wrappers ("Content" > "Section Title" > "Content"), cada uno con un solo hijo y gaps de 32, 16 y 24 que nunca se ven. Se colapso a un solo group (`.ocx-l55__intro`) con el H2 dentro. Es una desviacion a proposito: tres niveles de anidacion sin efecto visual solo agregan ruido en el Site Editor. Los gaps que si se ven (32 entre el H2 y su columna, 32 entre filas, 32 entre columnas) si se declararon.

Los 609 / 597 no son un gap

Figma pone las dos mitades con `sizing.horizontal:fixed` (609 la izquierda, 597 la derecha) y `justifyContent:space-between` en la fila, o sea que **el gutter no es una propiedad del gap sino el espacio sobrante: 1312 - 609 - 597 = 106px a 1440**.

Se respetan los dos anchos con `flex:0 1 <basis>` en vez del `flex:1 1 0` que usan las otras columnas del theme, porque aqui las mitades no son iguales a proposito y `1 1 0` las mediria 616/616, perdiendo los 12px de

diferencia. El `flex-shrink:1` las deja bajar proporcionalmente cuando el contenedor no entra, y el `gap:32px` es el piso del gutter para que nunca se peguen.

Consecuencia que hay que revisar: como el `flex-grow` es 0, por encima de 1440 las columnas **no crecen** y el gutter se abre. A 1920 el contenedor mide 1792 y el gutter queda en **586px**, con el H2 contra el borde izquierdo y el grid contra el derecho. Si eso no es lo que se quiere, la alternativa es dejar crecer las dos mitades (`flex:1 1 auto` con `width` en px, o `grow` proporcional), pero eso roba el `space-between` de Figma y a 1440 el gutter pasaria de 106 a 0 (solo 32 del `gap`). Queda a decision.

1. El grid se partia en dos lineas entre 1200 y ~1402. Core pone

`flex-wrap:wrap` en `.is-layout-flex`, y con el `basis` en px de las dos mitades ($609 + 597 + 32 = 1238$) el item salta de linea en cuanto el contenedor baja de 1238. A 1200 el H2 y el grid de stats quedaban en dos lineas distintas, **ambos arrancando en x=64**, con 463px de vacio a la derecha del grid.

Lo que lo hace peligroso es que **no se detecta con `scrollWidth`**: no hay desborde, hay un salto de linea, asi que el chequeo de overflow horizontal daba "no" con el layout roto. hace falta comparar la posicion vertical de las dos mitades. Fix: `flex-wrap:nowrap` en `.ocx-l55__content`.

Las otras dos columnas del theme usan `flex:1 1 0`, con `basis` 0, asi que nunca llegan a envolver: por eso el problema es exclusivo de esta seccion. `grep 'flex: 0 1 [0-9]'` sobre `patterns.css` devuelve solo las dos lineas de esta seccion.

- 2. A 390 las tarjetas se median contra su texto.** Con el grid en una sola columna, core le da `align-items:flex-start` a los groups verticales y cada tarjeta se encogia a `max-content`: medidos **254.38, 242.06 y 350** de ancho en la misma fila. Fix: `align-items:stretch` en `.ocx-l55__row`, pero **en la regla base y no en el corte de 600**, asi que el mismo fix cubre las dos cosas. Mismo criterio que en `Layout / 210 /`.
- 3. Las tarjetas de las filas 3 y 4 no estaban alineadas entre si, y no se veia en el `scrollWidth`.** Este es el que defines "para que queden todas alineadas". Como la fila heredaba el `align-items:flex-start` de core, cada tarjeta se media contra su propio texto en vez de contra la mas alta de la fila. Medido a 1440: fila 3 con **219.98 contra 189.19** y fila 4 con **189.19 contra 158.39**. Figma lo resuelve con su default MIN (que es stretch) y lo marca explicito como `alignSelf:stretch` en "#10218:72764".

Con `align-items:stretch` en la regla base, las 4 filas quedan con las tarjetas emparejadas a los 9 anchos, y el ancho de cada tarjeta igual dentro de la fila ($282.5 / 282.5$ a 1440; a 1200 y 1300 aparecen 241.41 contra 241.42, que es el reparto fraccionario de flex).

Las alturas: 219x155 y 219x124 de contenido

Los dos numeros del archivo salen de ahi, y el ancho de 219 es el mismo en las 8 tarjetas: $(597 - 32) / 2 = 282.5$ de caja, y $282.5 - 64$ de padding = **218.5** de contenido, los 219 del archivo al redondear.

- Fila 3, "219x155 con sus 32 a sus 4 lados"** = caja de 219 de alto. "Improve experiences" tiene 5 lineas ($5 \times 30.8 = 153.98$ de texto) y la fila mide **219.98**. "Cut hallucinations" tiene 4 lineas (123.19) y con el stretch queda igual: los 155 del archivo son los 154 del texto mas el rounding del line height.
- Fila 4, "219x124 con sus 32 a sus 4 lados"** = caja de 188 de alto, que es el `dimensions.height:188` de "#10310:911" ("Scale AI without losing what made it work in the first place"), el unico nodo del grid con alto declarado.

Este alto va como `min-height:188px` y no como `height:188px`, y la diferencia no es cosmetica: un `height` explicito **le gana al `align-self:stretch`** de la fila, asi que la tarjeta se queda clavada en 188 mientras la de al lado sigue al texto. Medido con `height:188` contra 189.19 a 1440, y **188 contra 219.98 a 1200 y 1300**, donde "Adapt as your business" pasa a 5 lineas y la fila quedaba visiblemente desapareja. Con `min-height` el 188 queda

como piso y el stretch vuelve a mandar: las dos tarjetas miden lo mismo a todos los anchos, y dan **188 exacto** a 1199, 768 y 600, que son los anchos donde el piso manda (ahi ambas caben en menos de 188).

Ademas coincide con lo que el Site Editor expone: los groups de core solo tienen `minHeight`, no `height`, asi que un `min-height` es ademas la unica forma de que el alto sea editable.

Por debajo de 600 el piso se levanta (`min-height:0`): apilada, una caja de 188 con 2 lineas de texto deja 32 de padding vacio arriba del texto.

Del `height:188px` solo queda el `sizing.vertical:fill` de "#10218:72767" y el `dimensions.height` de "#10310:911" sin replicar como alto fijo, y a proposito: con el stretch de la fila se resuelven solos y ninguna tarjeta queda con el texto cortado.

El numero: Roboto Bold 700 a 80px

Es el **unico nodo de todo el archivo que no usa Manrope**: `style_5de5124f` es Roboto Bold 700, 80px, line height 1.2, centrado. Se consulto y se decidio replicar Figma exacto.

No sumo ninguna request ni peso nuevo: Roboto ya venia cargado en el theme (Google Fonts, `Roboto:wght@400;500;600;700`, registrado en `theme.json` con slug `roboto`). Manrope queda como `font-family` de fallback. Verificado en los cinco anchos: `document.fonts.check('700 80px Roboto')` da `true`.

Los 80px son literales y no el preset `display-sm` de `theme.json` (80px con `fluid.min:48px`), porque ese preset bajaria el numero a 48px en mobile y Figma lo tiene fijo. El 700 tampoco tiene token de `font-weight`.

Count-up en los numeros: `assets/js/stat-count-up.js`

Los cuatro `.ocx-l55__value` (40%, 65%, 3x, 30%) **cuantan hacia arriba** cuando la tarjeta entra al 40% del viewport. Referencia: el componente `count-up` de 21st.dev, que es React + Motion y se instala con API key de pago; el efecto es reimplementado en vanilla, sin dependencias.

El objetivo se parsea del texto del elemento, no de un `data-count-to`. Es la unica fuente de copia que edita el Site Editor: un atributo aparte podria desincronizarse del texto y mentir en silencio, mientras que un fallo de parseo solo significa que no hay animacion. El patron es `/^\d+(?:\.\d+)?(\D*)$/`: un numero inicial, un sufijo que es la unidad, y **solo el punto** como separador decimal, a proposito, porque aceptar la coma leeria 1,200 como 1.2. Un `0` no cuenta nada, y un texto sin numero al principio no se toca.

Lo que agrega, y nada mas:

- cuenta una sola vez, cuando la tarjeta llega al 40% del viewport (`IntersectionObserver` con `threshold:[0.4]`, y se desconecta al disparar);
- **decelera hacia el objetivo en vez de pasarse**: es `easeOutQuint (1 - (1 - p)^5)` y **no** un spring, porque un spring se pasa del objetivo y en una estadística eso muestra 42% un instante en una tarjeta que dice 40%. El numero es lo unico de esta seccion que tiene que ser literalmente cierto;
- conserva la precision del origen, asi que un `3.5x` nunca pasa por `3.4781x`;
- conserva el sufijo, asi que `40%` y `3x` mantienen su unidad;
- el ultimo frame **restaura el texto exacto del origen** desde un `data-ocx-count-source` que se borra al terminar, en vez de dejar un numero reformateado.

Sin JS la seccion esta completa: los numeros son texto real en el HTML, asi que los leen los motores de busqueda y los lectores de pantalla, y el Site Editor sigue editando la copy. Con `prefers-reduced-motion:reduce` (o sin `IntersectionObserver`) el elemento no se toca **ni siquiera le queda el atributo**: el early return va antes de guardar el original.

En CSS, `.ocx-l55__value` gana `font-variant-numeric: tabular-nums`: sin eso cada dígito toma el ancho de su propio glifo y el número "se mueve" de lado mientras sube. El texto es el mismo, solo cambia el ancho de los dígitos.

Verificado (Chrome CDP). Al hacer scroll a la sección los cuatro suben en conjunto y frenan: 9/15/1/7 → 24/39/2/18 → 32/53/2/24 → 37/60/3/28 → 39/63/3/29 → 40/65/3/30, y el atributo temporal desaparece al terminar. **Cero layout shift:** la sección queda en 1440x1177 y las tarjetas en 283x197 / 283x251 antes, durante y después; a 390, 390x1793 con 350x194 / 350x218, overflow 0 en los dos. Casos límite del parser: 3.5x → 1 decimal, 0% no cuenta, 1,200 no se anima, Sin número y vacío no se tocan, 7 y 250 ms si cuentan. Consola vacía y ningún request fallido.

El borde de las tarjetas

`--Color-Neutral-Lightest` (#F2F2F2) de fondo, radio 16px, padding 32px y borde de 1px.

El borde es el `rgba(0, 0, 0, 0.15)` de Figma, **no** `--Color-Scheme-1-Border` (que exporta `#000000`). El token tiene su propio comentario documentando que la discrepancia es invisible mientras `--Stroke-Border-Width` sea 0px y que solo había que revisarlo *"if the width ever becomes 1px"*: acá es exactamente ese caso, y con `#000000` el borde sería negro sólido en vez de una línea al 15%. Mismo criterio que el divisor del timeline de Layout / 121 / .

El padding y los gaps de 32px son literales porque no hay preset de spacing para 32 (los slugs van 10, 20, 30, 40, 50, 60, 80, 100, 112, 150).

Verificado (Chrome CDP)

Viewport	sec	gutter	intro	stats	card	H2	value	label	text	overflow
1920	1177.17	586	609x187.17	597x953.17	282.5x197	52/62.4	Roboto 80/96 w700	18/27	22/30.8	no
1440	1177.17	106	609x187.17	597x953.17	282.5x197	52/62.4	Roboto 80/96 w700	18/27	22/30.8	no
1400	1177.17	66	609x187.17	597x953.17	282.5x197	52/62.4	Roboto 80/96 w700	18/27	22/30.8	no
1300	1296.56	32	575.67x249.56	564.33x1072.56	266.16x224	52/62.4	Roboto 80/96 w700	18/27	22/30.8	no
1200	1381.36	32	525.17x249.56	514.83x1157.36	241.41x224	52/62.4	Roboto 80/96 w700	18/27	22/30.8	no
1199	951.94	-	1159x43.19	1159x740.75	563.5x194	36/43.2	Roboto 80/96 w700	16/24	18/25.2	no
768	1070.69	-	728x86.38	728x816.31	348x194	36/43.2	Roboto 80/96 w700	16/24	18/25.2	no
600	1163.06	-	560x129.56	560x865.5	264x194	36/43.2	Roboto 80/96 w700	16/24	18/25.2	no

Viewport	sec	gutter	intro	stats	card	H2	value	label	text	overflow
390	1843	-	350x172.75	350x1502.25	350x194	36/43.2	Roboto 80/96 w700	16/24	18/25.2	no

A 1440 el gutter da **106 exacto**, que es el valor de Figma. En 1300 y 1200 el gutter se apoya en el piso de 32px del gap y las mitades bajan proporcionalmente. Tarjeta: padding 32px , fondo rgb(242, 242, 242) , radio 16px , borde 1px solid rgba(0, 0, 0, 0.15) , las 8 iguales en ancho en cada viewport.

Las 4 filas, con el alto de cada tarjeta por fila (la comprobacion de que queden emparejadas):

Viewport	direccion	fila 1 (40% / 65%)	fila 2 (3x / 30%)	fila 3 (texto 5l / 4l)	fila 4 (texto 4l / 3l)
1920	row	197 / 197	251 / 251	219.98 / 219.98	189.19 / 189.19
1440	row	197 / 197	251 / 251	219.98 / 219.98	189.19 / 189.19
1400	row	197 / 197	251 / 251	219.98 / 219.98	189.19 / 189.19
1300	row	224 / 224	251 / 251	281.58 / 281.58	219.98 / 219.98
1200	row	224 / 224	305 / 305	312.38 / 312.38	219.98 / 219.98
1199	row	194 / 194	218 / 218	116.38 / 116.38	188 / 188
768	row	194 / 194	218 / 218	166.75 / 166.75	188 / 188
600	row	194 / 194	242 / 242	191.94 / 191.94	188 / 188
390	column	194 / 194	218 / 218	166.75 / 141.56	141.56 / 66

Ninguna fila tiene tarjetas de distinta altura en ningun ancho, y el ancho de las 8 es el mismo dentro de cada fila. A 390 la fila pasa a column , y ahí el align-items:stretch estira el ancho (todas a 350) pero no el alto: cada tarjeta mide su texto, que es lo unico razonable apiladas. Los tres **188** en negrita son el piso del min-height de "Scale AI" mandando, que es exactamente el alto del nodo.

Mobile

Figma no define variante mobile para este frame. Es una instancia de la home y no un component set, así que no hay desktop ni mobile que consultar. El padding lo resuelven los tokens y H2, descripcion y H6 bajan por sus presets.

Dos cortes:

- **1200** para la fila (H2 arriba, grid abajo). Los flex:0 1 <px> de las dos mitades vuelven a 0 1 auto , porque en columna el basis pasaria a ser la altura.
- **600** para el grid: recién ahí las tarjetas pasan a una sola columna, porque a $(350 - 32) / 2 = 159$ de ancho el número de 80px y los H6 de varias líneas quedan ilegibles. A 768 y 600 el grid sigue en dos columnas (348 y 264 de ancho por tarjeta), que es lo que se ve bien con el tamaño de los textos. Los gaps bajan a 16 en ambos ejes.

parse_blocks() : 157 bloques, 8 unicos, 0 sin registrar, 0 wp:pattern , 0 tags PHP. Jerarquia: **h1 x1, h2 x6, h3 x2, h4 x10**. 8 botones, 1 imagen de bloque.

FAQ / 3 / — H2 + boton a la izquierda, acordeon de 6 items a la derecha

Nodo #10218:72614 , la novena seccion, antes del footer. Mismo marco que las ocho anteriores: padding 112px 64px (tokens) y fondo blanco, así que reusa .ocx-section--large sobre .ocx-section .

Estructura: Container > Component (fila) > "Section Title" (H2 + boton) de 500px fijos + "Accordion List" de 6 items. A 1440 la lista mide $1312 - 500 - 80 = 732$.

El `gap:80px` del "Container" no se declara: tiene un solo hijo, es inerte. Igual que en `Stats / 55 /`.

El boton reusa `.ocx-btn--link`

Es la instancia #10244:641 del set 4179:8871, el mismo control que en `Layout / 59 /` y `Layout / 210 /`: sin fondo, sin borde, Manrope 500 de 16px en `#B69E52` y `chevron_right` de 24px. Medido 167.27 x 24.

Su `borderRadius:"12px 0px 12px 0px"` **no se replica y no se puede replicar** notar: con `background:none`, `border:0` y `padding:0` el radio no dibuja nada, y el navegador lo reporta en `0px`. Es el valor por defecto del componente, no una decision de diseno.

El `#B69E52` es el token `--Color-Husk` del theme, pero el boton lo lleva literal en `.ocx-btn--link` desde antes, asi que no se toco.

El > de las preguntas es texto literal

Figma no pone un icono: los 6 nodos de pregunta son texto `Heading/H4` que **terminan en el caracter >**, y en 5 de los 6 hay doble espacio antes (`"Hospitality >"`, `"Retail >"`, `"Education >"`, `"Healthcare >"`, `"Manufacturing >"`); el primero tiene uno solo (`"Financial Services >"`).

Se renderizan literales, que es lo que dice el archivo. Dos cosas:

- **Los espacios dobles no sobreviven al HTML**: el navegador colapsa la secuencia, asi que las 6 preguntas quedan con un solo espacio. Es el resultado correcto, porque esa inconsistencia del archivo es casi con seguridad un typo del diseno, no una intencion de espaciado.
- **Si de verdad se queria un chevron**, no es un cambio de texto: `core/heading` no admite hijos, asi que habria que rehacer cada pregunta como un group con el H4 y un `core/image` al lado, o usar `core/details` (que cambia la semantica a `<summary>` y rompe el H4). Queda como decision abierta.

Las 6 respuestas estan desplegadas - o mas bien, visibles

Figma dibuja los 6 items con su respuesta **visible**, o sea desplegados. No hay un bloque de acordeon en core, y `core/details` meteria un `<summary>` en vez de un `<h4>`, rompiendo la jerarquia. Asi que quedan 6 grupos con H4 + parrafo, todos abiertos, que es exactamente lo que muestra el diseno. Si se quiere el comportamiento desplegar, hace falta JS.

Bordes sin gap

`strokeWeight:"1px 0px 0px"` en los **seis** items, incluido el primero: hay linea encima de la primera pregunta. No se le quita al primero, seria inventarse un cambio que el archivo no pide.

El "Accordion List" **no declara gap**, y tampoco el item. Los items se separan por el borde superior de 1px, no por espacio en blanco. Medido: `row-gap: 0px` y los 6 items de 135.8 de alto a 1440, todos iguales.

El borde va como atributo del bloque (soporte de bordes por lado de core), asi que queda editable, y con `#000000` de Figma (`fill_34dc0314`) y no con `--Color-Scheme-1-Border`: aqui el negro del token es justo lo que se quiere, y el token del theme es el unico que difiere del color real en toda la home.

Los `padding 20px 0` de la pregunta y `0 0 24px` de la respuesta van en CSS y no como atributo: el preset de 20px si existe, pero al setear solo `top` y `bottom` core deja el horizontal en su default y mete 24px de `var:preset|spacing|50` en los lados, que correria la lista 48px. Y el 24 de la respuesta no es preset.

4. El borde media lo que el texto, no la columna. Tercera vez que el

`align-items:flex-start` de core rompe algo (ver `Stats / 55 /` y `Layout / 210 /`).

`.ocx-f3__list` es una columna, y en columna el eje transversal es el horizontal, así que cada ítem se ajustaba a su contenido. Medido a 1440: la pregunta **323.55 de ancho dentro de una lista de 732**, con la respuesta al lado en 378.41. O sea que las 6 líneas de 1px eran desiguales, que es justo lo contrario de lo que hace un acordeón, y `scrollWidth` no lo detecta porque no hay desborde.

Fix: `align-items:stretch` en `.ocx-f3__list` y en `.ocx-f3__item`, porque si el ítem se ajusta a su contenido el heading y el paragraph tampoco se estiran y el borde se queda corto otra vez. Verificado: ítem, pregunta y respuesta miden lo mismo que la lista en los 9 anchos.

No hace falta `flex-wrap:nowrap`

Al contrario que en `Stats / 55 /`: aquí solo un hijo lleva `basis` en px (los 500 de la intro) y el otro usa `1 1 0`, o sea que la suma de los `basis` es 580, muy por debajo del contenedor incluso a 1200. Core puede envolver tranquilamente.

La intro lleva `flex:0 1 500px` en vez de `0 0` para que en un viewport improbable baje antes que romper la fila; a 1200 o más nunca se llega a encoger. Y a 1199 se vuelve a `0 1 auto` al apilar, porque en columna el `basis` pasaría a ser la altura.

Mobile

Figma no define variante mobile: es una instancia de la home y no un component set. El padding lo resuelven los tokens, el H2 baja 52 -> 36 y el H4 36 -> 24. Un solo corte, en **1200** (el mismo que en `Layout / 59 /`, `Layout / 121 /`, `Layout / 210 /` y `Stats / 55 /`): la fila pasa a columna con `gap:40px` y la intro con `16`.

A 1199 el contenido es 1159 y la lista toma los 1159 completos, con las preguntas en una línea. A 1440 la lista tiene 732 y las preguntas también entran en una línea, así que el corte podría bajarse bastante; se deja en 1200 por consistencia con las otras cinco secciones de dos columnas.

Un detalle al medir: a 1199 el H2 da **1130.23** de ancho dentro de una intro de 1159, porque `align-items:flex-start` lo deja en `fit-content`. No se ve: el texto está alineado a la izquierda y `fit-content = min(max-content, disponible)`, así que corta en el mismo punto que un `fill`. Se deja como dice Figma ("Section Title" no declara `alignItems`, o sea MIN).

Verificado (Chrome CDP)

Viewport	sec	top	intro	list	H2	pregunta	respuesta	item	overflow
1920	1920x1038.78	row/80	500x305.56	1212x814.78	52/62.4	87.8	48	135.8 x6	no
1440	1440x1038.78	row/80	500x305.56	732x814.78	52/62.4	87.8	48	135.8 x6	no
1400	1400x1038.78	row/80	500x305.56	692x814.78	52/62.4	87.8	48	135.8 x6	no
1300	1300x1038.78	row/80	500x305.56	592x814.78	52/62.4	87.8	48	135.8 x6	no
1200	1200x1038.78	row/80	500x305.56	492x814.78	52/62.4	87.8	48	135.8 x6	no
1199	1199x972.31	column/40	1159x83.19	1159x721.13	36/43.2	72.19	48	120.19 x6	no
768	768x1015.5	column/40	728x126.38	728x721.13	36/43.2	72.19	48	120.19 x6	no

Viewport	sec	top	intro	list	H2	pregunta	respuesta	item	overflow
600	600x1058.69	column/40	560x169.56	560x721.13	36/43.2	72.19	48	120.19 x6	no
390	390x1173.88	column/40	350x212.75	350x793.13	36/43.2	72.19	48/72	144.19 / 120.19	no

La altura de la seccion cuadra con la cuenta de Figma: 6 items de (20 + 46.8 + 20 + 1) de pregunta mas (24 + 24) de respuesta = 135.8, o sea 814.8, mas 112 + 112 de padding = **1038.78**. El H2 da 249.56 = 4 lineas de 62.4. A 390 los items miden 144.19 o 120.19 segun si la respuesta parte a dos lineas, que es lo esperable apilados.

Boton: 167.27 x 24, `rgb(182, 158, 82)`, chevron de 24x24 cargado. 0 errores de consola, 0 imagenes rotas y 0 requests 4xx en los 9 anchos.

`parse_blocks()` : 183 bloques, 8 unicos, 0 sin registrar, 0 `wp:pattern`, 0 tags PHP. Jerarquia: **h1 x1, h2 x7, h3 x2, h4 x16**. 9 botones, 1 imagen de bloque. Balance de comentarios de bloque: 180 pares, 3 autocerrados (`template-part x2`, `site-logo`), 0 sin cerrar, 0 desbalanceados. Ojo: el template usa la **forma corta** (`<!-- wp:group -->`, sin `core/`) en los 183 bloques de apertura; WordPress la resuelve igual, pero un validador que exija `namespace/nombre` no va a encontrar nada.

Pendiente

- `/industries/`, el destino de "Explore Industries", sigue en 404 (ver la lista de rutas al principio del documento).

Columna fija en las tres secciones de dos columnas

Pedido del usuario: dejar **inmóvil la columna de la izquierda** mientras avanza la de la derecha, y que se suelte sola cuando la derecha termine de pasar. Aplicado a las tres secciones que tienen esa anatomía, que son exactamente las que comparten el mismo corte de apilado:

Sección	Fila	Se ancla	Avanza
"Why OpenCX" (Layout / 121 /)	<code>.ocx-l121__component</code>	<code>.ocx-l121__column</code>	<code>.ocx-l121__timeline</code>
"Results..." (Stats / 55 /)	<code>.ocx-l55__content</code>	<code>.ocx-l55__intro</code>	<code>.ocx-l55__stats</code>
"Built for the industries" (FAQ / 3 /)	<code>.ocx-f3__top</code>	<code>.ocx-f3__intro</code>	<code>.ocx-f3__list</code>

Es **position: sticky pelado, sin una línea de JS ni un scroll listener**: con `top` fijo, el elemento se despegas solo al llegar al final de su contenedor, y su contenedor es la fila, o sea la columna derecha. El "hasta que finalice el scroll de la derecha" sale gratis de ahí.

Por qué un bloque compartido y no tres reglas

Las tres apilan en `max-width: 1199.98px`, así que el comportamiento es idéntico en las tres y repetirlo en la regla base de cada sección solo multiplica la chance de que una se quede sin actualizar. El bloque va en `assets/css/patterns.css` despues de las reglas de `FAQ / 3 /`, que es la ultima de las tres en el archivo.

El `top` es el padding de la seccion, no un numero inventado

`top: var(--Section-Padding-padding-section-large, 112px)`. Razon: cuando el borde superior de la seccion llega a `y=0`, la columna esta a `y=112` (el padding de la seccion), y `top:112px` la deja exactamente ahí. **El anclaje no da ningun salto**, y mientras queda clavada sigue alineada con el primer item de la columna derecha,

que tambien arranca a 112. Un `top` inventado (digamos 40px) dejaria la columna flotando con un margen distinto al del resto de la seccion.

`align-self: start` esta aunque la fila ya diga `flex-start`

Las tres filas ya declaran `align-items: flex-start` en el rango desktop, asi que hoy el `align-self: start` es redundante. Se deja igual porque **un item flex estirado no tiene recorrido**: `position: sticky` sobre el no se mueve nunca y no falla de ninguna forma visible, solo deja de hacer efecto. Es el fallo clasico de esto, y la defensa va al lado de la regla que la evita en vez de confiar en que nadie va a tocar la fila.

Nada puede romperlo

`position: sticky` no funciona si algun ancestro tiene `overflow: hidden`, `transform`, `filter`, `perspective` o `will-change`, porque ese ancestro pasa a ser el contenedor de scroll. Revisado el archivo entero: el unico `overflow: hidden` es el de `.ocx-t23__avatar-img` (un avatar de 48x48, que no es ancestro de ninguna de las tres) y el unico `filter` es el `drop-shadow` del beam, que vive en el `::after` de la caja del prompt del hero.

Medido (Chrome CDP, 1440x900)

Recorrido disponible para la columna fija, o sea cuanto puede avanzar la derecha mientras la izquierda esta clavada:

Sección	Fila	Izquierda	Derecha	Recorrido
L121	1328	221	1328	1107
L55	953	187	953	766
F3	815	306	815	509

En los cinco anchos probados el recorrido va de 509 a 1457px, asi que el efecto se ve en todos.

Anclaje y suelta, recorriendo cada seccion del 0% al 102% de su alto. La izquierda queda clavada en `y=112` mientras la derecha sube, y se suelta en el punto exacto en que `izq.bottom` iguala `der.bottom`:

Sección	Inmovil hasta	Se suelta entre	<code>izq.bottom = der.bottom</code>
L121	70%	70% → 80%	199 = 199
L55	60%	60% → 70%	124 = 124
F3	45%	45% → 60%	326 = 326

Ejemplo del L55, mostrando que la derecha avanza y la izquierda no: `der.y` 112 → -64 → -241 → -417 → -594, con `izq.y` clavado en 112 durante los cinco pasos.

Corte exacto: a 1200 las tres dan `position: sticky / top: 112px / align-self: start`; a 1199, 768 y 390 las tres dan `position: static / top: auto`, o sea el layout apilado sin tocar. En el apilado no tiene que haber sticky: la columna unica se scrollea con la pagina, que es lo correcto.

`position: sticky` no altera el flujo, asi que la regresion de los cinco anchos quedo **identica a la linea base** (docH 9669/9905/8936/9121/11788, footerH 582/518/518/756, overflow 0, 32 imagenes, 13 secciones), consola vacia y ningun request fallido. La unica adicion al archivo son 6 lineas de CSS.

Lo que NO se hizo

- **Sin JS de scroll**. Un listener con `getBoundingClientRect` en cada frame sería mucho más fácil de hacer mal (y hay que acordarse de cancelar el `rAF`) para terminar obteniendo exactamente lo que `sticky` ya hace bien.

- **Sin top distinto por seccion.** Con 112 las tres columnas quedan alineadas con el arranque de su propia columna derecha, que es el unico criterio que las tres comparten.
- **Sin efecto en el scroll** (fade, parallax, spotlight). El pedido era "inmóvil", y cualquier movimiento contradice "inmóvil". Si se quiere despues, se suma; no antes.

Testimonial / 23 / — H2 + carrusel de 6 tarjetas (CSS-only con scroll-snap)

Nodo #10218:72914 , "Testimonial / 23 / " , decima seccion de la home y ultima antes del footer. Clases `.ocx-t23__*`.

El track es un carrusel, no una fila

Lo primero que hay que sacar de Figma es el ancho: **6 x 405 + 5 x 32 = 2590px** de tarjetas dentro de un contenedor de **1312px** a 1440. No es una fila que se pueda reproducir, es un carrusel. Dos cosas del archivo lo confirman:

1. El frame es una instancia (`"instanceId"`), no un component set: es una foto de la home, y la foto esta **recortada**, no completa.
2. La franja de 1312 muestra 3.2 tarjetas y el borde de la 4ta asoma **31px** (`1212 + 32 + 31`), y la flecha derecha esta en `x=1260` , o sea **32px antes** de ese borde, apuntando a la 4ta asomada. Es el montaje clasico de "se ve un poco de la proxima para inviting a scrollear".

Implementado con `overflow-x:auto + scroll-snap-type:x mandatory + scroll-snap-align:start` en cada tarjeta. Es lo unico que da un carrusel **real sin JS**: se mueve con el dedo, con la rueda y con shift+rueda, y engancha en cada tarjeta. Verificado por CDP: pidiendo `scrollLeft = 437` con el snap activo el navegador deja **437** a 1440 (justo el pitch de tarjeta) y **382** a 390 (el pitch de ese ancho), o sea rechaza las posiciones que no son punto de enganche. `scrollbar-width:none` porque Figma no dibuja scrollbar.

flex-wrap:nowrap es obligatorio y es la quinta vez que aparece el mismo problema de core (`.is-layout-flex` trae `flex-wrap:wrap`): sin el, las 6 tarjetas se van a 3 filas de 2 y no hay carrusel. Y es un fallo silencioso, porque ademas **el chequeo de overflow no lo detecta**: el track scrollea, asi que su `scrollWidth` da el del contenedor y `document.scrollWidth == clientWidth` sigue en verde. Hay que mirar `flexWrap` y `scrollWidth` del track, no el overflow de la pagina.

Ancho de tarjeta: `min(405px, 100%)` , sin breakpoint

`flex: 0 0 min(405px, 100%)` resuelve el responsive sin ningun `@media` : 405px mientras entra y 100% cuando no, que recien pasa a 437px de viewport (405 + los 32 de padding lateral de la seccion en mobile). A 390 las 6 tarjetas miden 350 y el `scrollWidth` da `2260 = 6 x 350 + 5 x 32`. Figma **no tiene variante mobile** de esta seccion (es una instancia, no un component set), asi que el comportamiento de abajo de 1200 sale de los tokens, no de un spec.

Alturas desiguales: es lo que dice el archivo

El track es un `row` que **no declara** `alignItems` y las 6 tarjetas son `vertical:hug` . O sea `align-items:flex-start` (el default) con alturas propias. Medido a 1440: **300 / 246 / 165 / 90 / 371 / 398**, un disparate de 90 a 398.

Se implemento asi porque es lo que dice Figma, pero conviene revisarlo con contenido real: 6 testimonios de largo parecido van a dar tarjetas parejas y el `flex-start` se va a ver perfecto. Con el contenido placeholder actual (una cita de una linea, una tarjeta con solo un boton) el `flex-start` es lo que menos se favorece. La alternativa seria `align-items:stretch` , y esa esta anotada en "Pendiente de decision".

Dots y flechas: botones reales, y `core/html` por una razon

Los dots son <button> dentro de un core/html , no core/spacer . Empezaron como seis spacers estaticos de 8x8 con la forma larga del bloque, pero el carrusel se volvio navegable y los spacers no sirven: no son enfocables, no aceptan `aria-current` y no pueden pulsar. Hoy el template trae **cuatro** botones (el numero lo ajusta el JS, ver abajo) y el CSS los estila con `.ocx-t23__dot`, `opacity:1` en `[aria-current]` y `0.2` en el resto.

El `core/html` es obligatorio y no por estetica: **display:flex** hay que ponerlo a mano en `.ocx-t23__dots`, porque un `core/html` no recibe las clases `is-layout-*` que core le pone a un `core/group` flex. Sin eso los botones quedan inline y el whitespace entre ellos cuenta como separador (medido: pitch 11.61 en vez de 16, y la banda 28.8 en vez de 28 por la linea base). El contenedor queda con `gap:8`, `justify-content:center` y `min-height:28px` (28 no es preset de spacing, asi que va en CSS).

Las **flechas si estan implementadas**, como `<button>` en el mismo `core/html`, con los SVG exportados de Figma de 48x48, que traen el fondo `#F2F2F2` y el borde blanco **cocinados dentro** (por eso el boton no necesita fondo, borde ni padding). Viven en `.ocx-t23__arrows`, con `display:none` por defecto y `display:flex` solo cuando la seccion tiene `[data-ocx-carousel]`, que es la marca que pone el JS. No van dentro del track a proposito: un `position:absolute` dentro de un contenedor con scroll se posiciona contra su *padding box* pero **viaja con el contenido**.

`assets/js/testimonial-carousel.js` : mejor progressive enhancement, y por que 4 dots y no 6

El carrusel funciona **sin JS**: el track es un contenedor de `scroll-snap` (`overflow-x:auto`, `snap-type: x mandatory`), asi que se desplaza y se ancla solo. Lo que el JS agrega, y nada mas:

- destapa las dos flechas (`display:none -> flex`) y quita el `disabled` a los dots;
- mueve un paso por click, con scroll suave (o instante si hay `prefers-reduced-motion`);
- mantiene los dots sincronizados con la posicion de scroll, venga de donde venga;
- hace el track alcanzable con teclado (es un contenedor de scroll, asi que el navegador no lo enfoca solo) y le cablea `Left / Right / Home / End`.

El numero de dots sale del layout, no de las tarjetas. A 1440 entran tres tarjetas y media, asi que solo cuatro de las seis pueden llegar al borde izquierdo: seis dots tendrian tres apuntando al mismo offset. El JS mide el *pitch* real entre tarjetas (`offsetLeft` de la segunda menos el de la primera, para no depender de leer el `gap` computado), divide el `scrollWidth` y **crea o borra dots** para que coincida con las posiciones alcanzables. Medido: **4 dots a 1440** y **6 a 390**, donde la tarjeta es `100%` y cada una es una posicion propia. El template trae 4 y el JS los reutiliza.

El `disabled` de los dots viene del template a proposito: sin JS no deben ser tab stops, porque no hacen nada. Con JS, el que no tiene a donde ir se vuelve a deshabilitar (`prev` en la posicion 0). El estado de "cual es el actual" va en `aria-current="true"`, que es lo que el CSS lee para la opacidad.

Medido en vivo: 4 dots a 1440 (ninguno deshabilitado, `aria-current` en el 0), 6 a 390, 2 flechas visibles con la anterior deshabilitada al estar en la posicion 0, track con `tabindex="0"`, `scrollWidth` 2590 contra `clientWidth` 1312, `scrollLeft` 0.

Avatar: el recorte va en el `img`, no en la figura

`core/image` pone el `className` en el `<figure>`, no en el `<img>`. Con la clase en la figura, `border-radius:50%` y `object-fit:cover` quedan en el elemento equivocado: el `img` conserva su relacion natural y se ve **achatado a 48x27** dentro de un circulo vacio (medido asi antes de corregirlo). El tamaño va en la figura y el recorte en el hijo:

```
.ocx-t23__avatar .ocx-t23__avatar-img img { /* (0,2,1) */
  display: block;
```

```
width: 100%;
height: 100%;
max-width: none;
border-radius: 50%;
object-fit: cover;
}
```

El selector lleva dos clases a proposito: `core` declara `.wp-block-image img` con `height:auto` y `max-width:100%`, que tiene **la misma especificidad (0,1,1)** que `.ocx-t23__avatar-img img`, así que con un solo nivel el resultado dependería del orden de carga.

El archivo es `assets/images/avatar-1.png`, que ya estaba en el theme **sin usarse** y resulto ser **byte-identico** al `imageRef` que exporta Figma para los 6 avatares: mismo md5 (`3e4a862abafeecc87ecada3720a7943d`), 1260x709. El `imageRef` es uno solo porque las 6 tarjetas son instancias de la misma plantilla (`EL-18ba1cef`): son el mismo avatar, no seis personas distintas. No se agrego una copia del asset.

Boton "Read case study": variante negra y 400

Es la misma instancia del set `4179:8871` que "Explore Testimonials", con dos overrides en el texto: `#000000` en vez de `#B69E52` y `Text/Regular/Normal (400)` en vez de `Text/Regular/Medium (500)`. El chevron no lleva override, así que sigue siendo el `chevron-right` negro de siempre, igual que en el boton dorado.

El selector necesita **las dos clases**:

```
.wp-block-button.ocx-btn--link.ocx-btn--link--plain .wp-block-button__link { color: #000; font-weight: 400; }
```

porque `.ocx-btn--link` define color y peso con la misma especificidad (0,2,0) y el orden en el archivo no alcanza para desempatar. El `borderRadius:"12px 0px 12px 0px"` del nodo no se replica, por lo mismo que en `FAQ / 3 /`: sin fondo ni borde ni padding el radio no dibuja nada.

Contenido: se replicó tal cual, con las inconsistencias de Figma

Figma tiene 6 estados distintos de tarjeta, y no son un error de maquetado, son 6 placeholders distintos:

#	Contenido	Cita	Nombre / Rol	Logo	Boton
1	cita + avatar	larga, 5 lineas	Dimitrios Politis / Nexx Burger	no	no
2	cita + avatar	media	Catalina Aristizabal / RL Funeral Services	no	no
3	cita + avatar	"lorem ipsum..."	Sarah Jones / Lorem ipsum	no	no
4	solo boton	—	—	no	si
5	logo + cita + avatar + boton	media	Elena Rossi / CTO, Finova SaaS	Logo=2	si
6	logo + cita + avatar + boton	media	Ben Carter / Director of Logistics, EduServe	Logo=1	si

Se implementó exactamente eso, sin inventar contenido. Lo que queda para revisar con el contenido real:

- La **tarjeta 4** es prácticamente un placeholder vacío (90px de alto: 32 + 24 + 32 + borde).
- La **tarjeta 3** tiene "lorem ipsum" de texto y de rol.
- Los **logos** solo están en 2 de las 6.
- Las **comillas** vienen mezcladas: `"..."` (curly) en 1, 2, 5 y 6, y `"lorem ipsum..."` con comillas rectas en la 3. Se normalizaron a curly.
- Falta el `alt` de los logos: hoy van vacíos como el resto de imágenes del theme.

Verificado (Chrome CDP)

Viewport	sec	h2	track	tarjetas	ancho tarjeta	overflow	errores
1920	1920x974.78	675x124.78 / 52	1792x398	6	405	no	0
1440	1440x974.78	675x124.78 / 52	1312x398	6	405	no	0
1300	1300x974.78	675x124.78 / 52	1172x398	6	405	no	0
1200	1200x974.78	675x124.78 / 52	1072x398	6	405	no	0
1199	1199x804.38	675x86.38 / 36	1159x362	6	405	no	0
768	768x804.38	675x86.38 / 36	728x362	6	405	no	0
600	600x804.38	560x86.38 / 36	560x362	6	405	no	0
390	390x847.56	350x129.56 / 36	350x450	6	350	no	0

Corte unico en **1200**, el mismo de las otras secciones de dos columnas: 52 -> 36 en el H2 por token (`--Text-Size-Heading-2`) y 18 -> 16 en la cita por `--Text-Size-Text-Medium` . Gaps del `container` 80 -> 40 y del `body` 48 -> 24. El ancho de tarjeta **no** necesita breakpoint (ver mas arriba).

`scrollWidth` del track: 2590 a ≥ 600 y 2260 a 390 (6 x ancho + 5 x 32). Pitch tarjeta a tarjeta: 437 a ≥ 600 y 382 a 390, o sea el `gap:32` se respeta siempre. Los dots miden 8x8 (**4 a 1440 y 6 a 390**, ver `assets/js/testimonial-carousel.js`), el activo con `opacity:1` y el resto con `0.2` , y la franja mide 28 con los dots centrados. El avatar sale **48x48 con `object-fit:cover` y `border-radius:50%`** , y la fila del avatar 48 de alto con `gap:16` y `align-items:center` ; nombre y rol quedan pegados (`row-gap:0`) y miden 48, que es lo que hace que la fila combine. Logos 120x48 con su `naturalWidth` correcta.

`parse_blocks()` : 243 bloques con cuerpo, 3 autocerrados, 0 desbalanceados, 0 sin registrar, 0 `wp:pattern` , 0 tags PHP. Jerarquia: **h1 x1, h2 x8, h3 x2, h4 x16**. 9 `.ocx-section` en orden, la 9ima y ultima es `ocx-t23__section` , antes del footer. 0 imagenes rotas, 0 requests 4xx, 0 errores de consola en los anchos revisados.

Pendiente

- `/customer-stories/` , destino de "Explore Testimonials" y de los tres "Read case study", sigue en 404 (ver la lista de rutas al principio del documento).
- Alturas desiguales** (`flex-start` + `hug`): ver "Alturas desiguales" mas arriba.
- Contenido placeholder de las tarjetas 3 y 4, y los `alt` de los logos.
- El teclado solo responde con el track enfocado. Un click de raton deja el foco en la flecha, y ahi `ArrowRight` no hace nada porque el evento no le llega al track. Es el comportamiento estandar de un carrusel; si se quiere que las flechas tambien respondan a las flechas del teclado, hay que escuchar `keydown` ahi tambien.

Testimonial / 23 / — carrusel con JS (`.ocx-t23` interactivo)

`assets/js/testimonial-carousel.js` , encolado por `inc/enqueue.php` con `defer` en el footer y version por `filemtime` . Sin dependencias ni build step, ~10 KB sin comprimir.

El `enqueue.php` paso de un bloque por script a un `foreach` sobre `array( 'navigation' => ..., 'testimonial-carousel' => ... )` . El handle de navegacion queda igual (`opencx-navigation`) y el `file_exists` se mantiene, que es lo que dejo `navigation.js` como stub inocuo mientras el nav es CSS-only.

El contrato sin JS / con JS

La regla del `enqueue.php` ("pattern markup ships fully functional in HTML") se cumple con tres piezas, todas verificadas con `Emulation.setScriptExecutionDisabled`:

	sin JS	con JS
track	scrollea y engancha por CSS (2590/1312 a 1440)	igual, y ademas navegable con teclado
flechas	<code>display: none</code>	visibles, 48x48, en los bordes del area visible
dots	4 <code><button disabled></code> , 8x8, el 1 opaco y 3 al 20%	los mismos botones, mas los que hagan falta, habilitados y navegables
<code>tabindex</code> del track	-1 (no existe)	0, con <code>role=group</code> y <code>aria-roledescription=carousel</code>

El `disabled` de los dots y el `display:none` de las flechas son el estado "sin JS" escrito en el markup. El JS hace `dots[i].disabled = false` y recién al final de `init()` pone `data-ocx-carousel="ready"` en la seccion, que es lo que revela las flechas por CSS: **las flechas no pueden aparecer un solo frame antes de estar conectadas.**

Por que `core/html` para los controles y no `core/button`

`core/button` se descarto despues de mirarlo en el contenedor:

- Es un bloque **estatico** con un filtro de render (`render_block_core_button` en `wp-includes/blocks/button.php`) que **devuelve '' si el boton no tiene contenido**. Un dot vacio disappears.
- Sus atributos son `tagName`, `type`, `url`, `title`, `text`, `linkTarget`, `rel`, `placeholder`, `backgroundColor`, `textColor`, `gradient`. **Ninguno es un `aria-label`**, y un boton de flecha sin nombre accesible es un problema de a11y, no un detalle.
- Un boton no navegable requiere `disabled`, y `core/button` tampoco lo expone.

`core/html` si permite `<button>` de verdad con `aria-label` y `disabled`. Es un bloque de core, así que la plantilla sigue siendo valida y el cluster se puede mover en el Site Editor, pero **no se edita dot por dot**. Es el trade-off: semantica correcta a cambio de granularidad. Los `aria-label` se escriben en el markup ("Ir al testimonio 3 de 6") y el JS los reescribe con el conteo real de tarjetas, así que agregar una tarjeta en el editor no deja un "de 6" colgado.

Las flechas no pueden vivir dentro del track

Un `position:absolute` dentro de un contenedor con scroll se posiciona contra su *padding box* pero **se mueve con el contenido**: viaja con el scroll. Por eso se creo `.ocx-t23__viewport`, un wrapper `layout:default` con `position:relative` que aporta un contexto de posicionamiento y contiene el track mas las flechas, con las flechas en `inset:0 + pointer-events:none` (los botones, `auto`) para no tapar el scroll.

Medido en los 8 anchos: flechas de 48x48, `padding-inline:16px` o sea a 16 del borde del area visible, y centradas verticalmente con delta 0 respecto del centro del track.

La posicion de Figma se descarto a proposito: los nodos `#10218:73000` y `#10218:73001` estan en absoluto en `x:-28` y `x:1260`, `y:210.11` respecto de la seccion. La izquierda quedaria **fuera del viewport** (el padding lateral es 64) y la derecha a 52 del borde, no en el borde. Superpuestas sobre los bordes del area visible es lo unico que las hace usables.

El `width:100%` del viewport no es cosmetico

El body es una columna con `align-items:center`, así que el viewport es un flex item `fit-content` y su ancho se derivaba de su contenido: el `100%` del track se volvía **cíclico** (el padre se define por el hijo que se define por el padre) y el navegador lo resolvía con la parte en px. Medido: a 390 las tarjetas quedaban en **405** en vez de 350 y la pagina desbordaba, `scrollWidth 1084` con viewport 390. Con `width:100%; min-width:0` en el viewport, el `min(405px, 100%)` de la tarjeta resuelve a 350 y el pitch pasa a 382.

core/html no trae is-layout-flex

Al pasar los dots de `core/group` a `core/html` perdieron el `display:flex` que core inyecta en los grupos con layout. Medido: pitch de **11.61** en vez de 16 (los botones inline, con el whitespace del HTML de por medio) y banda de **28.8** en vez de 28 por la línea base. El `display:flex` quedó declarado en `.ocx-t23__dots`. Las flechas no lo sufrieron porque su regla ya declaraba `display:flex` para el estado con JS.

El título estaba 318px a la izquierda de su centro

El `max-width: 675px` estaba en el `.ocx-t23__heading`, o sea **en el h2**, y el `text-align: center` centraba el texto *dentro* de esa caja de 675. Pero la caja misma era un item flex del contenedor con `align-items: flex-start`, así que quedaba **pegada al inicio**: medido, su centro estaba 318.5px a la izquierda del centro del contenedor a 1440, y 558.5px a 1920. A 390 pasaba por estar centrada sola, porque el contenedor (350) es mas angosto que 675.

El fix es mover el tope al grupo del título (`.ocx-t23__title`) y ahí centrarlo.

	bloque	desvio del centro
antes, 1920	675px en <code>x:64</code>	-558.5px
antes, 1440	675px en <code>x:64</code>	-318.5px
antes, 768	675px en <code>x:20</code>	-26.5px
antes, 390	350px en <code>x:20</code>	0 (por casualidad)
después, todos	768px o el ancho del contenedor	0

align-self: stretch de Figma no centra, y es medido

El layout que da Figma es `display:flex; flex-direction:column; align-items:center; gap:24px; align-self:stretch; max-width: 768px`. Aplicado **literal** al navegador **no centra**:

variante	1920	1440
<code>stretch + max-width (literal)</code>	768px en <code>x:64</code> , desvio -512	768px en <code>x:64</code> , desvio -272
<code>stretch + width:100% + max-width</code>	768px en <code>x:64</code> , desvio -512	768px en <code>x:64</code> , desvio -272
<code>stretch + width:100% + margin-inline:auto</code>	768px en <code>x:576</code> , desvio 0	768px en <code>x:336</code> , desvio 0
<code>center + width:100% + max-width</code>	768px en <code>x:576</code> , desvio 0	768px en <code>x:336</code> , desvio 0

La razón es la regla de flexbox: cuando `align-self: stretch` encuentra que el tamaño transversal queda **limitado por max-width**, el item se alinea como `flex-start`, no se estira. O sea que `stretch + max-width` no es "llenar hasta 768", es "768 pegado a la izquierda". Como el objetivo declarado era centrar, se usó `align-self: center + width: 100%`, que conserva el comportamiento de "llena lo disponible hasta el tope" (lo aporta el `width: 100%`) y además centra. El `margin-inline: auto` da el mismo resultado; se eligió `center` por ser la forma declarativa de "centrar en el eje transversal" y no depender de la resolución de auto-margenes.

Medido final, con el texto real medido con `Range` y no con la caja del bloque:

ancho	contenedor	titulo	x	texto	desv. bloque	desv. texto	lineas
1920	1792	768	576	715.53	0	0	2
1440	1312	768	336	715.53	0	0	2
1300	1172	768	266	715.53	0	0	2
1200	1072	768	216	715.53	0	0	2
768	728	728	20	698.41	0	0	2
600	560	560	20	495.38	0	0	2
390	350	350	20	295.95	0	-0.01	3

El gap: 24px gana al `:where(.is-layout-flex) { gap: 0px }` de los global styles porque `:where()` no aporta especificidad y `patterns.css` se carga despues, que es el mismo mecanismo que ya usan `.ocx-t23__container` (80) y `.ocx-t23__body` (48). Con un solo hijo el gap es inerte, pero queda declarado para el dia que se agregue un eyebrow o un subtitulo.

El alto de la seccion no cambio (975 a 1440, 848 a 390) porque el titulo ya ocupaba dos lineas con 675 y sigue ocupando dos con 768, asi que el carrusel de abajo no se movio.

Un dot por posicion alcanzable, no uno por tarjeta

A 1440 entran 3.2 tarjetas, o sea que de las 6 solo **cuatro** pueden llegar al borde izquierdo: las posiciones son 0, 437, 874 y 1278 (el final del track), y $4 \times 437 = 1748 > 1278$. Los 6 dots de Figma apuntaban a 4 offsets, asi que tres de ellos eran el mismo boton. A 390 la tarjeta es 100% y si son seis posiciones (0, 382, 764, 1146, 1528, 1910).

Decision: **un dot por posicion alcanzable**, derivado del layout y no del markup. El conteo sale de `Math.ceil( max / pitch ) + 1` con tope en la cantidad de tarjetas:

ancho	track	pitch	max	dots	posiciones
1920	1792	437	798	3	se ven 4.1 tarjetas
1440	1312	437	1278	4	3.2 visibles
1300	1172	437	1418	5	
1200	1072	437	1518	5	
1199	1159	437	1431	5	
768	728	437	1862	6	
600	560	437	2030	6	
390	350	382	1910	6	1 visible

Que a 1920 salgan 3 y no mas es la consecuencia honesta del modelo: cuanto mas se ve, menos posiciones hay. Los dots no mienten en ningun ancho.

El redondeo es `ceil` y no `floor`, y esa es toda la trampa. El final del track casi nunca es un multiplo exacto del pitch: a 1440 $1278 / 437 = 2.92$, y $\text{floor}(2.92) + 1 = 3$ **descartaba el final del track** y reportaba 3 posiciones cuando 4 son alcanzables. Con `ceil` son $3 + 1 = 4$. El $+ 1$ final es el offset del final, que se cuenta aparte justamente por no ser multiplo.

Efectos colaterales que se pudieron borrar:

- El salto de intencion en el borde de `goTo` ya no hace falta: la ultima posicion es exactamente el final del track, entonces `offsetFor( last )` siempre cae dentro y el ida y vuelta posicion-indice es exacto.

- Las flechas avanzan **una posicion**, que es lo que hace que un dot y una flecha lleguen al mismo sitio. Antes avanzaban una tarjeta y eso obligaba a los tricks del punto anterior.
- `syncDots` reutiliza los dots del template y agrega o quita los que faltan: el markup trae 4 (los de Figma desktop), asi que en 1440 no se crea ni se borra nada, y en 390 se crean 2. Se llama en el `init` y en el `resize`.
- El click de los dots va **delegado** en `.ocx-t23__dots`, no por dot, porque un listener atado a un dot que `syncDots` borra se va con el. Verificado: un dot creado en runtime por un `resize` responde a un click de raton real.

El `aria-label` dice **posiciones**, no testimonios: "Ir a la posicion 4 de 4". Un dot es una ventana sobre la fila, y a 1440 el ultimo muestra las tres tarjetas finales, no el testimonio 4. El texto se reescribe en cada `render` con el conteo real, asi que un breakpoint no deja un "de 4" colgado.

El bug que los tests no podia ver: `pointer-events`

Las flechas no funcionaban con raton y los dots si. No era el JS: el listener estaba conectado y `goTo` hacia lo que se le pidiera. La causa era que `.ocx-t23__arrows` tiene `pointer-events: none` (para no tapar el scroll del track) y **`pointer-events` es una propiedad heredada**, asi que los botones heredaban `none`. Los dots no lo sufrían porque su contenedor no apaga la propiedad.

Lo que lo escondió: toda la suite de verificación usaba `element.click()` desde JS, y `click()` **dispara el evento directo sobre el elemento, saltandose el hit-testing y el `pointer-events` por completo**. Daba verde con el bug puesto. Los dots "funcionaban" en el test porque nunca lo tuvieron, no porque estuvieran bien.

Regla: cualquier control que se pruebe con `.click()` tiene que probarse además con `Input.dispatchEvent` de CDP en las coordenadas reales del `getBoundingClientRect`. Si el elemento esta fuera del viewport el click sale en el vacío y el test pasa sin hacer nada, asi que hay que centrar la seccion con `scrollIntoView` **antes** de leer las coordenadas, y no en cada paso: un `scrollIntoView` por paso también desplaza el track y falsea las lecturas.

Con el `pointer-events: auto` en `.ocx-t23__arrow`, el click real recorre todo el recorrido: `next 437 -> 874 -> 1278` con `next` deshabilitado al final, `prev 874 -> 437 -> 0` con `prev` deshabilitado al inicio, y los dots 1 y ultimo. Cero errores de consola.

Verificado (Chrome CDP)

Con **mouse real** (`Input.dispatchEvent`), con el track enfocado para el teclado:

paso	1440 (4 dots)	390 (6 dots)
inicial	<code>sl 0</code> , dot 1, prev off	<code>sl 0</code> , dot 1, prev off
next x1 / x2 / x3	437 / 874 / 1278, dots 2 / 3 / 4	382 / 764 / 1146, dots 2 / 3 / 4
next en el tope	sin cambio, <code>next disabled</code>	<code>sl 1528</code> , dot 5
prev	874, dot 3	1146, dot 4
prev hasta el tope	<code>sl 0</code> , prev disabled	<code>sl 382</code> , dot 2
dot ultimo / dot 1	1278 dot 4 / 0 dot 1	1910 dot 6 / 0 dot 1
ArrowRight x2	874, dot 3	764, dot 3
End / Home	1278 dot 4 / 0 dot 1	1910 dot 6 / 0 dot 1
arrastre a 42%	1278, dot 4	764, dot 3

Resize en vivo 1440 -> 390 -> 1440: los dots pasan 4 -> 6 -> 4, el indice se re-deriva del offset ya recortado y `next` sigue funcionando. Un click de raton real sobre un dot creado por ese `resize` (`act 1 -> 6`, `sl 0 ->`

1910).

`prefers-reduced-motion: reduce` con `Emulation.setEmulatedMedia`: el scroll es **instantaneo** (437px ya movidos a los 60ms, con `smooth` serian 0).

Sin JS en 1440 y en 390: 4 dots estaticos, **0 de 4** habilitados, `data-ocx-carousel` ausente, flechas en `display: none`, `tabIndex -1`, sin overflow.

La trampa del `<div>` desbalanceado

Al meter el wrapper del viewport, un `str.index()` localizo el `<!-- /wp:group -->` equivocado y el markup quedo con los `<div>` cruzados. **Los comentarios de bloque seguian balanceados**: el parser de bloques de WordPress trabaja sobre comentarios, y el parser de HTML del navegador "arregla" solo el desbalance, asi que la pagina se ve bien y no hay ningun error en consola. Lo unico que lo delata es el conteo:

`t.querySelectorAll('.ocx-t23__card').length` daba **1** mientras `s.querySelectorAll('.ocx-t23__card').length` daba 6.

Regla para el validator: **contar `<div y </div>` por seccion ademas de los comentarios**. Los comentarios balanceados no implican HTML balanceado.

CTA / 25 / — H2 + parrafo + un boton, todo centrado (id 10218:73343)

Decima seccion de la home, entre `Testimonial / 23 /` y el footer. Nodo volcado de Figma:

```
CTA / 25 / 1920x415 padding 112 / 64, fill #FFFFFF, alignItems stretch, gap 80
└ Container fill, alignSelf stretch, alignItems stretch, gap 80
  └ Content 1279.81x191.39, alignSelf stretch, alignItems center, gap 32
    └ Content (texto) alignSelf stretch, alignItems stretch, gap 24
      └ Heading/H2 "Ready to run your business on human understanding?"
        └ Manrope Medium 52 / 62.4 / ls -0.01em / center / #000
          └ Text/Medium/Normal 18 / 27, Manrope Regular, center, #000
      └ Actions hug, row, gap 16, alignSelf center
        └ Button 127.86x46 Style=Primary (`4179:8872`) "Let's meet" -> /lets-meet/
```

Copy

Se replico literal, sin reescribir, con dos correcciones de presentacion pedidas por el usuario:

- `<br>` en el H2, entre `Ready to run your business on` y `human understanding?`.
- `<br>` en el parrafo, entre `...finally working from the` y `same understanding of your business..`.

Los dos cortes son **exactos** y no aproximados: con `getClientRects()` el ancho de la linea 1 del H2 mide `714.52px` en 1920, y un `<span>` de probe con la misma fuente y el texto `Ready to run your business on` mide `714.52px` tambien. Lo mismo en la linea 2 (`555.42px`) y en las dos del parrafo (`719.33px / 315.33px`). O sea que el corte cae donde se pidio y no donde caeria por balanceo.

El unico detalle de copy del nodo es que el boton usa la **comilla tipografica** (`Let's meet`), no el apostrofo recto.

Padding: los cuatro valores van en el CSS, no en el atributo del bloque

`112` arriba, `64` izquierda, `64` derecha, `112` abajo, **fijos en todos los anchos**. Quedan en `.ocx-section.ocx-c25__section { padding: 112px 64px; }` y el `style.spacing.padding` se saco del markup. Tres motivos, todos medidos:

1. El atributo del bloque no se aplica en un core/group con layout: flex. El rendered llega con `style="margin-top:0;margin-bottom:0"` y **sin padding**: el `style.spacing.padding` del comentario de bloque se descarta en silencio. No es un problema de presets: se probó con los `var:preset|spacing|...` (los presets 112 y 80 existen en `theme.json`) y también con px duros (`"top":"112px"`), y en los dos casos el padding no aparece en el HTML servido. El único bloque del tema que sí recibe su padding del atributo es el **hero**, porque es `layout: default`. O sea que los otros ocho `.ocx-section` tienen un `style.spacing.padding` **muerto** en el markup: nunca aplico y no se toca ninguno, para no mover secciones que ya funcionan.

2. La cadena de clases da los valores solo en desktop. `.ocx-section (0,1,0)` pone `padding: var(--Section-Padding-padding-section-medium, 80px) var(--Page-Padding-padding-global, 64px)` y `.ocx-section--large (0,1,0)` sube el vertical a `var(--Section-Padding-padding-section-large, 112px)`: en 1920 y 1440 eso ya daba 112/64. Pero `tokens.css` redefine esos tokens a valores phone dentro de `@media (max-width: 1199.98px)` (`--Page-Padding-padding-global: 20px`, `--Section-Padding-padding-section-large: 64px`), y por debajo de 1200 **todas** las secciones caen a 64/20. Con dos clases (0,2,0) el override gana en cualquier ancho y sin depender del orden.

3. El fondo si viene del markup: `backgroundColor:"base"` (`has-base-background-color has-background`), que es el `#FFFFFF` del nodo.

Consecuencia en mobile que hay que revisar

Con 64px de padding lateral fijos, a **390px** la columna de texto queda en **262.8px**, contra los **350px** que tienen las demás secciones (que usan 20px laterales). El H2 pasa de 3 líneas a **4** y la sección crece de 431.56px a **594.75px** de alto. No es un error — son los valores que se pidieron —, pero la sección queda visible más "angosta" que el resto de la página en el mismo ancho. Si en mobile se quiere el padding de las otras secciones, alcanza con envolver la regla en `@media (min-width: 1200px)`.

Los dos `gap: 80px` (sección y Container) son **inertes**: cada uno de esos dos nodos tiene un solo hijo. Se declaran igual para que el editor muestre la estructura del archivo, que es el criterio que se viene usando en el resto del backlog.

align-items: center en la sección esta anulado por el align-self del container

El layout que paso el usuario para el nodo dice `align-items: center`, mientras que Figma declara `alignItems: stretch` ahí. Se implementó la instrucción del usuario, y el resultado es el mismo que en Figma: para un item flex, `align-self` siempre le gana al `align-items` del padre, así que el `align-self: stretch` del Container lo deja ocupando el ancho completo igual. Se declara el `center` porque es lo que dice el nodo, y depende del orden: esta regla está **después** de `.ocx-section--stretch`, que también pone `align-items: stretch` con la misma especificidad (0,1,0).

El flex-start de core: tercera vez que hace falta align-self: stretch

El primer render dio Content de **1279.81px** en vez de 1792, con el H2 y el párrafo **256px a la izquierda** de su centro. La causa es la misma que descentra el título de Testimonial / 23 / : **core pone align-items: flex-start en los grupos con orientation: vertical**, así que cada nivel se encoge a su max-content en vez de llenar, y el `text-align: center` de los hijos centraba dentro de una caja que ya estaba corrida a la izquierda.

Se resuelve con lo que declara el propio nodo, y hay que hacerlo en **los dos** niveles, no solo en el primero:

nivel	Figma	sin esto
<code>.ocx-c25__container</code>	<code>alignItems:stretch</code>	el Content se encoge al max-content del H2

nivel	Figma	sin esto
<code>.ocx-c25__content</code>	<code>alignSelf:stretch</code>	el grupo de texto no llena el Container
<code>.ocx-c25__text</code>	<code>alignSelf:stretch,</code> <code>alignItems:stretch</code>	H2 y parrafo quedan fit-content, alineados al inicio

`.ocx-c25__actions` si queda en `hug` (127.86), que es lo que dice Figma, porque su padre lo centra con `align-items: center`.

El tope de 768: donde va, y por que `stretch` y `margin: auto` no conviven

El nodo declara `max-width: 768px` en el Content, y eso es lo que hace que los dos cortes `<br>` se cumplan: la linea 1 del parrafo necesita 719.33px a 18px. Sin el tope, la caja mide 1792 y cada linea queda flotando en un bloque de 1792 de ancho.

El tope esta en la caja de 768, pero **la regla que lo produce esta en `.ocx-c25__container`, no en `.ocx-c25__content`**. Es la unica forma de que el Content mida 768 y quede centrado, y la razon es que en flexbox **`align-self: stretch` y los margenes auto son mutuamente excluyentes**: el `stretch` solo aplica "if neither of the cross-axis margins are auto". Se probó de las dos formas y las dos fallan, con midiendo:

intento	resultado medido en 1920
<code>max-width: 768</code> solo en el Content, con <code>align-self: stretch</code>	caja de 768 en <code>x=64</code> , 512px a la izquierda del centro
<code>max-width: 768 + margin-inline: auto</code> en el Content	centrada, pero de 719.33px (el max-content del parrafo): los margenes auto cancelan el <code>stretch</code>
<code>max-width: 768 + margin-inline: auto</code> en el Container	768 exactos y centrados , y el Content si se estira a 768 con su <code>align-self: stretch</code>

En la ultima, el `width: 100%` mas el `max-width` fijan el Container en 768, los margenes auto lo centran, y el Content se estira a 768 porque no tiene margenes auto. El `max-width: 768px` se deja tambien declarado en el Content, que es lo que pide el nodo, pero ahi es **no-op**: el que produce la caja es el del Container.

WordPress resetea los margenes de los hijos de un flex layout

`margin-inline: auto` en `.ocx-c25__container` pelado **no hacia nada**: computaba `0px/0px`. No era un problema de cascada propia sino de core, que trae

```
.is-layout-flex > :is(*, div) { margin: 0; }
```

Eso es **(0,1,1)** — la clase mas el tipo que `:is()` toma de su argumento mas especifico, `div` — y le gana a un `.ocx-c25__container` pelado, que es (0,1,0). Por eso el centrado va en `.ocx-section > .ocx-c25__container`, que es (0,2,0). Sirve para cualquier `margin` auto dentro de un grupo con layout flex en este tema, no solo para esta seccion.

Ojo con la aritmetica: el `gap: 80px` de la seccion y del Container son **inertes** (un solo hijo cada uno), y `margin: 0` de core es lo que evita que el `align-items: center` de la seccion genere un margen vertical. No hay que "arreglar" nada de eso.

Aire entre la seccion y el footer: `spacer` de 112 (resuelto: el `spacer` se borro)

La seccion ya tenia 112px de `padding-bottom`, asi que el hueco de caja era 112. Aun asi se veía pegada al footer, y la razon es que **`parts/footer.html` no tiene `padding-top`**: el grupo del footer solo trae `padding-`

left/right: 64px , de modo que el logo arranca en el mismo pixel donde termina la seccion y el unico aire era el de la seccion.

Se agrego un `core/spacer` de 112px entre la seccion y el footer, en vez de subir el `padding-bottom` de la seccion, por dos razones: los 112px del nodo quedan intactos (el usuario los pidio de forma explicita dos veces) y el extra queda como bloque editable en el Site Editor, que es el mismo patron que ya usan los dots de `Testimonial / 23 /`.

Medido en 1440 y en 390:

	antes	despues
seccion	8818 - 9323 (505)	8818 - 9323 (505)
spacer	—	9323 - 9435 (112), bg transparente, aria-hidden="true"
footer	9323 - 9734	9435 - 9846
aire del boton al logo del footer	112px	224px
alto de pagina (1440)	9734	9846

La seccion no cambio de alto ni de padding, y las diez secciones siguen siendo diez (el spacer no es una `.ocx-section`). Sigue siendo 10 el conteo de `h2`.

Alternativa que no se eligio: `padding-top: 112px` en `.ocx-footer`. Es el lugar "correcto" arquitectonicamente, pero el footer es un template part compartido: el padding se sumaria en todas las paginas, no solo en la home. Si en el futuro se quiere aire antes del footer en todo el sitio, esa es la regla que hay que tocar, y el spacer de la home se puede sacar.

Como quedo al final: el spacer se borro (112px de aire, no 318)

Los dos 112 se acumularon con el `padding-top: 80px` que pidio el usuario para la raiz del footer, y el aire quedo en **318px** del boton al logo, contra los 224 del diseno: demasiado. Decidio el usuario **borrar el spacer**, dejando que el aire sea **112px hasta que comienza el footer**.

`<!-- wp:spacer {"height":"112px"} -->` salio de `front-page.html` (con su `<!-- /wp:spacer -->`, que en este tema hay que escribir siempre cerrado: ver `core/spacer` no renderiza). El `padding-top: 80px` de `.ocx-footer` **se queda**, porque ese si es explicito y aplica a todas las paginas.

Medido, del borde inferior del boton de CTA/25 al **comienzo de la caja** del footer:

	1440	1199	768	390
boton → comienzo del footer	112	112	112	112
boton → logo del footer	192	160	160	160
padding-top del footer	80	48	48	48
alto del footer	582	518	518	756
alto de pagina	9905	8936	9121	11788

O sea que los **112px** que pide el nodo son los del `padding-bottom` de la ultima seccion, que ya estaban, y el logo queda 192px despues del boton a 1440 (112 del aire + 80 del padding del footer) y 160px por debajo de 1200 (112 + 48). El alto de pagina bajo exactamente 112 en los cinco anchos, y la seccion no cambio: siguen siendo 13 bloques en `main`, `h2` 9, 32 imagenes sin ninguna rota, CTA con 112 / 64 / 112 / 64.

El boton reusa `.ocx-btn--primary` y le agrega tres detalles

Es el mismo componente `Button / Style=Primary ( 4179:8872 )` que usan los dos headers, así que se reusa la clase y **no se toca**: lo que cambia son tres cosas del nodo que la clase no tiene.

propiedad	.ocx-btn--primary	nodo 10218:73350	como se resuelve
border	0	strokes: #B69E52 , strokeWidth: 1px	override scoped
font-weight	300	Text/Regular/Medium = Manrope 500	override scoped
color	--Color-Spring-Wood #F8F6F0	#FFFFFF	override scoped

El color es un blanco con un pelo de crema: la diferencia no se ve, pero el nodo dice blanco y el token `--Color-White: #FFFFFF` existe en `tokens.css`. El override es `.wp-block-button.ocx-c25__btn .wp-block-button__link (0,3,0)` contra `(0,2,0)` de la clase compartida, así que gana sin depender del orden en el archivo. Queda **pendiente de decisión** si estos tres detalles deberían ser globales para los tres usos del botón dorado (ver `Pendiente`).

Verificado (Chrome CDP)

Sin errores de consola, sin 404 de assets, sin imágenes rotas, `overflow` igual al `clientWidth` en los cinco anchos:

	1920	1440	1199	768	390
seccion	1920x415.39	1440x415.39	1199x393.19	768x460.38	390x594.75
padding t / r / b / l	112 / 64 / 112 / 64	112 / 64 / 112 / 64	112 / 64 / 112 / 64	112 / 64 / 112 / 64	112 / 64 / 112 / 64
container / content	768	768	768	640	262.81
H2 lineas	2	2	2	2	5
H2 ancho linea 1 / 2	714.52 / 555.42	idem	494.67 / 384.53	idem	207.88 / 230.47 / 42.67 / 114.89 / 262.81
H2 desvio del centro	0	0	0	0	0.41
parrafo lineas	2	2	2	2	5
parrafo ancho linea 1 / 2	719.33 / 315.33	idem	639.41 / 280.28	idem	231.7 / 230.34 / 170.97 / 206.31 / 70.78
parrafo desvio del centro	0	0	0	0	0.41
gaps (contenido / texto / actions)	32 / 24 / 16	igual	igual	igual	igual
boton	127.86x46, pad 10 / 24, #B69E52 , borde 1px, #FFFFFF , w500, tabIndex 0				

Los cuatro lados del padding se midieron con `getComputedStyle` en los cinco anchos: 112px / 64px / 112px / 64px en todos, sin excepciones. Container y Content miden **768 exactos** en 1920, 1440 y 1199 (por el tope), y en

768 y 390 los limita el ancho disponible. El desvío del centro del H2, del párrafo y del botón es **0** en los cinco anchos. Las otras ocho secciones **no** se movieron (siguen en 112/64 en desktop y 64/20 en ≤1199).

Umbral exacto de las dos líneas: 767.41px de viewport. La línea 1 del párrafo necesita 639.41px a 16px, y con 64px de padding lateral por lado eso da $639.41 + 128 = 767.41$. Mientras el viewport mida 767.41px o más, las dos líneas se cumplen; por debajo, el párrafo pasa a 3 líneas. Es decir que el layout aguanta en 1920, 1440, 1199 y 768 —todos los anchos reales de escritorio y tablet— y solo se rompe en 390, que es el caso de mobile que ya estaba anotado.

En el navegador la sección mide **415.39px** contra los **415px** que declara Figma: los 0.39 restantes son la diferencia de la métrica de Manrope en `line-height` (62.4 vs 62), no del layout.

Regresión de la página completa: **10** secciones, `ocx-c25__section` es la última y el footer viene después, `h1 1 / h2 9 / h3 2`, carrusel intacto (4 dots en 1440, 6 en 390, `data-ocx-carousel=ready`), `<div> 157 / 157` balanceados.

Pendiente

- **Las dos líneas no pueden ser dos líneas a 390px.** Los cortes `<br>` están fijos y se cumplen en 1920, 1440, 1199 y 768, pero a 390 ambos bloques dan **5 líneas**: la columna mide 262.81px (por el padding lateral de 64px) y la línea 1 del H2 necesita 494.67px a 36px, y la del párrafo 639.41px a 16px. No es un problema del corte: aun sin `<br>` el texto no entra, porque $494.67 > 390$. El límite real es 767.41px de viewport (ver la nota de la tabla). Lo que chocan son dos decisiones del usuario —**padding fijo de 64px y dos líneas exactas**— y por debajo de 767px no pueden ser las dos. Se eligió el padding, que fue lo pedido.
- **El padding en mobile.** Los cuatro valores del nodo se aplican fijos, y a 390px la columna de texto queda en 262.8px contra los 350px de las demás secciones. Es lo que se pidió, así que quedo así; si se prefiere que en mobile siga el 64/20 de las otras, se envuelve la regla en `@media (min-width: 1200px)` y listo.
- **El aire antes del footer solo está en la home.** El `core/spacer` de 112px **ya no existe** (ver Aire entre la sección y el footer), así que el aire de 112px ahora es el `padding-bottom` de la última sección de la home, y el resto de las páginas siguen con el footer pegado a su última sección. Si el diseñador quiere ese aire en todo el sitio, va como `padding-top` en `.ocx-footer` (afecta a todas las páginas).
- **`style.spacing.padding` muerto en los otros ocho `.ocx-section`.** El atributo está en el markup y core lo descarta en silencio porque son grupos con `layout: flex` (ver la nota de padding). Hoy no molesta porque el padding real llega por las clases, pero es una trampa para el que edite en el Site Editor esperando que el control de padding del bloque haga algo. Corregirlo implicaría migrar esas ocho secciones a `layout: default` o a padding en CSS: son decisiones de maquetación, no un fix de una línea.
- **`/lets-meet/` devuelve 404.** La ruta viene del nodo y se dejó tal cual; no se inventó ningún destino. Es el mismo caso que `/customer-stories/`: el wireframe apunta a páginas que todavía no existen. Hay que crear la página o confirmar la ruta final con el diseñador.
- **¿Los tres detalles del botón dorado son globales?** Hoy viven en el override de CTA/25 y los headers siguen con `border: 0`, `font-weight: 300` y `#F8F6F0`. Si el componente `Primary` del archivo de diseño es uno solo, corresponde subir los tres a `.ocx-btn--primary` y borrar el override; si los headers son una variante, el override se queda.

Footer / 3 / — template part compartido reconstruido desde 10244:544

`parts/footer.html` es el único template part que comparten `front-page.html` e `index.html`, así que todo lo que se escriba ahí afecta a **todas** las páginas del sitio, no solo a la home. Antes de cambiar nada se midió el footer viejo (columnas 66.67% / 33.33%, `site-logo` con `width:159`, cinco redes sociales, dirección real y una fila de créditos) y se confirmó que **no tenía ni una regla CSS propia** en `assets/css/*.css`: todo era `style` inline sobre el grupo.

El arbol del nodo, y el bloque que lo representa:

nodo	bloque	notas
Footer / 3 / #10244:544	core/group.ocx-footer.ocx-section	layout: flex vertical, width:1440px
Container #10218:70189	core/group.ocx-footer__container	columna, gap:80
Content #10218:70190	core/group.ocx-footer__main	fila, gap:64
Column #10218:70191	core/group.ocx-footer__brand	columna, gap:32
Company Logo	core/site-logo {"width":148}	frame de 148x63
Content #10218:70193	core/group.ocx-footer__contact	columna, gap:24
Content #10218:70194 / 70197	core/group.ocx-footer__block x2	columna, gap:4
Content #10218:70199	core/group.ocx-footer__block-links	columna, sin gap
Social Links #10218:70202	core/social-links	blockGap:12 , logos, 24x24
Column #10218:70208	core/group.ocx-footer__links	fila, gap:24
Link List #10218:70209 / 70220	core/navigation x2	una por menu
Link (x8)	items del menu	padding: 8px 0 cada uno
Credits #10218:70231	core/group.ocx-footer__credits	columna, gap:32
Divider #10218:70232	core/separator	1px , rgba(0,0,0,.15)
Row #10218:70233	core/group.ocx-footer__legal	space-between
Footer Links #10218:70235	core/group.ocx-footer__legal-links	gap:24

Los dos menus son menus clasicos de verdad, pero ref no es el term id

El punto que mas tiempo llevo: **ref de core/navigation no acepta un term de nav_menu** . Con {"ref":7} los dos navs renderizan **vacios** (<nav class="ocx-footer__nav ..." ></nav> , sin). Leyendo wp-includes/blocks/navigation.php esta escrito: ref es el id de un post de tipo wp_navigation y core reemplaza los inner blocks por el post_content de ese post. Ademas el unico wp_navigation que hay (id 5) tiene como contenido <!-- wp:page-list /--> , o sea que ni siquiera sirvia de ejemplo.

Lo que si funciona ya lo tenia el tema: inc/navigation.php inyecta un **menu clasico** como inner blocks de cualquier core/navigation cuya className contenga ocx-nav--menu , en un render_block_data . El problema es que estaba fijo a la location primary :

```
$location = get_nav_menu_locations()[ 'primary' ] ?? 0;
```

Tres cambios, todos en el tema:

1. **opencx_navigation_location()** (nuevo, en inc/navigation.php) resuelve la location a partir de la className : ocx-nav--menu → primary , ocx-nav--footer--a → footer_a , ocx-nav--footer--b → footer_b . Sigue siendo el unico punto de decision, asi que agregar un tercer menu al footer es una linea mas en el mapa de clases.
2. **register_nav_menus()** (en inc/setup.php) declara las tres locations. Las dos nuevas estan asignadas en la base, en theme_mods_OpenCX.nav_menu_locations : primary => 6 , footer_a => 7 , footer_b => 8 .

Esa asignacion vive en la base, no en el tema: si alguien clona el repo sin la base, los menus del footer salen vacios aunque las locations esten registradas. Es el mismo contrato que ya tenia `primary`.

- 3. Marcador nuevo para el footer.** Los navs del footer **no** pueden llevar `ocx-nav--menu : patterns.css:156` le pone `margin-left: 64px` (y 217px desde 1340) y `patterns.css:173` le fija `width: 532px` con `flex-wrap: nowrap` al `<ul>`, que es la medida de la fila de links del header. Con dos listas de 5 y 3 items eso las aplasta en una sola via de 532px. Por eso son `ocx-nav--footer--a / ocx-nav--footer--b`.

Con eso, los ocho items salen de Appearance → Menus y se pueden reordenar sin tocar markup: Platform, Solution, Industries, About, Insights (menu A) y Contact, Careers, Support (menu B). Se verificó que `overlayMenu: "never"` no genera el boton hamburguesa y que el header sigue saliendo del menu `primary` (primer item: Ecosystem, 42 `wp-block-navigation-item`).

width: 1440px pero centrado, no pegado a la izquierda

`.ocx-section--frame` ya hacia exactamente el ancho de 1440 del nodo, asi que se copia su parte `width: 1440px; max-width: 100%`, pero **sin** el pegado a la izquierda: ese helper deja el bloque contra el borde del viewport por encima de 1440, lo cual para un footer queda mal (en 1920 serian 480px vacio a la derecha). Se le agrega `margin-inline: auto`. Figma solo esta definido para 1440, asi que **elegir el centrado en 1920 es una decision**, no una lectura del nodo; queda anotada en Pendiente.

El padding si viene tokenizado como pidio la directiva: se reusa `.ocx-section ( var(--Section-Padding-padding-section-medium, 80px) / var(--Page-Padding-padding-global, 64px) )`, que ya resuelve el mobile a `48 / 20`. No se declaro en el atributo `style.spacing.padding` del bloque porque **core descarta ese atributo en silencio en los grupos con layout: flex** (mismo bug que documento CTA/25): si se deja solo en el Site Editor se ve un padding que no existe.

`align-items: center` (la directiva) y `alignItems: stretch` (el nodo) conviven: la raiz queda en `center` y su unico hijo lleva `align-self: stretch`, que es el `stretch` del nodo.

Las medidas del nodo, y la directiva que las cambio

El nodo define `Content` como `fill` y las dos `Column` tambien, o sea que se reparten el sobrante a partes iguales: $(1312 - 64) / 2 = 624$ cada una, y $(624 - 24) / 2 = 300$ cada lista de menus. Ese calculo es el de un container de 1312, que a su vez sale de $1440 - 64 - 64$.

La directiva del usuario cambio esa aritmetica: el `Content` va a medir **1280** (el tope del container, no los 1312 disponibles), la columna izquierda **816** y la derecha **400**, con `gap: 64` en el medio ($816 + 64 + 400 = 1280$, cuadra exacto). Las dos columnas dejan de ser `fill` repartido y pasan a `flex: 1 0 0` con `max-width: 400px` en la derecha: de los 1216px libres la derecha se queda con 400 y los 816 que sobran son para la izquierda. A 1440 la columna derecha arranca en $x = 80 + 816 + 64 = 960$ y cada lista de menus mide $(400 - 24) / 2 = 188$.

La directiva tambien fijo la altura de cada columna (**816x288** y **400x185**), el `gap: 32` de la izquierda, el `gap: 24` entre las dos listas y el `align-items: flex-start` de las tres, que en Figma estaba como `stretch`.

El **padding de la raiz no se toco**: sigue en `80 / 64` de `.ocx-section`, asi que la caja util de 1440 mide 1312. Lo que cambia es que el `Container` ahora se capea a **1280** (`--Container-container-large`, la misma medida del navbar) y se centra con su `margin-inline: auto` (mas el `align-items: center` de la raiz), de modo que cae en $x = 64 + (1312 - 1280) / 2 = 80$. Por eso el ancho pedido (1280) y el centrado se resolvieron sin tocar el padding que comparten las otras nueve secciones.

Cuatro correcciones que el layout de Figma exige y core no da

1. **align-items: stretch en Container**. Core pone `align-items: flex-start` en las columnas. Sin el `override`, `Content` se mide al contenido, y las dos columnas quedan al tamaño de su texto en vez de repartirse el ancho disponible.
2. **align-items: flex-start en Content**. Core pone `align-items: center` en las filas. La columna izquierda mide 288px y la derecha 185, así que centrarlas las separaría 48px, que es justo lo que Figma no hace.
3. **flex: 1 0 0 en las dos columnas** (y en las dos `Link List`). Sin esto las columnas se pegarían al contenido y el menú B no llegaría al borde derecho. El `flex-shrink: 0` es lo que evita que el reparto se desmonte en los anchos donde las dos columnas no entran.
4. **min-width: 320px en la columna izquierda**: un workaround de layout, no una medida del diseño. Ver la sección siguiente.

El divider necesita lo mismo: como la raíz es `align-items: center`, el `core/separator` se mide al contenido, así que lleva `align-self: stretch + width: 100%` y el borde explícito, en vez de depender de los `is-style-*` de core.

El `min-width` de 320px: un artefacto de Chrome que costaba 21px

Con la dirección real (1007 Shotgun Rd, Sunrise FL 33326, United States, 318.08px de ancho máximo) el footer media **603px** en vez de 582, con una franja vacía de 21px entre las columnas y el divisor. `Content` media 309px cuando la columna izquierda mide 288px, y no había ninguna caja en esa franja.

La causa no es el CSS: al repartir el sobrante, Chrome calcula la **altura de la línea de flex con el ancho intermedio** de la columna, que es su `max-content` (318px, el ancho de la línea de dirección), y no la recalcula después de que la columna crece a 816px. A 318px la dirección parte en dos líneas (21px de extra), y esa diferencia queda congelada. Medido forzando el ancho de la columna:

ancho de la columna izquierda	0	50	100	150	200-318	400	600	816
alto de la columna	486	486	372	330	309	288	288	288

`min-width: 320px` hace que esa pasada intermedia nunca llegue a partir la dirección, así que las dos alturas coinciden en 288px y el footer vuelve a 582. El ancho **visible** no cambia: la columna nunca baja de 332px con el breakpoint actual, así que el `min-width` de 320 nunca muere.

Se descarto lo contrario, con el mismo resultado 309: cambiar el `flex-shrink` a 1, cambiar `flex-basis` a `auto`, `align-content`, `flex-wrap`, `row-gap`, el `display` o la `height` de la columna derecha, y quitar los nodos de texto en blanco. Lo que si funcionaba era forzar el ancho final de la columna izquierda con `flex: 1 1 816px`, pero eso desbalancea a 768 (200/400 en vez de 332/332), así que se descarto. Ojo: si la dirección llega a medir más de 320px en una línea, el workaround hay que subirlo.

El logo: el frame es de 63px, la imagen de 35

`Company Logo` es un frame fijo de 148x63 con la imagen de 147x35 adentro. `core/site-logo {"width":148}` ya entrega `width="148" height="35"` (el PNG natural es 798x190), pero sin el alto de 63px el bloque mediría 35 y **Address** subía **28px**, moviendo todo el bloque de contacto. Se le da `height: 63px` con `align-items: center`. La imagen queda 4px más arriba que en el nodo, que la tiene descentrada (18px arriba contra 10px abajo); se centeriza en vez de copiar el desajuste.

Contenido: todo viene del usuario, nada es placeholder

El nodo dice `Address + Lorem ipsum`, con **dos** redes (Instagram y LinkedIn) y los links legales en `#`. Los datos definitivos los dio el usuario y se copiaron tal cual:

dato	valor	destino
Address	1007 Shotgun Rd, Sunrise FL 33326, United States	texto plano
Contact	info@intico.ai	mailto:info@intico.ai
Contact	+1 (786) 233 7216	tel:+17862337216
social	facebook.com/InticoCorp/	core/social-links
social	instagram.com/intico.ai/	core/social-links
social	tiktok.com/@intico.ai	core/social-links

Lo que quedo del nodo: los **tres** labels (Address , Contact , Social Links), el orden de los datos de contacto (correo antes que telefono, como en el nodo) y los tres links legales en # , porque no hay paginas que los respalden y no se invento ninguna ruta. Los servicios de las redes los resuelve el propio core/social-links ; tiktok esta soportado por core, asi que no hizo falta registrar nada.

Cambio de marca sin cerrar: los datos son de Intico, pero el logo sigue siendo el de OpenCX y el copyright sigue diciendo © 2026 OpenCX. All rights reserved. . El usuario no pidio cambiar ninguna de las dos cosas, asi que no se tocaron, pero conviven los dos nombres en el mismo footer.

Verificado (Chrome CDP)

10244:544 a 1440 da 80 + 422 + 80 = 582 de alto y **todo el arbol cae en el pixel:**

caja	Figma	medido
root	1440, padding 80 / 64	0,0 1440x582, padding 80 / 64
Container	1280, gap:80	80 1280x422
Content	1280, gap:64	80 1280x288
Column izquierda	816	80 816x288
frame del logo	148x63	148x63 (imagen 148x35.02)
Address	80 + 63 + 32 = 175	y = 175 desde el root
valor de Address	175 + 21 + 4 = 200	y = 200, 318.08 de ancho (una linea)
Contact	200 + 21 + 24 = 245	y = 245
social	344 , hug	y = 344, 96 de ancho (24 x 3 + 12 x 2)
Column derecha	400, x = 80 + 816 + 64	x = 960, 400x185
Link List A/B	188 cada una	960x185 / 1172x185
item de menu	21 + 8 + 8 = 37	37 de alto, padding: 8px 0 , 188 de ancho
Credits	448	y = 448, 1280x54
divider	1px , fill	1280x1, rgba(0,0,0, .15)
fila legal	space-between	80 → 1360, links terminando en 1360

Tipografia medida por getComputedStyle en los tres bloques: Manrope, 14px , line-height: 21px (1.5), 600 en labels e items de menu, 400 en los valores, margin: 0 , y text-decoration: underline en los links de contacto y legales.

Los cinco anchos, sin overflow en ninguno y con los dos menus completos siempre:

	1920	1440	1199	768	390
root	1440 (x = 240, centrado)	1440	1199	768	390

	1920	1440	1199	768	390
padding	80 / 64	80 / 64	48 / 20	48 / 20	48 / 20
root alto	582	582	518	518	756 (apilado)
Content	1280	1280	1159	728	350
columnas	816 / 400	816 / 400	695 / 400	332 / 332	350 apiladas
items de menu	8	8	8	8	8
hamburguesas	0	0	0	0	0
overflow	0	0	0	0	0

El footer tambien se midio en la otra plantilla (`index.html` , entrando por `/sample-page/`): mismo 582 de alto a 1440, mismos 8 items, `overflow` 0. Regresion de la home: `h1` 1 / `h2` 9 / `h3` 2, 32 imagenes sin ninguna rota, CTA intacto con `112 / 64 / 112 / 64` y el container en 768, carrusel intacto (4 dots).

A **320px** (por debajo del breakpoint, donde el container queda en 280) el footer **no** desborda: el `min-width: 0` de la media mobile hace su trabajo y ningun descendiente pasa del ancho del container. Los 30px de `overflow` que quedan en la pagina a ese ancho **no son del footer**, son de `ocx-h90__content` (297.7px dentro de 280) y del track horizontal de `ocx-t23__card` , que es preexistente de esas dos secciones y quedo fuera de este trabajo.

Pendiente

- **El aire antes del footer son 112px, y eso quedo cerrado.** El `core/spacer` de 112px de `front-page.html` se borro y el `padding-top: 80px` de la raiz se queda, asi que del boton al comienzo de la caja del footer hay 112 en los cinco anchos (y 192 / 160 hasta el logo). Ver Aire entre la seccion y el footer .
- **El logo y el copyright son de OpenCX y los datos de contacto son de Intico.** No se pidio cambiar ninguna de las dos cosas, asi que quedaron como estaban, pero hay que decidir si el logo, el texto © 2026 OpenCX. All rights reserved. y las redes pasan a Intico.
- **Los tres links legales van a #** . No hay rutas; hay que crearlas o confirmarlas.
- **En 1920 el footer queda centrado y en 1440 y menos llena el viewport.** El nodo solo define 1440 y con `align-items: center` de la raiz, centrar es lo coherente, pero si el resto del sitio se alinea a la izquierda por encima de 1440 (ver la nota de `.ocx-section--frame`), el footer va a quedar desalineado del header.
- **El breakpoint de apilado es 768px y no sale del nodo.** Abajo de 768 las columnas ($816 + 64 + 400 = 1280$) no entran, asi que `Content` y la fila legal se apilan. Con las columnas de 332 que quedan a 768 todavia entran, asi que siguen en fila: el Footer A mide 5 items y el B 3, y a 390 las listas miden 163 cada una.
- **El min-width: 320px de la columna izquierda depende del ancho del texto de la direccion.** Es un workaround para un artefacto de Chrome, no una medida del nodo: si la direccion pasa de 320px en una sola linea, hay que subirlo. Ver la seccion dedicada mas arriba.
- **Los locations footer_a y footer_b estan asignadas en la base, no en el tema.** Si el menu se desasigna desde Appearance → Menus, los navs se vacian en silencio (cae al `page-list` de core).

Shimmer de las preguntas de FAQ / 3 / — efecto hover "esta pensando"

No viene de Figma. Es una interaccion pedida a mano: al pasar el mouse por encima del titulo de cada industria, el texto se recorre con una banda de luz, como el indicador de que el agente esta pensando. Se aplica a los seis `.ocx-f3__question` .

Que se ve y por que el relleno va dentro de un @supports

Lo que se ve **no** es un overlay ni un `::after` encima: es el propio texto con un degradado recortado a la forma de las letras (`background-clip: text`) y el color puesto a `transparent` . Por eso sigue siendo texto de verdad, se

selecciona y un lector de pantalla lo lee igual.

El relleno a transparente va dentro de un `@supports` que exige **las dos** capacidades:

```
@supports ((background-clip: text) or (-webkit-background-clip: text)) and (color: color-mix(in oklab, red, blue)) {
```

La segunda condicion no es paranoia, es el fallo grave de esta tecnica. `color-mix()` es Baseline 2023; si un navegador recortara el fondo pero no resolviera el `color-mix()`, `--ocx-shimmer-band` seria un valor invalido, el gradiente entero seria invalido, no habria fondo que recortar y el texto ya estaria invisible. Con el color de texto declarado primero y solo pisado cuando se sabe que las dos cosas existen, ese navegador no ve nada del bloque y se queda con el texto normal.

Los dos colores salen de Color Scheme 1, asi que el tema oscuro no toca nada

En `tokens.css`:

```
--ocx-shimmer-band: color-mix(in oklab, var(--Color-Scheme-1-Text, #000000) 65%, var(--Color-Scheme-1-Background, #FFFFFF));
--ocx-shimmer-text: var(--Color-Scheme-1-Text, #000000);
```

El futuro tema oscuro solo tiene que invertir `--Color-Scheme-1-Text` y `--Color-Scheme-1-Background`, que ya tiene que hacer para el resto del theme, y el shimmer se recalcula solo. No hay bloque dark ni reglas que remember.

La banda se mezcla con el **fondo** y no con un blanco fijo a proposito: contra un blanco fijo, en dark el barrido seria invisible al no poder aclarar un texto que ya es blanco puro. Al mezclar con el fondo, el resultado es simetrico y el contraste se mantiene en los dos esquemas.

El recorrido va de `--ocx-shimmer-text` a `--ocx-shimmer-band` y de vuelta al texto, nunca al revés, por un motivo practico: en claro el texto negro se aclara al pasar la banda, y en oscuro el blanco se atenúa. En los dos casos la banda es el momento de **menor** énfasis de la palabra, que es justo lo que deja el shimmer legible en vez de apagarla.

Los numeros del recorrido

Por que el recorrido no es de 0% a 100%

La primera version no se veía, y el motivo no era el color. El fondo estaba en el H4, que mide lo que la columna, 732px, y el texto solo ocupa 324px. El fondo se recorta a las letras, pero **se mueve sobre la caja entera**: la banda entra y sale de los bordes de una caja que es mas del doble de ancha que las palabras. Con el recorrido completo de `0%` a `100%`:

- la banda tardaba **0.9s** en llegar al texto, o sea que al pasar el mouse no pasaba nada visible;
- y despues solo estaba encima el **35% del ciclo**, en destellos de 50 a 200ms.

Medido asi, el efecto entero pasaba desapercibido, que es exactamente lo que reporto quien lo probó. Por eso el recorrido se acota a la parte del ciclo donde la banda esta sobre el texto.

Los numeros actuales, y por que ya no dependen del copy

Ahora el portador del fondo es el `<a>` del link, no el H4. El H4 sigue midiendo 732px porque el borde superior de Figma tiene que llegar de borde a borde de la lista, pero el `<a>` es en linea y se ajusta a su contenido: su caja es

el texto. Medido, [644, 62, 324, 49] : los mismos 324px de letras que ocupaba dentro del H4, ahora como caja propia.

Eso convierte los numeros en constantes. La posicion de la banda sigue siendo $x = 1.4 - 1.8p$ (en fracciones de la caja, con p el `background-position`), pero como el texto va de 0 a 1.0 de la caja en vez de a 0.44, y la banda de 10% sobre 280% de fondo mide 28% de la caja (la mitad, 14%), la banda esta sobre el texto entre $p = 14\%$ (borde izquierdo) y $p = 86\%$ (borde derecho). De ahí los dos keyframes:

keyframe	background-position	que pasa
0%	86%	la banda entra por la izquierda, el texto ya esta en el frame 1
58%	14%	cruza las palabras y sale por la derecha
100%	14%	pausa con el texto solido

Estos numeros dependen **solo** de `background-size` y del ancho de la banda. Antes iban atados a la proporcion texto/caja, que ademas cambiaba con el ancho de la ventana, asi que habia que remedirlos cada vez que se tocaba el copy o el layout. Si se cambia el ancho de la banda, el 280% o los stops, hay que rehacer la cuenta, y sale sola.

Diferencia medida entre la version invisible y la actual, con la geometria real de los cinco anchos:

Con el portador en el `<a>`, el %caja del texto da **0.00..1.00 en los cinco anchos**: la caja es el texto, y por eso el duty ya no sale por el ancho de la ventana.

ancho	caja del <code><a></code>	texto	1er cambio	banda sobre el texto	pasada visible
1920	324	324 (100%)	8ms	58% del ciclo	1266ms
1440	324	324 (100%)	9ms	58% del ciclo	1266ms
1199	216	216 (100%)	13ms	58% del ciclo	1267ms
768	216	216 (100%)	12ms	58% del ciclo	1266ms
390	216	216 (100%)	10ms	58% del ciclo	1267ms

El primer cambio quedo en **8-13ms**, o sea que el efecto arranca en el frame en que el mouse entra. La pausa del 42% del ciclo es deliberada: es lo que hace que se lea como un destello y no como un texto que late.

Lo que cambia frente a la version con el H4 como portador es que el **duty se vuelve plano**: antes era 100/42/36/45/100 segun el ancho, con el 1199 a 36% por ser el caso feo, porque ahí el texto era solo el 19% de la caja. Ahora son 58% en los cinco, y la pasada visible mide 1266-1267ms con una dispersion de 1ms. La banda esta encima el 100% del recorrido, sin zonas muertas, porque los dos keyframes se calcularon con la proporcion real de esa caja.

Cuanto se nota: tres iteraciones, y el color es el que mas pesa

El efecto se ajusto tres veces segun lo que reporto quien lo probó, y las tres variables actuan en lugares distintos:

version	banda	contraste	ancho de banda	ciclo	pasada	delta medido	como se veia
1	#3A3A3A (65%)	11.37:1	68px	1.6s	0.53s	+75	nada : 0.9s hasta el primer cambio
2	#4D4D4D (58%)	8.45:1	68px	1.6s	0.53s	+75	se veia, pero timido
3	#808080 (40%)	3.95:1	96-122px	2.2s	0.78-0.98s	+126	se nota claramente

version	banda	contraste	ancho de banda	ciclo	pasada	delta medido	como se veia
3b	#808080 (40%)	3.95:1	46-52px pleno, 60-68px total	2.2s	1.27s	+119	se nota claramente

El salto de 65% a 58% no cambio nada porque el problema no era el contraste: era que la banda tardaba 0.9s en llegar. Arreglado eso, el siguiente salto que si se noto fue el de color **y** de ancho **y** de velocidad juntos.

El color tiene un piso, y es 3:1. El texto es un H4 de 36px, o sea texto grande, donde el umbral de AA es 3:1 y no 4.5:1. Por eso 3.95:1 cumple. #808080 sobre #FFFFFF da exactamente eso. No bajar de 40% de mezcla: por debajo de ~3:1 el trazo empieza a costarse leer durante el paso, y el efecto deja de ser un brillo para volverse un fallo de legibilidad. Un brillo que no se puede leer no vale la pena.

El ancho importa mas de lo que parece. Lo que se percibe es la parte a plena intensidad, no el total. Con la banda al 8% del gradiente esa parte media 68px; al 10% son 96-122px con el portador en el H4, y **46-52px a plena intensidad con 60-68px de banda completa** con el portador en el `<a>`, que es donde quedo. El ancho de banda medido en pixeles baja al cambiar de portador porque el ancho relativo de la banda es respecto a la caja, y la caja paso de 732px a 324px; lo que no baja es el delta de +119, o sea que el recorrido del gris sigue siendo Entero y visible. Un cambio de dos puntos en los stops del gradiente se ve mas que un cambio de siete puntos en el contraste, porque el ojo integra sobre area y no sobre un punto.

La deuda de que los keyframes dependieran de la proporcion texto/caja: resuelta

Mientras el portador fue el H4, los keyframes dependian de que el texto ocupara una fraccion concreta de la caja, y esa fraccion cambiaba con el ancho de la ventana. Por eso en su momento **no se metio un `<span>` alrededor del texto**: un span en linea si se ajusta al texto, pero `core/heading` serializa el contenido como texto plano, asi que en cuanto alguien editase la pregunta desde el Site Editor el span se perdia y el efecto se rompia sin avisar.

La salida fue la que ya estaba pedida por el diseno: **las preguntas son links**, y el link es un `<a>` en linea, o sea exactamente el elemento que hace falta, y ademas es un RichText valido de `core/heading`, o sea que **sobrevive a la edicion en el Site Editor** a diferencia del span. Con el link puesto, la caja es el texto, el duty se puso plano y los 84% / 47% se sustituyeron por 86% / 14%, que ya no dependen ni del copy ni del layout.

Los numeros no son 0% - 100% como se chegou a pensar: con la caja siendo el texto, la banda esta sobre el entre `p = 14%` y `p = 86%`, asi que 0% - 100% habria metido de nuevo las zonas muertas en los dos extremos, solo que mas cortas. La cifra no es la del rango completo, es la del rango util.

Se anima `background-position`, que no se compone en GPU y repinta por frame. Aqui sale barato porque solo hay una linea de 36px y solo mientras hay hover, pero queda anotado por si el patron se copia a algo mas grande.

La seleccion

Con el relleno transparente, lo seleccionado se ve como un bloque de color sin letras dentro. El arreglo va dentro del mismo `@supports` y **no** puede usar `currentColor`, porque ahi `currentColor` ya es el propio `transparent`:

```
.ocx-f3__question a::selection {
  -webkit-text-fill-color: var(--Color-Scheme-1-Text, #000000);
}
```

El sujeto es el `<a>`, que es quien lleva el relleno transparente. Con el portador en el H4 hacia falta el `::selection` del propio H4; ahora el relleno solo existe en el link, asi que la regla descendente tambien.

Reduced motion: la banda se congela, no desaparece

```
@media (prefers-reduced-motion: reduce) { ... animation: none; background-position: 50% 0; }
```

El hover sigue cambiando el aspecto del texto, pero ya no hay animacion. Es un cambio de color y no un desplazamiento, asi que no es lo que la preferencia pide evitar; quitar el efecto entero habria dejado el `:hover` sin ninguna respuesta visible.

El 50% es el centro exacto de la caja, y como la caja es el texto, es el numero que siempre se esperaria. Cuando el portador era el H4 hacia falta un 66% porque la caja era mas ancha que el texto y el 50% dejaba la banda a un lado de las palabras; con el portador en el `<a>` eso ya no aplica.

El hover y el foco comparten el mismo disparador

```
.ocx-f3__question a:is(:hover, :focus-visible) { ... }
```

Solo el link. Antes estaba el H4 tambien, por el caso de que todavia no fuera un link; ahora que lo es, tener los dos disparadores pintaba el gradiente dos veces a la vez, una sobre la caja del H4 y otra sobre la del link, con dos animaciones desfasadas encima. Se dejo un unico portador.

`:focus-within` ya no hace falta y se quito: el link dispara con su propio `:focus-visible`, que es lo que checked. Verificado con `a.focus()` real: el `background-clip` y la animacion arrancan igual que con el mouse.

Verificado (Chrome CDP)

- **Contraste de la banda:** `--ocx-shimmer-band` resuelve a `oklab(0.60 0 0) = #808080`, que son **3.95:1** sobre `#FFFFFF`. El texto en reposo son 21:1. El umbral que le toca es el de **texto grande, 3:1**, no el normal de 4.5:1, porque la banda cae sobre un H4 de 36px. El test lee el `font-size` real y aplica el umbral que corresponde en vez de hardcodear 4.5, para que no se rompa en silencio si alguien baja el tamaño de la fuente.
- **La banda entra ya sobre el texto, medido por pixeles.** Con la animacion pausada en instantes fijos (`animation-play-state:paused` + `animation-delay` negativo, para que cada captura sea determinista y no una carrera) y el raton realmente encima: la caja del `<a>` mide 324px y el texto tambien 324px, y la banda aparece ya a **8-13ms** del hover. Entra por la izquierda, cruza y sale por la derecha en $\sim 1.27s$.
- **El pico del degradado se ve entero:** el delta contra el reposo llega a **+119 de 255**, y el token vale 128 (`#808080`). La banda completa mide 60-68px de los 324 del texto y la parte a plena intensidad 46-52px, y a lo largo de las capturas se la ve avanzar de `x=666` (borde izquierdo) a `x=962` (borde derecho).
- **En reposo no hay fondo, ni animacion, ni cambio de color:** `background-image:none`, `animation-name:none`, `color:rgb(0, 0, 0)`. El bloque solo existe sobre el hover.
- **El reinicio del bucle no se ve:** entre el 58% y el 100% del ciclo el `background-position` se queda en 14%, donde la banda ya esta fuera del texto, asi que no hay salto; la pasada y la pausa son continuas. Los dos extremos pintan las palabras enteras en negro.
- **Con `prefers-reduced-motion:reduce`:** `animation-name:none` y `background-position:50% 0`.
- **Foco por teclado:** `a.focus()` sobre el link real dispara el mismo `background-clip` y la misma animacion que el hover.
- **Los seis links, verificados en el DOM:** los seis `h4.ocx-f3__question` contienen un `<a>` con `href="/industries/"`, el link envuelve el texto completo del H4 (el `>` incluido), `text-decoration-line:none` y `color:rgb(0, 0, 0)` heredado. El heading sigue siendo H4 en los seis, y la regla `.ocx-f3__question a::selection` existe dentro del `@supports` y matchea a los seis. Nota al medir: `Element.matches()` devuelve `false` con un selector que trae pseudo-elemento, asi que para comprobar esa regla hay que quitarle el `::selection` antes de hacer el `matches`.
- **Regresion:** 1920 / 1440 / 1199 / 768 / 390 sin cambios, 0 de overflow, 13 secciones, 32 imagenes, 0 rotas, consola y requests limpios. El efecto no toca la geometria: solo existe con el mouse encima.

- Herramientas de medicion en `/var/folders/rl/ppqp_22531g26852pt5d38_m0000gn/T/opencode/` : `shimmer_test.py` (tokens, hover, teclado, reduced motion), `shimmer_duty.py` (muestrea `backgroundPosition` con `rAF` y calcula el tiempo real sobre el texto en los cinco anchos, sin decodificar nada), `shimmer_pausa.py` (pixeles con la animacion pausada en instantes exactos), `shimmer_shots.py` y `png_profile.py` (decodifica el PNG sin PIL, porque no hay PIL instalado).
- **Dos trampas del entorno, para no volver a perder tiempo:** `Page.captureScreenshot` con `clip` devuelve imagen **en blanco** en este Chrome, asi que hay que capturar el viewport entero y recortar leyendo pixeles. Y decodificar el PNG a mano en Python cuesta ~7s por frame a `deviceScaleFactor:2`, asi que para medir tiempos hay que usar `shimmer_duty.py` (`rAF`) y para medir pixeles hay que pausar la animacion con `shimmer_pausa.py`.

Pendiente

- `/industries/` **sigue en 404**. Los seis links ya apuntan ahi y el boton `Explore Industries` de la misma seccion tambien, asi que el 404 esta pendiente de la pagina, no del markup. Cuando exista, no hay que tocar ni el template ni el CSS: los `href` ya son correctos. Si la pagina se crea con otro slug, hay que cambiar los seis `href` y el del boton, que hoy comparten destino.
- **El > de cada pregunta se recorre con el texto**. Es el mismo nodo de texto, y partirlo en un `<span aria-hidden="true">` es churn de markup para un ganho chico. Queda anotado por si se nota raro al verlo en vivo.
- **En dark el shimmer va a atenuar en vez de aclarar**, porque el texto blanco ya esta en el extremo y la banda no lo puede aclarar mas. Es el comportamiento correcto de un shimmer, pero es un cambio de look respecto del light y conviene mirarlo cuando exista el tema.
- **Los keyframes ya no dependen de la proporcion texto/caja**, asi que el caso feo del 1199 (33% de duty) desaparece: ahora son 58% en los cinco anchos. Lo que si queda por vigilar es el copy: si una pregunta pasa a **dos lineas**, el `<a>` se parte en dos cajas en linea y cada una lleva su propia animacion, con el barrido empezando a la vez en las dos. Se mide con `shimmer_duty.py`: si `text0..text1` deja de dar `0.00..1.00` o el duty baja de ~58%, el copy crecio.
- **Si se cambia el ancho de la banda, el 280% o los stops**, hay que rehacer la cuenta de la derivacion de arriba, porque los dos keyframes salen de ahi. Con banda de 10% sobre 280% son `86% / 14%`; con 8% pasarian a ser `84% / 16%`.

Navbar / 1 / — la navbar de Ecosystem, que es otra navbar

La primera pagina nueva (`Ecosystem`, id 61) se abre con una navbar que **no es la que ya teniamos**. En Figma es el nodo `10227:110690`, instancia de `Navbar / 1 /` (`10227:110689`), y se ve igual en la home, asi que el componente es compartido y lo que cambia es que `Ecosystem` es la primera pagina que lo monta. No es un retoque: mide 92px de alto donde la otra mide 160, y sus links van en 16px donde los otros van en 18.

- **La pagina se creo con `wp_insert_post()`**, no con `wp-cli`, porque no hay `wp-cli` dentro de `wp-env` ni cliente de `mysql`. Se lanzo un PHP de un solo uso por `docker exec ... php` contra `/var/www/html/wp-load.php` y se borro despues. Con el slug `ecosystem` basta con que exista `templates/page-ecosystem.html`: WordPress lo asocia solo, sin guardar nada en la BD.
- **La fila va al fondo de los 92px, no al centro**, porque el frame exterior es `flex-direction: column` con `justify-content: flex-end`. El `padding` es el de pagina (`--Page-Padding-padding-global`, 64px), no el `--Header-padding` de 80.5px de la otra.
- **Dentro, "Container" (#10218:66544) es `space-between` y se estira con `align-self: stretch`** — hace falta explicito, porque el padre esta en `align-items: center` y sin el la fila se encogeria al contenido en vez de cruzar los 1312px. "Column" (#10218:66547) es el grupo de la derecha con `gap: 32px` entre los links y las acciones. Los 32px del gap salen dos veces, una vez en el bloque y otra vez en el CSS, porque core aplica `--wp--style--block-gap` y el reset de abajo necesita ganarle.

- **Logo:** slot de 157x42, que es lo que ya daba la regla global. El nodo declara 158x42 para la instancia y 147x35 para la imagen; el 147 del ancho de la imagen ya venia de `site-logo` y el alto de 35 lo impone el `max-height` de la regla global. No se toco nada de eso.

Los resets que hacen falta, y por que la otra navbar no se entera

La navbar que ya existia compone sus links con dos decisiones que aqui no sirven:

- un carril fijo de **532px** con `padding-right: 33px`, `gap: 26px` y `align-items: flex-start`, y
- un desplazamiento de **217px** (`margin-left`) que pone la columna de links en el centro de la barra.

Las dos existen para llenar los 1280px de un header de 160px. En una de 92px, con el logo a la izquierda y `space-between` empujando, los links se pegan al borde derecho y el carril fijo dejaria un hueco de 40px a la derecha. Los dos resets viven bajo `.ocx-header--ecosystem`, con `.wp-block-navigation__container` a `(0,3,1)` contra el `(0,2,1)` de la regla original, asi que ganan sin depender del orden en el archivo. La home sigue en 160px con sus 2 botones: nada de lo que se escribio la alcanza.

El empate de especificidad del boton

`.ocx-header--ecosystem .ocx-btn--primary .wp-block-button__link` y `.wp-block-button.ocx-btn--primary .wp-block-button__link` son las dos **(0,3,0)**, y la segunda esta mas abajo en el archivo, asi que ganaba el `padding: 10px 24px` de siempre y el boton se quedaba en 120x44 en vez de 120x42. Todo lo demas de la regla si aplicaba, porque ahi no habia empate: `border`, `color` y `font-weight` solo estan en un lado. **La trampa es que el fallo se ve solo en una de las cuatro declaraciones.**

La regla tiene que llevar `.wp-block-button` para subir a `(0,4,0)`, porque es la clase que la regla original usa y a la que le falta la mia. `CSS.getMatchedStylesForNode` es la forma de verlo sin adivinar: devuelve las reglas que matchean con su origen, y ahi se ve el empate.

Los otros tres valores que cambian, por que Figma y la otra navbar discrepan:

	Figma	la otra navbar
padding	8px 20px	10px 24px
texto	#FFFFFF	--Color-Spring-Wood
font-weight	500	300

El fondo no hace falta tocarlo: `--Color-Husk` ya es `#B69E52`, el mismo dorado que Figma. El borde es una segunda declaracion del mismo color, y con el relleno detras solo se lee como un filo interior de 1px a este tamano.

El caret de Ecosystem : decorativo, y por que no puede ser un submenu

Core solo dibuja chevron donde hay submenu de verdad, porque ahi el boton abre algo. Figma pone uno en `Ecosystem`, que en el menu no tiene hijos: es decorativo.

- **No se puede marcar como `core/navigation-submenu`**, porque eso convierte el item en un `<button>` toggle que abre un submenu vacio. El unico workaround de core seria un submenu con un hijo invisible, que es peor.
- Se reutiliza **el mismo SVG y la misma clase** de core:
`block_core_shared_navigation_render_submenu_icon()` dentro de un `<span class="wp-block-navigation__submenu-icon">`. Esa clase es la que aporta la caja de `0.6em`, el `margin-left: 0.25em` y el `stroke: currentColor`. Reutilizarla significa que el caret de "Ecosystem" es el mismo glifo y del mismo color que el de "Industries", sin CSS nuevo que se pueda desincronizar.

- **El `margin-left: 0.25em` da exactamente los 4px del gap que declara el frame "Link"** (EL-67fa1d50): `0.25` de un link de 16px son 4px. No hubo que ajustarlo.
- **El `svg` trae `aria-hidden="true"`**, así que el nombre accesible del link sigue siendo "Ecosystem" y el caret no se anuncia. Verificado: `textContent.trim()` del `<a>` es "Ecosystem" y no es un botón.
- **Va con una clase propia, `ocx-nav--caret`, no con la `primary` entera.** Las páginas que todavía montan el header viejo no han sido migradas, y un caret que solo existe en el diseño nuevo no debería aparecer en el viejo. Sumar la clase al bloque de `parts/header.html` es todo lo que haría falta para hacerlo global; el filtro ya lee la clase con `str_contains`, así que `ocx-nav--caret` resuelve igual que `ocx-nav--menu` a la hora de ubicar el menú.

El filtro va en `render_block`, no en `render_block_data`: este último ya se usa para inyectar el menú, pero ahí solo llegan los atributos y el contenido HTML de los links lo escribe `render_block_core_navigation_link`. El atributo `ocxCaret` es desconocido para core, que lo ignora sin quejarse, y se filtra por `empty( $block['attrs']['ocxCaret'] )`.

Dos cosas que parecen fallos y no lo son

- **El caret del overlay móvil mide 0x0.** No es el nuestro: core tiene `.wp-block-navigation__responsive-container.is-menu-open:where(:not(.disable-default-overlay)) .wp-block-navigation__responsive-container-content .wp-block-navigation__submenu-icon { display: none }` y oculta **los tres** carets, el decorativo y los dos de submenu real. Medido: los tres dan 0x0 con `display: none`. No hace falta un override, y no habría que hacerlo: si el caret de "Ecosystem" apareciera ahí y los de "Industries" y "About" no, sería el elemento raro.
- **El caret es negro.** Figma lo declara en `fill_34dc0314` (#000000), el mismo relleno que el texto del link, así que `stroke:currentColor` resolviendo a negro es lo correcto. El link mide `rgb(0, 0, 0)`.

El tamaño del caret: 9.6px contra 10px, y por que se deja

Figma pone el ▼ en `ts1 / ts3`, que son `fontSize: 10`. La caja de core es `0.6em`, y a 16px son **9.6px**. Los 0.4px se dejan a propósito: fijarlo en 10px obligaría a una regla propia y **el caret decorativo se vería 0.4px más grande que el de "Industries" y el de "About"**, que salen de la misma clase. Que los tres midan lo mismo pesa más que una fracción de píxel que no se ve.

Verificado (Chrome CDP)

	Figma	medido
navbar	1440x92, 0 64px, column, flex-end, #FFFFFF	92px, 0px 64px, column, flex-end, rgb(255,255,255)
container	space-between, center, stretch	space-between, center, 1312px de ancho
logo	157x42	157x42
column gap	32px	32px
gap de links	32px	32px
links	hug, 16px/400, 1.5em	489x24, 16px / 400 / 24px
botón	8px 20px, borde 1px #B69E52, #FFFFFF, 500	8px 20px, 1px solid rgb(182,158,82), blanco, 500, 120x42

- **1440x92 exacto, y el 1441 es la convención del tema**, no un descuido: la navbar base mide 1441 a propósito para no dejar 1px de scroll horizontal.

- **Regresion de /ecosystem/** en 1440 / 1200 / 1199 / 1024 / 768 / 390: 92px de alto en los seis, 0 64px hasta 1200 y 0 20px por debajo, **0 de overflow**, logo presente, **1 solo boton** ("Let's meet", sin "Sign In", como en Figma).
- **La home no se movio:** 160px, 0 80.5px, 2 botones, y sin logo a 1440 (la intro de JS lo borra a proposito). Sus carets siguen siendo 2; los de /ecosystem/ son 3.
- **Teclado:** el orden es logo -> Ecosystem -> Solution -> el boton de "Industries" -> Insights -> el boton de "About" -> "Let's meet", con outline: 1px en los siete. El caret no mete un tab stop propio, que es justo lo que se queria al no ser un boton.
- Consola y requests limpios en /ecosystem/.
- **Al medir el overlay hay que usar `getBoundingClientRect()`, no `offsetParent`**: el contenedor del menu es `position: fixed`, asi que `offsetParent` da null y parece oculto cuando esta abierto. Y el selector del caret no puede exigir que este **dentro** del `<a>`, porque el de un submenu real es hermano del `<a>`, no hijo: asi se cuenta 1 en vez de 3.

Pendiente

- **Solo el header.** El resto de Ecosystem sigue en Figma como placeholder ("Ecosystem.") dentro de `ocx-eco__body`; la pagina tiene 12 bloques en el nodo 10218:66032, y el siguiente es un Header / 62. El placeholder es deliberado, para que la pagina exista y el header se pueda mirar por separado.
- **El caret se puede volver global** con `ocx-nav--caret` en el bloque de navegacion de `parts/header.html`, sin tocar PHP. La decision esta tomada solo por no cambiar paginas que todavia no estan migradas.
- **/ecosystem/ no esta en ningun menu todavia**, porque el item ya existe en el `primary` que se creo antes: el enlace es real y la pagina existia en 404. Con esto ya responde.
- **justify-content: center en "Column"** es lo que dice Figma, pero como el frame es `hug` no tiene efecto visible; si el grupo llega a tener `fill` en horizontal, habria que revisarlo.
- **Sin cache busting a mano:** la hoja se sirve con el hash de `filemtime` que ya usa el tema.

Header / 62 / — el hero de Ecosystem, con el texto centrado

Segundo bloque de la pagina Ecosystem, el nodo 10218:66853. Es el primero de los doce de Platform • Desktop (10218:66032) que no es la navbar, y el primero donde el copy **centrado** cambia el criterio: los dos text styles que usa declaran `textAlignHorizontal: CENTER`, y todo lo demas de la pagina es alineado a la izquierda.

El arbol de Figma tiene cinco niveles — seccion, Container, Component, Section Title, Content — para holdar un H1 y un parrafo. Los cinco estan en el markup, con su clase y su gap, aunque dos gaps sean inertes (ver abajo).

La caja sale de la cadena .ocx-section que ya existia

No hacia falta escribir una seccion nueva. De los siete valores que da el nodo:

valor	de donde sale
<code>align-self: stretch</code>	<code>.ocx-section--stretch</code>
<code>padding: 112px 64px 16px</code>	<code>.ocx-section--large</code> + una regla (0,2,0)
<code>background: #FFFFFF</code>	<code>.ocx-section</code>
<code>flex-direction: column</code>	<code>layout.orientation</code> en el atributo
<code>gap: 80px</code>	<code>style.spacing.blockGap</code> en el atributo
<code>align-items: center</code>	<code>layout.justifyContent</code> en el atributo

El padding va con dos clases, `.ocx-section.ocx-h62__section`, y no confiando en el orden. `.ocx-section` y `.ocx-section--large` son (0,1,0) las dos, y los tokens saltan a valores de telefono dentro de `@media` (`max-width: 1199.98px`) en `tokens.css` (`--Section-Padding-padding-section-large: 64px`, `--Page-Padding-padding-global: 20px`), con lo que la cadena sola bajaria la seccion a **64/20** por debajo de 1200. Con (0,2,0) gana en cualquier ancho y sin importar el orden.

Los 112/64/16 quedan **fijos en todos los anchos** porque Figma no define frame movil para Platform • Desktop . Es el mismo criterio que `.ocx-c25__section` y que los 55px del H1. Decidido por el usuario cuando se le planteo el coste: a 390px la seccion queda en 714px de alto con el H1 partido en 4 lineas y el parrafo en 10.

El bottom es 16px, no 112px, y es el unico valor del nodo que se aparta de el. El nodo declara `padding 112px 64px`; el bottom en 16 lo pidio el usuario, asi que el shorthand queda en tres valores, `112 / 64 / 16`. Como el hero va pegado a "Layout / 71 /", que abre con 60px de top, lo que se busca es que el aire entre las dos secciones sea el del bloque de abajo y no el del hero: de 172px a **76px** (`16 + 60`), medido del borde inferior del boton al superior del tagline en los siete anchos. El alto del hero baja **96px exactos**, que es `112 - 16`.

Recordatorio que ya estaba en el BACKLOG y que reaparece: **style.spacing.padding en el atributo del bloque no sirve en un `.ocx-section`**, core lo descarta en silencio y el rendered llega con `style="margin-top:0;margin-bottom:0"` y sin padding. Solo el hero de la home, que es `layout: default`, recibe el suyo. El padding tiene que estar en el CSS.

El bug del hero: en un grupo vertical, core invierte los dos

`Component` (#10218:66855) es `alignItems: center`, y eso es lo que centra el boton bajo el texto. La primera version lo decia con `layout.horizontalAlignment`, que **core ignora en silencio**: no hay error ni aviso, el grupo simplemente se queda con el `align-items: flex-start` de la linea 808 de `block-supports/layout.php`, que es el default cuando no hay `justifyContent`. Con ahi "Let's meet" quedaba pegado a `x=64`.

La rama `else` de ese archivo (linea 794, la de `orientation: vertical`) mapea al revés:

- `layout.justifyContent` -> **`align-items`**
- `layout.verticalAlignment` -> **`justify-content`**

y `horizontalAlignment` no aparece en esa rama. El `center` va por `justifyContent`.

`align-self: stretch` y `align-items: center` no se contradicen: en flexbox el `align-self` siempre le gana al `align-items` del padre para ese item, asi que el Title sigue cruzando los 1280px del Container y lo que se centra es el Actions. Es el mismo reparto que ya hace `.ocx-c25__section` con su Container.

El tope de 1280 va en el Container, y el centrado necesita una regla aparte

`Container` (#10218:66854) trae `max-width: var(--Container-container-large, 1280px)`. El token existe en `tokens.css` y **no cambia en movil** (esta fuera del `@media`, asi que las dos declaraciones valen 1280), o sea que el tope solo muere de 1408px para arriba:

ancho	contenido de la seccion	Container	sobra
1920	1792	1280	256 a cada lado
1440	1312	1280	16 a cada lado
1408	1344	1280	0, tope justo
1200	1072	1072	0, el tope ya no aplica
1024	896	896	0, el tope ya no aplica

El `margin-inline: auto` va en una regla aparte, `.ocx-section > .ocx-h62__container`, y no en la del **Container**. Es el mismo truco de `.ocx-c25__container: .is-layout-flex > :is(*, div) { margin: 0 }` es (0,1,1) y le gana a un `margin-inline: auto` pelado, que además es (0,1,0) y computa a `0px/0px`. Con el `.ocx-section >` delante la regla llega a (0,2,0) y el `auto` si absorbe el sobrante. A 1920 computa `margin-left: 256px` y a 1440 `16px`.

El `width: 100%` va con el `align-self: stretch` porque el `stretch` de flexbox solo aplica "if neither of the cross-axis margins are auto", y el centrado de arriba necesita `auto`. `min-width: 0` evita que una palabra larga del H1 empuje el Container en vez de partirlo.

El `align-items: center` del Container es el cuarto `center` de la cadena

Llega por `layout.justifyContent` en el atributo del bloque, igual que los otros tres, y no por CSS. Como el Component tiene `align-self: stretch`, sigue llenando los 1280 igual.

El bloque de texto va a 768, y por eso el parrafo sube a 4 lineas

Content (#10218:66859) lleva `max-width: var(--Container-container-small, 768px)`, o sea el token `--Container-container-small` y no el `large` del Container. Son dos toques anidados e independientes, y cada uno muerde en su franja:

	Container	Content	parrafo	seccion
1920	1280 (tope)	768 (tope)	4 lineas	393
1440	1280 (tope)	768 (tope)	4 lineas	393
1200	1072 (lleno)	768 (tope)	4 lineas	393
1024	896 (lleno)	768 (tope)	4 lineas	381
768	640 (lleno)	640 (lleno)	4 lineas	460
390	262 (lleno)	262 (lleno)	10 lineas	714

El Content mide 768 exacto de 832px para arriba, que es `64 + 768 + 64` de padding mas el tope. El `margin-inline: auto` va otra vez en regla aparte, `.ocx-h62__title > .ocx-h62__content`, por el mismo (0,1,1) > (0,1,0) de `.is-layout-flex > :is(*, div) { margin: 0 }`. Computa 256px a cada lado a 1920 y a 1440, 152px a 1200 y 64px a 1024, o sea $(\text{ancho del padre} - 768) / 2$ siempre que el padre sea mas ancho.

El `width: 100%` hace el trabajo del `align-self: stretch` cuando hay tope. El `stretch` de flexbox solo aplica "if neither of the cross-axis margins are auto", y el centrado necesita `auto`, así que con los márgenes puestos el `align-self` queda inerte y el ancho lo fija el `width: 100%` topado por el `max-width`. Se dejan los dos, que es el idioma que ya usa `.ocx-footer__container`.

El `align-items` de Title y Content es `center`, y por eso no se ve

Section Title (#10218:66856) y Content (#10218:66859) son los dos `align-items: center`, y llega igual que el del Component: por `layout.justifyContent` en el atributo del bloque, no por CSS. Los dos necesitan `align-self: stretch` para seguir cruzando los 1280px, porque en flexbox el `align-self` le gana siempre al `align-items` del padre para ese item.

El reparto completo queda así:

```
seccion      align-self:stretch  align-items:center  <- el unico center que si centra algo
Container    align-self:stretch  align-items:center  <- 4to center, y ademas tope de 1280
Component    align-self:stretch  align-items:center  <- centra el Actions
```

Title	align-self:stretch	align-items:center	
Content	align-self:stretch	align-items:center	<- centra el H1 y el parrafo

Con `center`, el centrado sale del `align-items` y no del `text-align`; con `stretch`, saldria del `text-align`. Con el copy actual los dos caminos ponen el texto en el mismo pixel, asi que el cambio es invisible. Medido con los rects de las lineas de texto, no de las cajas, porque las cajas si cambian:

	center (Figma)	stretch
caja del H1	754.39px (su max-content)	768px
caja del parrafo	768px (topado)	768px
linea 1 del H1	x=342.80 der=1097.19	x=342.80 der=1097.19
alto de la seccion	393px	393px

Los glifos estan en el mismo pixel y el alto no se mueve. Lo unico que cambia es el ancho de la caja del H1, que con `center` pasa a medir lo que ocupa en vez de todo el contenedor: mas fiel al nodo, y sin coste.

55px es un literal, y no hay token que lo cubra

Heading/H1 es Manrope 500 55px con -0.01em. El token real `--Text-Size-Heading-1` es 72px en escritorio y 40px en movil, asi que no hay nada que aliasear y el literal tiene que quedarse en el CSS. El parrafo si usa el token: `Text/Medium/Normal` es 18px y `--Text-Size-Text-Medium` es exactamente 18px.

El tipo va con **selectores de hijo** (`.ocx-h62__content` `.wp-block-heading` y `.ocx-h62__content p`) y no con clases en los bloques, para que el copy siga siendo un heading y un parrafo core planos que cualquiera pueda reescribir desde el editor sin arrastrar una clase. Es el criterio que ya usan `.ocx-h90__card` y `.ocx-l71__col`.

Los dos `gap` inertes, y por que se declaran igual

`Container` tiene `gap: 80px` y `Section Title` tiene `gap: 16px`, y los dos son inertes: cada uno tiene un solo hijo. Se dejan declarados en el atributo del bloque para que el editor muestre la estructura del archivo, que es el criterio que se viene usando.

`.ocx-btn--primary` no es el componente Button de Figma

El nodo usa el componente `Style=Primary, Small=False (#10218:66863)`: relleno `#B69E52`, borde 1px del mismo color, label en blanco y "Text/Regular/Medium" (peso 500), `padding 10/24`. `.ocx-btn--primary` coincide en el relleno y en el `padding`, y difiere en **tres de cuatro**: `border: 0`, label en Spring-Wood y peso 300.

Esto no es solo de este hero. Los siete botones primary de la home tienen los mismos tres desvios, y tambien la navbar de `Ecosystem`. Los dos usos que el componente tiene hoy se resuelven con una sola regla para las tres declaraciones compartidas:

```
:is(.ocx-header--ecosystem, .ocx-h62__actions) .wp-block-button.ocx-btn--primary .wp-block-button__link
```

El `padding` queda fuera de esa regla a proposito, porque es lo unico que separa las dos variantes: `Small=True` (la navbar) es 8/20 y `Small=False` (el hero) es 10/24. Ese 8/20 de la navbar tambien explica por que su boton **si** necesita una regla propia: `.ocx-btn--primary` trae 10/24, o sea el valor del hermano grande.

Sumar una seccion nueva que use el boton es sumar su clase a la lista de `:is()`.

Los 418px de Figma no cuadran con el copy, y no se imponen

El nodo declara `designedHeight: 418px`, que en el detalle aparece como `vertical: contextual (hug)` y en el listado de la pagina como `vertical: fixed`. Medido, la seccion da **393px**:

```
128 padding (112 arriba + 16 abajo)
55 H1, lh 1em
24 gap de Content
108 parrafo, 4 lineas x 27px
32 gap of Component
46 Actions: 10 + 24 de linea + 10 + 1 + 1 de borde
---
393
```

Para que salieran 418 habria que medir 290 de contenido, y los valores fijos del stack son $55 + 24 + 32 + 46 = 157$. Quedan 133px para el parrafo, o sea $133 / 27 = 4.93$ lineas. No es un entero: con 4 lineas salen 393 y con 5 salen 420, y los 418 caen entre medio. Ningun numero entero de lineas reproduce los 418 con los text styles declarados, asi que el alto guardado quedo viejo respecto del copy — el texto del nodo ademas termina en un `\n`, que es la firma de un texto editado despues de que se midiera el frame. Y el nodo tampoco lleva los 418 con el padding real, que es 112/16 y no 112/112.

El `height` **no** se impone por eso: con el copy actual la seccion es 393 y queda correcta, y un `height:418px` fijo dejaria un hueco o cortaria el parrafo en cuanto el copy cambie. La altura la define el tipo y los gaps declarados.

Verificado (Chrome CDP)

	Figma	medido a 1440
seccion	1440, 112px 64px 16px , column, center , stretch , #FFF	1440, 112px 64px 16px , column, center , stretch , rgb(255,255,255)
Container	stretch, center , gap 80, max-width: 1280px	stretch, align-items: center , gap 80, 1280px de ancho , margin-inline: auto
Component	stretch, center , gap 32	stretch, align-items: center , gap 32
Content	stretch, gap 24	stretch, gap 24, 768px de ancho , margin-inline: auto
H1	55px / 500 / 1em / -0.01em / center	55px / 500 / 55px / -0.55px / center
parrafo	18px / 400 / 1.5em / center	18px / 400 / 27px / center
Actions	hug, gap 16	128x46, gap 16
boton	10/24, borde 1px #B69E52 , blanco, 500	10px 24px , 1px solid rgb(182,158,82) , blanco, 500

- **Todo centrado en 720**, que es $x + width/2$ de los cuatro: seccion, H1, parrafo y boton. El desvio real del boton es de **0.0078px** (1/128), porque su ancho de 128px se apoya en un centro de medio pixel; no es desalineacion. Con `Math.round` a los dos lados da 599 vs 600 y parece un error, asi que hay que medir sin redondear.
- **Regresion** en 1920 / 1440 / 1200 / 1199 / 1024 / 768 / 390: 112 64 16 fijo en los siete, **0 de overflow**, Container y Content centrados en los seis anchos de escritorio, navbar en 92 en los siete. A 390 el Content se sale 31.45px por la derecha, que es el problema preexistente del H1 de abajo, no del tope. Con el bloque de texto a 768 la seccion da **393 / 393 / 393 / 381 / 381 / 460 / 714**. El Container da 1280 / 1280 / 1072 / 1071 / 896 / 640 / 262 y el Content da **768 / 768 / 768 / 768 / 768 / 640 / 262**: los dos topes muerden por separado, el de 1280 de 1408 para arriba y el de 768 de 832 para arriba, y por debajo cada caja es exactamente `ancho - 128`. El parrafo pasa a 4 lineas de 1920 a 1024.
- **La aritmetica de la altura a 1440 cuadra con el gap del Component, no con los del Title**. Con el bloque a 768 y el bottom en 16 mide **393** = 112+16 de padding + 187 de Section Title + 32 de gap del Component + 46 de Actions. El 187 del Title es su unico hijo, el Content, que a su vez son 55 del H1 + 24 + 108 de

parrafo (4 lineas de 27). Antes, con el Content a 1280 y el bottom en 112, eran $462 = 224 + 160 + 32 + 46$, con $160 = 55 + 24 + 81$ (3 lineas). Los gaps de 80 del Container y del Section Title son inertes porque cada uno tiene un solo hijo, y el de 16 del Title tambien, porque dentro solo vive el Content. Con el paso de 27px por linea de parrafo y el padding de 128, 4 lineas dan 393 y 5 dan 420; con el padding de 224 de antes, 3 lineas daban 462.

- **La home no se movio**, y en particular sus botones primary siguen con `border: 0`, Spring-Wood y peso 300.
- Consola y requests limpios.
- Para traducir un valor de Figma a un atributo de bloque, `block-supports/layout.php` es el archivo que hay que abrir: los nombres de las claves no dicen que propiedad CSS producen, y en la rama de vertical estan invertidos respecto de la de horizontal.

Pendiente

- **Subir el boton a `.ocx-btn--primary`**. Los siete botones de la home tienen los mismos tres desvios que el navbar y el hero, y la regla compartida seria el unico lugar donde arreglarlos. **Se dejo sin hacer a proposito:** cambia una pagina ya terminada, y la decision es de quienarla. Si sale adelante, la regla `:is()` de arriba se borra y su `padding: 8px 20px` se queda como la unica excepcion, por ser `Small=True`.
- **Ojo al medir botones: la home tiene 15 `.wp-block-button`, no 7.** Inventario real, porque un `querySelector('.wp-block-button__link')` se lleva el primero del DOM, que es el Sign In **secondary**, y hace creer que los primary cambiaron. Los que si llevan `.ocx-btn--primary` en la home son **dos**: el del navbar, con `border: 0`, Spring-Wood y peso 300, y el del bloque `.ocx-c25__btn`, que **ya trae el tratamiento correcto de Figma** (`border: 1px #B69E52`, peso 500). Los otros 13 son secondary, invert, link y hero. `.ocx-c25__btn` es un tercer precedente de override con scope de bloque, junto al `:is()` del navbar/hero y al `.ocx-h62__actions`, asi que el patron "corregir por bloque en vez de tocar la base" ya esta establecido tres veces en el tema.
- **La regla `:is()` no se filtra a la home, confirmado elemento por elemento.** Ningun boton de `/ matchea .ocx-header--ecosystem .wp-block-button ni .ocx-h62__actions .wp-block-button, porque esas clases solo existen en el ecosistema.`
- **Los 112/64 fijos en movil** son una lectura de "no hay frame movil", no una decision de diseno. A 390px la seccion queda en 714px. Si aparece el frame movil, el cambio es una sola regla.
- **A 390px el H1 se come 31.45px del padding derecho, y es preexistente al `center`**. La palabra mas ancha del H1, "foundation.", mide **293.45px** a 55px, y la caja de contenido a 390px son 262px. Como item flex con `min-width: auto` no baja de su min-content, asi que se queda en 293.45 y desborda la caja por la derecha. Comprobado con A/B sobre el mismo DOM poniendole `align-items: stretch` a mano: los anchos y las posiciones son **identicos**, de modo que no lo causa el `center` del nodo sino el 55px fijo. **No hay overflow de pagina** (el `scrollWidth` da 390 exactos), porque $64 + 293.45 = 357.45$ entra de sobra en 390. Lo que se ve es que el H1 se acerca 31.45px de mas al borde derecho. Se arregla bajando el H1 en movil o con `min-width: 0`, no con el `align-items`. Ojo: `min-width: 0` deja la caja en 262 y la palabra igual desborda, porque una palabra no se parte.
- **El copy manda sobre los 418px.** Si hay que llegar a 418 de verdad, la palanca es el texto del parrafo (5 lineas darian 420), no un `height` fijo.
- **La pagina tiene otro Header / 62 /** mas abajo, el `#10344:1758`, que es el mismo nombre pero otro frame: `padding: 112px 64px 15px`, `justifyContent: flex-end` y alto hug. Cuando se llegue a ese, las clases `ocx-h62__*` estan ocupadas y hay que decidir si se reusan con un modificador o se crea `ocx-h62b__*`.
- **Quedan ocho bloques de los doce** sin montar. `Layout / 71 /` ya esta montado en el ecosistema con el prefijo `ocx-l71b__` (arriba), que es un nodo distinto del `#10218:67647` de la home. Sigue pendiente `Layout / 121 /`, el segundo Header / 62 / (`#10344:1758`, que **no** es este: trae `padding: 112px 64px 15px` y `justifyContent: flex-end`, asi que pide `ocx-h62b__*`), y el resto.

Layout / 71 / — el bloque de "Data Lakehouse" (id 10338:1742)

Arbol de cinco niveles, todos `column` salvo el tagline:

```
Layout / 71 / #10338:1742 padding 60px 64px, gap 80, jc center, ai center, #FFFFFF
Container      #10338:1743 gap 80, ai stretch
Component      #10338:1744 gap 80, ai stretch, jc flex-end
Column         #10338:1745 gap 16
  Tagline Wrapper #10338:1746 row, ai center, hug    -> NO se creo, ver abajo
  H2            #10338:1748 Heading/H2
Column         #10338:1749 gap 32
Content        #10338:1750 gap 32
Parrafo        #10338:1751 Text/Medium/Normal
```

El prefijo es `ocx-l71b__`, no `ocx-l71__`, y no es cosmetico. El bloque de la home (`#10218:67647`) tiene el `Component` en **fila** de dos columnas; este lo tiene en **columna** de dos columnas. `patterns.css` es global, asi que compartir prefijo cruzaria las dos implementaciones. Es el mismo motivo por el que quedo reservado `ocx-h62b__` para el segundo "Header / 62 /". Verificado: en la home `.ocx-l71__component` sigue computando `flex-direction: row` y no aparece ningun `.ocx-l71b__*`.

Los dos `center` del nodo salen de los dos atributos de `core`

A diferencia del hero, aqui el bloque trae `justify-content: center` y `align-items: center`, o sea los dos ejes, asi que hacen falta los dos atributos y cada uno va a su eje:

CSS del nodo	atributo	por
<code>align-items: center</code> (eje transversal)	<code>layout.justifyContent: center</code>	el nombre de core esta volteado
<code>justify-content: center</code> (eje principal)	<code>layout.verticalAlignment: center</code>	idem
<code>justify-content: flex-end</code> del Component	<code>layout.verticalAlignment: bottom</code>	<code>bottom</code> es el nombre de core para <code>flex-end</code>

Comprobado que `verticalAlignment` **si** se emite en los grupos flex de core: computa `justify-content: center` en la seccion y `flex-end` en el Component. Los tres son inertes en la practica porque el alto es `hug` y no hay espacio libre que repartir, pero quedan declarados porque es lo que dice el nodo.

El `background` no necesita regla, y el `padding` sí

El `Style` del nodo pide `background: var(--Color-Scheme-1-Background, #FFF)`, y `.ocx-section` ya lo trae, asi que no hay regla nueva para el fondo. El `padding` si hay que declararlo entero, porque el lateral va literal y no por token:

```
.ocx-section.ocx-l71b__section {
  padding: 60px 64px;
  align-items: center;
}
```

El (0,2,0) es para ganarle a `.ocx-section--stretch`, que pone `align-items: stretch` con (0,1,0).

Este bloque si es responsivo en la tipografia, y el `padding` no

A diferencia del hero, aca los 52px del H2 y los 18px del parrafo coinciden con `--Text-Size-Heading-2` y `--Text-Size-Text-Medium` en escritorio, asi que van **por token** y no como literales. Bajo 1200px el H2 pasa a 36px y el parrafo a 16px. El H1 del hero lleva literal porque a 55px no hay token que lo cubra.

El padding lateral va literal 64px , no con `var(--Page-Padding-padding-global)` . Ese token es 64px en escritorio pero **20px bajo 1200px**, y el hero de arriba usa 64px fijos en todos los anchos. Con el token, a 390 el hero tenia 262px de contenido y este bloque 350px: dos secciones contiguas con distinto padding lateral, que es justo lo que se pidio evitar. Con el literal los dos quedan en ancho – 128 en todo el movil, y no cambia nada de 1200px para arriba porque el valor de escritorio ya era 64. Si aparece el frame movil, el token es el camino de vuelta.

ancho	padding	Container	H2	parrafo	seccion
1920	60 / 64	1792	52px, 1 linea	18px, 2 lineas	356
1440	60 / 64	1312	52px, 1 linea	18px, 3 lineas	383
1200	60 / 64	1072	52px, 1 linea	18px, 3 lineas	383
1199	60 / 64	1071	36px, 1 linea	16px, 3 lineas	355
1024	60 / 64	896	36px, 1 linea	16px, 3 lineas	355
768	60 / 64	640	36px, 1 linea	16px, 5 lineas	446
390	60 / 64	262	36px, 3 lineas	16px, 10 lineas	653

La aritmetica a 1440: 60+60 de padding + 24 del tagline + 16 de gap + 62.4 del H2 + 80 de gap del Component + 81 del parrafo = **383**. Los gaps de 32 de la segunda columna y del Content son **inertes**: cada uno tiene un solo hijo. Igual el de 80 del Container.

El "Tagline Wrapper" no se creo, y el motivo esta reaprovechado

Tagline Wrapper (#10338:1746) es row, `alignItems: center` y `hug`, con un solo texto y sin gap, asi que no aporta nada visual. Es exactamente el caso que ya se resolvió así en "Layout / 121 /" y en "Layout / 59 /", donde el parrafo va directo como hijo de la columna. Se sigue el mismo criterio y el tagline queda como `core/paragraph` con el peso por clase (`.ocx-l71b__label`), que es `Heading/Tagline` : Manrope 600 a 16px con 150% de line height.

Para que el tagline quede hug como en el nodo, la columna NO lleva `align-items: stretch` . El `flex-start` que core pone en las columnas verticales deja que el parrafo se mida a su `max-content`, que para "Data Lakehouse" son 122px en vez de los 1312 de la caja. Medido: 122px en los siete anchos, y a 390 sigue siendo 122 contra una caja de 350, así que no desborda. Si se declarara `stretch`, el tagline se estiraría a lo ancho de la columna; con texto `LEFT` el renderizado seria identico, pero la caja dejaria de coincidir con el nodo.

El `align-items: stretch` si se declara en el Container y el Component porque ahí es lo que dice el nodo, aunque queda **anulado**: cada uno tiene un solo hijo con `align-self: stretch`, y el `align-self` del hijo siempre le gana al `align-items` del padre.

El `align-self: stretch` de los cuatro niveles intermedios es obligatorio

El `align-items: center` de la seccion encoge a los hijos salvo que lleven `align-self: stretch`, igual que en el hero. Se declara una sola vez para los cuatro:

```
.ocx-l71b__container, .ocx-l71b__component, .ocx-l71b__col, .ocx-l71b__content {
  align-self: stretch; width: 100%; min-width: 0;
}
```

Pendiente

- **Sin decisiones abiertas en este bloque.** Todo lo verificado en los siete anchos: 0 de overflow, el gap de 80 entre columnas en los siete, el tagline hug en los siete, consola y requests limpios, y el hero de arriba con padding 112 64 16 (393 / 393 / 381 / 460 / 714). El aire entre las dos secciones contiguas es de **76px**, del borde inferior del boton al superior del tagline, o sea 16 del bottom del hero + 60 del top de este bloque. Los dos sectores comparten fondo blanco, asi que el borde entre secciones es invisible y lo unico que se ve es ese aire.
- **El Style del nodo no se reprodujo como regla propia** porque `.ocx-section` ya cubre el `background`. Si alguien cambia `.ocx-section`, este bloque lo sigue porque hereda, no porque tenga regla propia.

Generado automáticamente desde BACKLOG.md el 28/09/2026 13:49 · OpenCX Block Theme